FROM SCRATCH

# BUILD A DeepSeek Model

Raj Dandekar
Rajat Dandekar
Sreedath Panat
Naman Dwivedi

MEAP

MANNING

# Build a DeepSeek Model (From Scratch)

# welcome

Thank you for purchasing the MEAP for *Build a DeepSeek Model (From Scratch)*.

The ideas for this book grew out of our YouTube series, "Vizuara's Build DeepSeek from Scratch," which launched in February 2025. The series showed a clear demand for hands-on, first-principles material, encouraging us to create this more structured and detailed written guide.

To get the most from this book, a foundation in machine learning and deep learning concepts is required. You should be comfortable with Python, be familiar with the basic operations in a framework like PyTorch, and have some exposure to the transformer architecture, even if you haven't implemented one yourself.

This book is a hands-on guide to the technical innovations that make the DeepSeek model family work. We chose DeepSeek because it marked a significant moment in open-source AI, demonstrating that an open model could achieve performance comparable to leading proprietary systems.

Our approach is to build the model's key components from scratch. The book is structured around a four-stage roadmap, covering the innovations in a logical order:

1. The foundational Key-Value (KV) Cache for efficient inference.
2. The core architectural components: Multi-Head Latent Attention (MLA) and Deepseek Mixture-of-Experts (MoE).
3. Advanced training techniques, including Multi-Token Prediction (MTP) and FP8 quantization.
4. Post-training methods like Reinforcement Learning (RL) and Knowledge Distillation.

The implementations are designed to be accessible. We will work with scaled-down versions that run on consumer hardware, so a supercomputer is

not necessary.

The goal is for you to gain both a theoretical understanding and the practical skills to implement these modern AI techniques. Your feedback during the MEAP process is valuable for improving the final book. Please use the liveBook discussion forum to post any questions, comments, or suggestions.

Let's begin.

â€”Raj Abhijit Dandekar, Rajat Dandekar, Sreedath Panat, Naman Dwivedi

**In this book**

# 1 Introduction to DeepSeek

## This chapter covers

- Why DeepSeek represents a turning point in open-source AI
- A high-level roadmap of the key innovations we will build throughout this book
- The book's structure, scope, and prerequisites

Large Language Models (LLMs) have transformed the technology landscape in recent years. We now live in a world where AI systems can carry on conversations, write code, draft essays, and even solve complex problems in ways that feel almost human. But what if you, a technically curious reader, could build one of these powerful AI models from scratch?

What if you could understand the inner workings of a state-of-the-art LLM by constructing it step by step with code and theory hand-in-hand? That is what we plan to teach you in this book.

We will understand the layers of a cutting-edge open-source LLM named DeepSeek, recreating its key innovations from the ground up. By the end, you will not only understand what makes DeepSeek unique but also how to implement those innovations yourself, gaining invaluable insights into modern AI development along the way.

We will begin by understanding why DeepSeek matters: how it emerged as a turning point in open-source AI and why we chose it as the focal point of our book. Following that, we will present a roadmap of DeepSeek’s core innovations – terms like Multi-Head Latent Attention, Mixture-of-Experts, Multi-Token Prediction, and 8-bit floating-point format (FP8) quantization may sound scary now, but we will introduce them in an accessible way and explain why they are so important.

We will then clarify the book structure, what you can expect to learn in this book (and what is outside our scope), ensuring our goals align with yours. Next, we will outline what you need to follow along, from background

knowledge to hardware and software requirements. Don't worry, you won’t need a supercomputer!

Before we dive in, a bit of context: this book was inspired by Vizuara’s YouTube series “Build DeepSeek from Scratch.” In that series, thousands of learners watched as we walked through coding each component of an LLM inspired by DeepSeek. Through this book, we have distilled our experience from academia and industry, as well as the hands-on lessons from our popular DeepSeek YouTube series, into a structured journey.

# 1.1 Why DeepSeek? A turning point in open-source AI

With so many language models out there, you might wonder: why focus on DeepSeek? What makes this model so special that we decided to write an entire book about building it? The short answer is that DeepSeek represents a turning point in open-source AI, demonstrating for the first time that an openly available model can rival the performance of the best proprietary models.

Figure 1.1 shows a simple interaction with the DeepSeek chat interface.

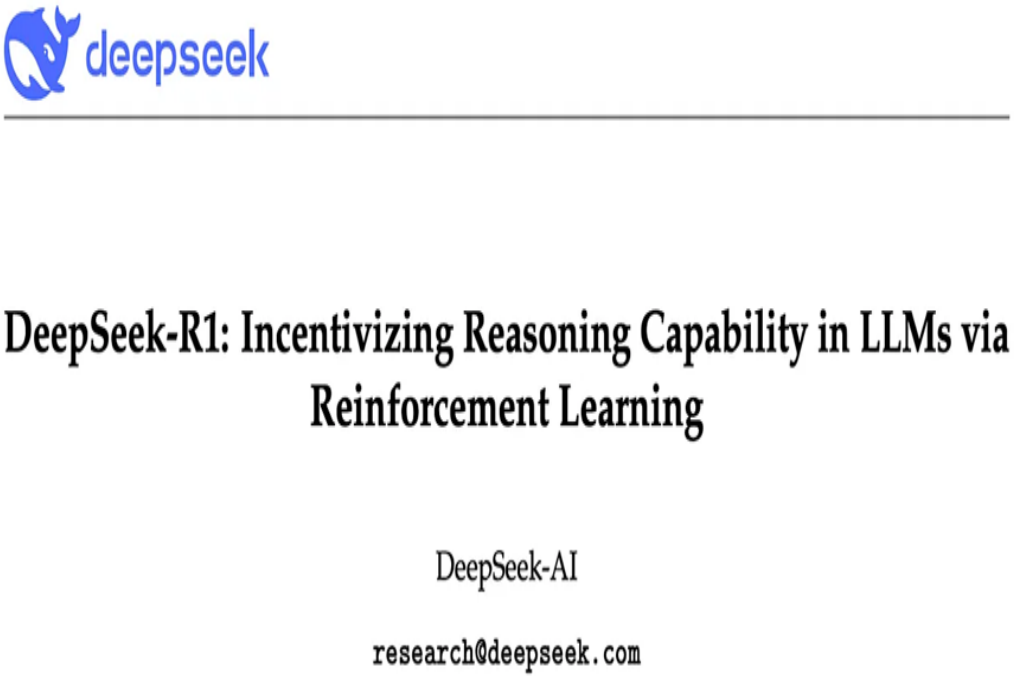
**Figure 1.1 A simple interaction with the DeepSeek chat interface.**


Let us step back for a moment and consider the state of AI before DeepSeek arrived. In the early 2020s, large language models were dominated by a few tech giants and research labs with vast resources. OpenAI's GPT series, Google's models like PaLM, and similar closed-source models led the field in capability, but they were (and remain) proprietary, often accessible only via paid APIs. The open-source community had successes like BERT and smaller GPT-style models, but there was a gap in performance. Open models tended to lag a generation behind closed models. Then, around 2023-2024, we saw a shift: Meta released LLaMA and later LLaMA-2 openly to researchers, and other organizations started emphasizing open science in AI.

DeepSeek emerged in this context, but it took openness to a new level, both in terms of releasing weights freely to the public and in terms of pushing technical boundaries. DeepSeek was founded in 2023 (as an AI lab based in China, led by researcher Liang Wenfeng), and within a short span, it made waves by open-sourcing extremely large LLMs with performance on par with the very best.

The significance of DeepSeek became clear with the release of its first major model, often referred to as DeepSeek-R1. A screenshot of the paper that introduced DeepSeek-R1 is shown in figure 1.2 (https://arxiv.org/pdf/2501.12948).

**Figure 1.2 The title and abstract of the DeepSeek-R1 research paper.**




This model immediately stunned the AI community. Despite being openly available, R1 demonstrated an intelligence level that rivaled top-tier models from giants like OpenAI and Google, effectively narrowing the gap between open-source and closed-source AI to the smallest it had ever been. At the

time of its release in early 2025, it was on par with or outperformed leading models like OpenAI's o1-1217 across a range of demanding reasoning benchmarks, including mathematical problem-solving (AIME 2024) and competitive coding (Codeforces).

This achievement effectively narrowed the gap between open-source and closed-source AI to the smallest it had ever been.

Another shocking announcement was that DeepSeek was trained at a fraction of the cost of leading OpenAI models. For us as learners and builders, DeepSeek-R1 is a perfect case study because its success comes from technical breakthroughs that we can understand. This raises several key questions that we will answer throughout this book:

- How did DeepSeek-R1 achieve state-of-the-art results with a fraction of the training cost?
- What was so novel in the DeepSeek-R1 architecture?
- What was so novel in the DeepSeek-R1 pre-training and post-training?

You will learn answers to all the above questions in the subsequent chapters. DeepSeek’s team openly published many of their methods, and we will leverage those insights in this book. By reproducing key elements of DeepSeek, we get to explore state-of-the-art techniques in practice. This includes the topics we previewed earlier: new forms of attention, new training objectives, massive model scaling strategies, and novel ways to compress models.

From a historical perspective, we can say that DeepSeek marked the moment when open-source AI truly went head-to-head with the tech giants and held its own. By replicating parts of DeepSeek, we effectively retrace the steps of some of the most advanced AI research of the day. This is immensely valuable if you aspire to work in AI research.

Finally, there is a philosophical reason: DeepSeek embodies the spirit of democratization of AI. Knowledge that was once confined is now shared. In writing this book, we align with that spirit. The authors of DeepSeek published technical reports detailing their methods, and we take it a step further by translating those into an approachable tutorial.

When you build something yourself, you own that knowledge in a way that reading a paper or using an API can’t provide. The hope is that by empowering more people to understand and build advanced models, we accelerate innovation and broaden the base of who can contribute to AI. Today it’s DeepSeek. Tomorrow, perhaps it will be you inventing the next big idea after having learned from this experience.

# 1.2 The key innovations we will build

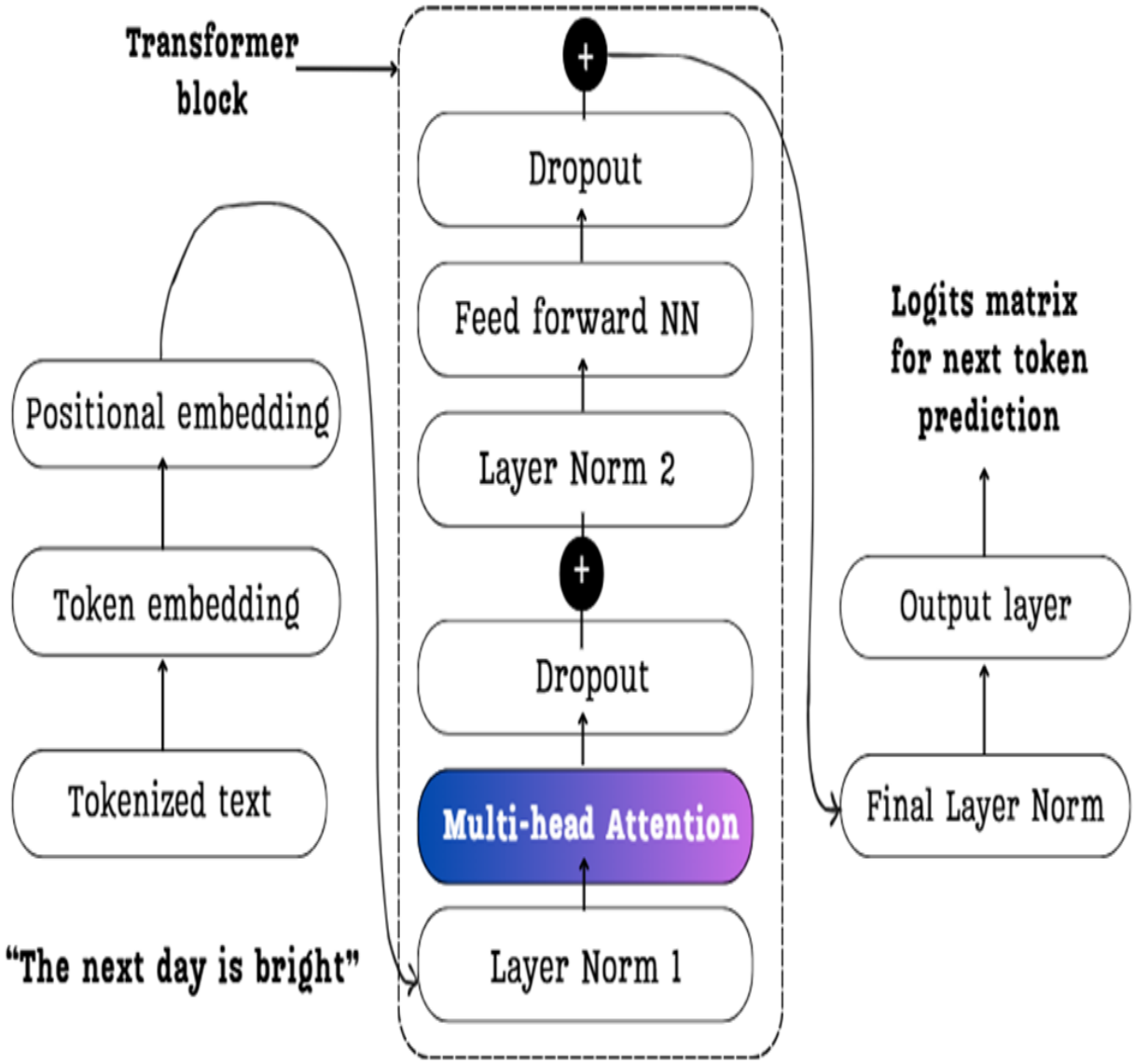
Now, let’s lay out the architectural roadmap for building DeepSeek from scratch, identifying the key innovations that differentiate DeepSeek from a standard Transformer-based language model. Understanding these specific components is crucial, as they are targeted solutions to fundamental bottlenecks in scaling language models. Each innovation from Multi-Head Latent Attention to Mixture-of-Experts addresses a distinct challenge related to computational complexity, memory bandwidth, or parameter scaling. By deconstructing and implementing these systems, we gain a first-principles understanding of the engineering trade-offs involved in modern LLM design. Our approach in this book will be to treat each innovation as a case study, first analyzing the limitations of the standard approach and then building its advanced replacement from the ground up

## 1.2.1 Architecture

DeepSeek's architecture builds upon the well-established Transformer foundation that powers models like GPT-3 and ChatGPT. However, it introduces significant innovations to overcome key performance bottlenecks. To understand what makes DeepSeek unique, we must first look at the standard Transformer building block.

The core of most modern LLMs is a stack of identical layers. Each layer consists of two primary sub-components: a multi-head self-attention mechanism, which allows the model to weigh the importance of different tokens in the input, and a feed-forward neural network, which processes the information further. Figure 1.4 shows a detailed view of this standard architecture..

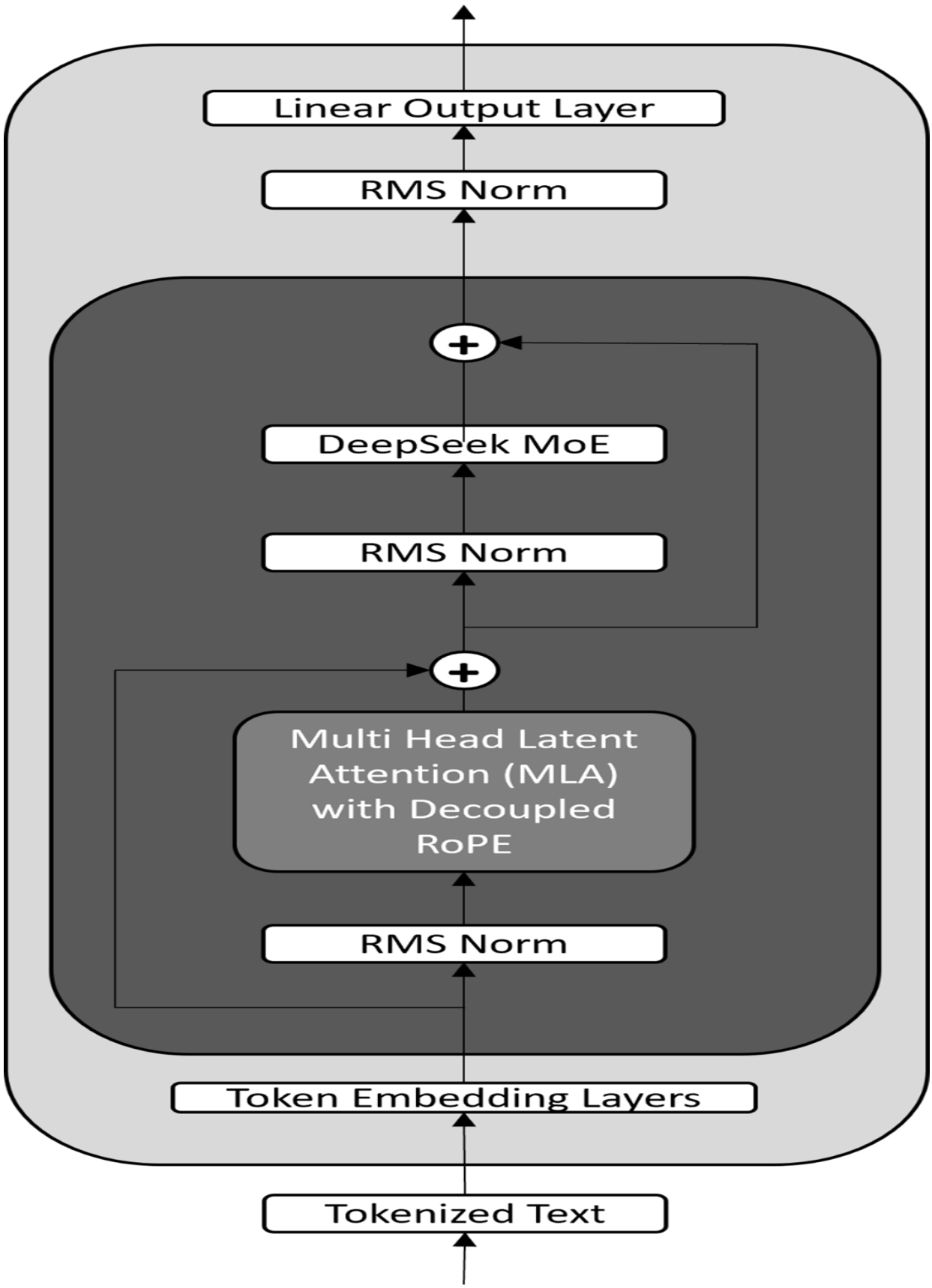
**Figure 1.3 A detailed view of a standard Transformer block, the foundational architecture used in models like LLaMA and the GPT series. It is composed of a multi-head attention block and a feed-forward network (NN).**




DeepSeek's key architectural innovation lies in replacing both of these standard sub-components with more efficient and powerful alternatives. As illustrated in Figure 1.4, the standard multi-head attention is replaced by **Multi-Head Latent Attention (MLA)**, and the feed-forward network is replaced by a **DeepSeek-Mixture-of-Experts (MoE)** structure.

**Figure 1.4 A simplified view of the DeepSeek model architecture. It modifies the standard Transformer by replacing the core components with Multi-Head Latent Attention (MLA) and a**

**Mixture-of-Experts (MoE) layer. This design also utilizes RMS Norm (Root Mean Square Normalization) and a specialized Decoupled RoPE (Rotary Position Embedding).**


Linear Output Layer
RMS Norm
DeepSeek MoE
RMS Norm
Multi Head Latent Attention (MLA) with Decoupled RoPE
RMS Norm
Token Embedding Layers
Tokenized Text

These two architectural changes, MLA and DeepSeek-MoE, are targeted solutions to major challenges in scaling LLMs.

On top of these architectural changes, DeepSeek introduces advanced training and efficiency techniques. A new training objective called Multi-Token Prediction (MTP) improves learning and inference speed, while FP8 quantization (an 8-bit floating-point format) addresses computational efficiency and resource utilization.

Together, these four innovations MLA, MoE, MTP, and FP8 quantization form the pillars of DeepSeek's technical advancement. Each one is a targeted solution to a different fundamental challenge in scaling language models:quantization for inference) to push efficiency to the limit..

Each of these addresses a different problem area:

- MLA tackles the *speed and memory bottleneck* in attention for long sequences.
- MoE tackles the scaling and model capacity issue
- MTP improves *learning and inference speed* by predicting more than one token at a time
- FP8 quantization addresses the computational efficiency and resource utilization problem.

## 1.2.2 Training

Beyond the core architecture, DeepSeek also innovates in how the model is trained and refined. The training pipeline is carefully designed to make large-scale training as efficient as possible. For example, DeepSeek employs an optimized scheduling strategy (internally nicknamed DualPipe) that overlaps different training tasks to keep hardware utilization high. In practice, this means data loading, preprocessing, and neural network computations are coordinated so that the GPU is never idle. When one batch is being processed by the model, the next batch is being prepared in parallel.

**Figure 1.5 An illustration of the DualPipe training pipeline on a single device. By overlapping the forward pass (the initial blocks), backward pass (the hatched blocks), and combined computations, this scheduling strategy minimizes GPU idle time and maximizes hardware utilization during large-scale training.**

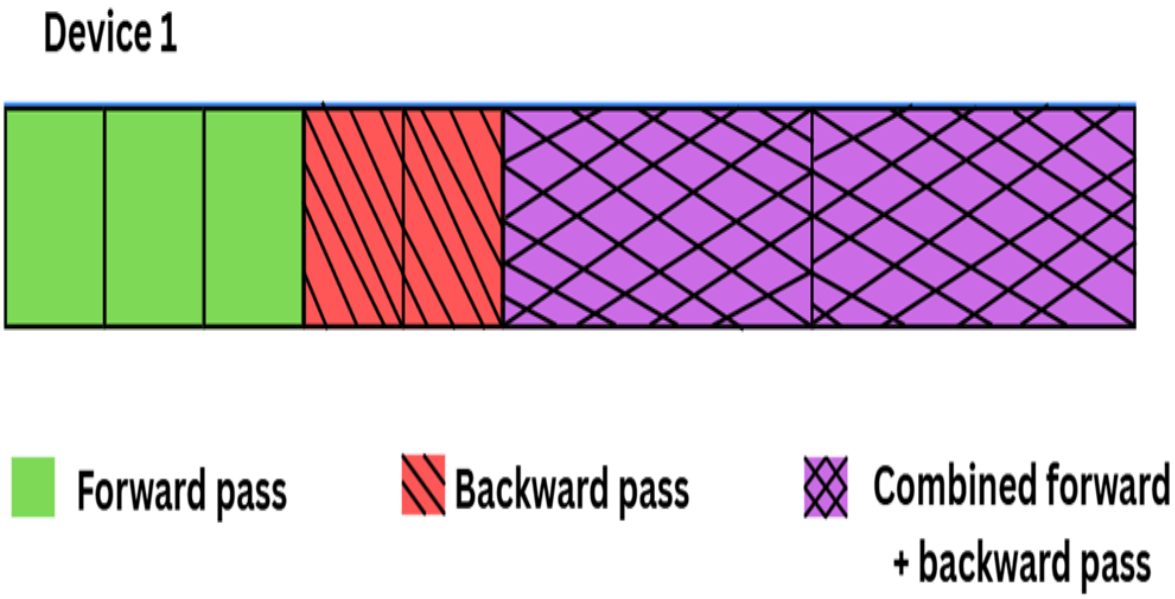
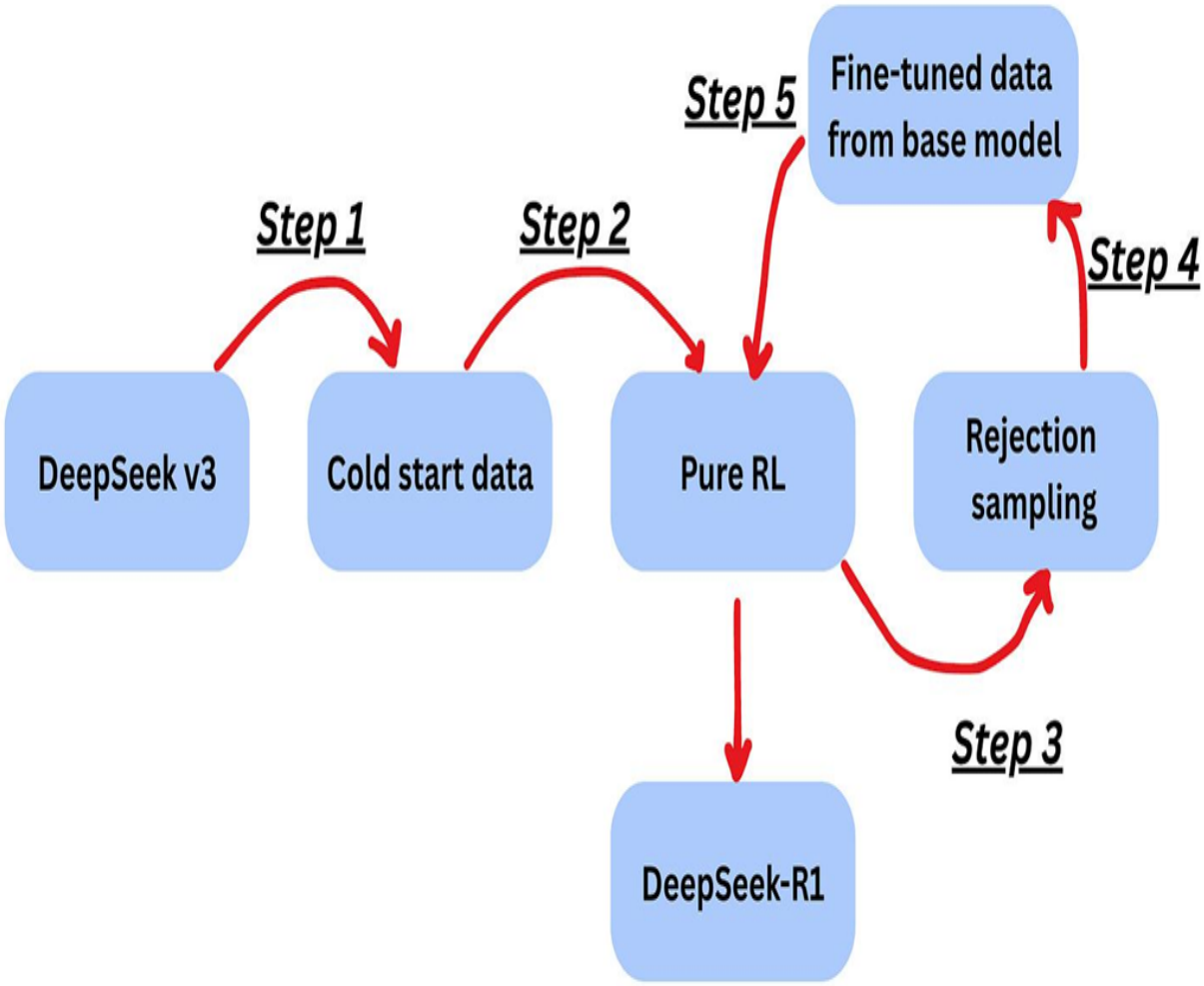



Figure 1.5 shows a timeline of Device 1 running tasks in a dual-pipe pipeline. The process begins with the forward pass, followed by the backward pass. Crucially, the pipeline then enters a steady state where the forward pass of a new batch is performed concurrently with the backward pass of the previous one (represented by the cross-hatched blocks). This overlapping ensures the device remains fully utilized throughout the training process.

### 1.2.3 Post-training

The base model, which was trained by DeepSeek, was called DeepSeek-v3. DeepSeek-v3 went through several post-training steps, ultimately resulting in DeepSeek-R1. These steps are shown in figure 1.6.

**Figure 1.6 The multi-step post-training pipeline used to create DeepSeek-R1 from the DeepSeek-V3 base model. This process involves a combination of reinforcement learning (Pure RL), data generation (Rejection sampling), and fine-tuning to instill advanced reasoning capabilities.**



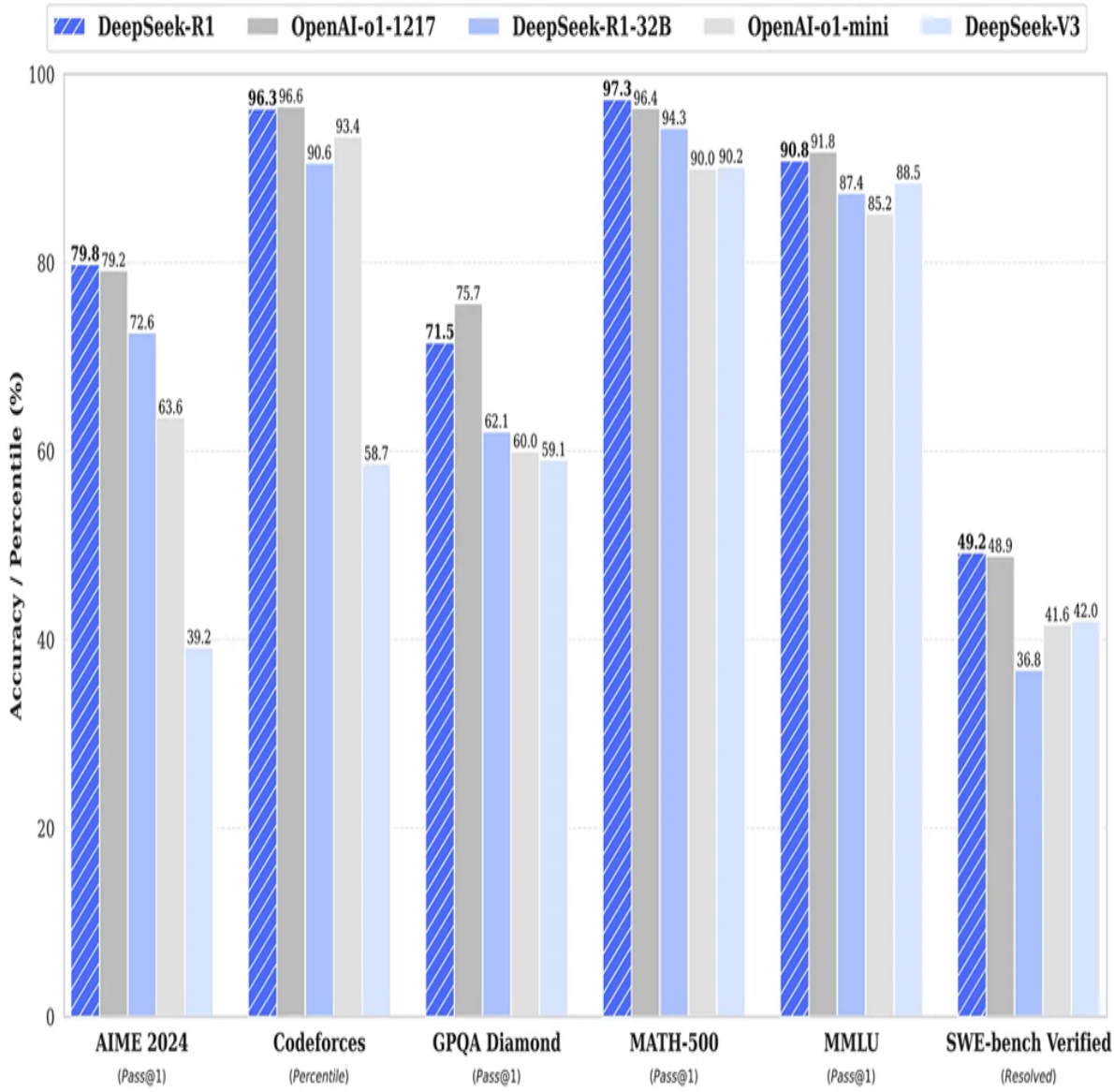

Below is a very simplified explanation of the five steps involved in DeepSeek R1 post-training:

- **Step 1 (Foundation):** Start with a lightly fine-tuned base (DeepSeek-V3) using a relatively small “cold-start” dataset.
- **Step 2 (Pure RL):** Implement reinforcement learning algorithms to allow the model to explore and develop reasoning patterns through trial-and-error learning. This unsupervised approach enabled the model to discover effective problem-solving strategies without explicit human guidance.
- **Step 3 (Self-Labeling):** Introduce rejection sampling techniques. The model generated multiple candidate responses and selected the highest-quality outputs to create its own synthetic training data.

- **Step 4 (Blending Data):** Merge this synthetic data with supervised examples to balance quality with domain breadth.
- **Step 5 (Final RL):** The training concluded with a comprehensive reinforcement learning phase using diverse prompt distributions. This final optimization step enhanced the model's robustness and generalization across various task categories and input formats.

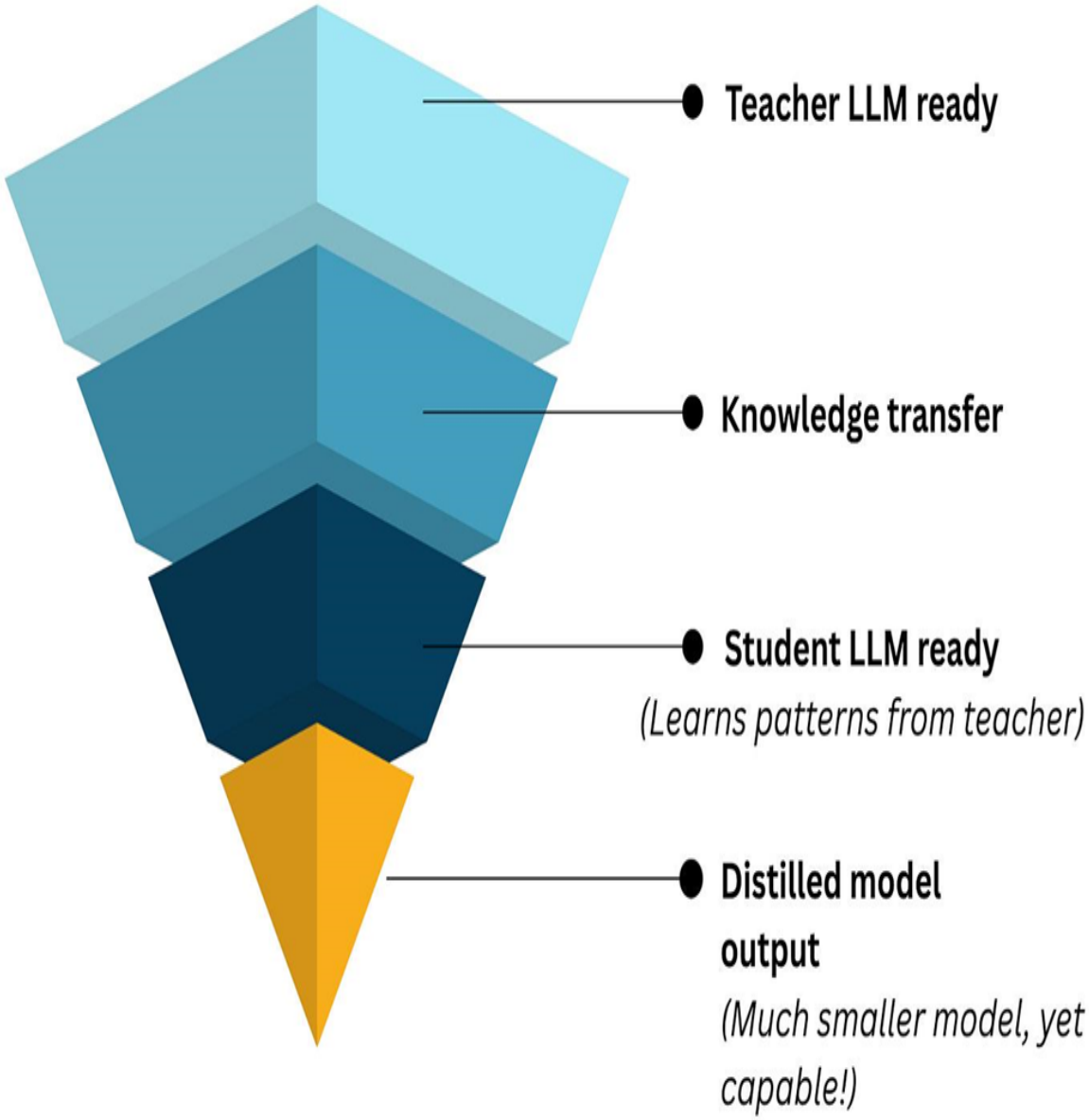
Figure 1.7 is taken from the DeepSeek-R1 paper, which was released in January 2025. Here we can see that DeepSeek-R1 is on par with or performs better than OpenAI reasoning models on several benchmarks that were tested.

**Figure 1.7 Benchmark performance of DeepSeek-R1 against other leading models (as of January 2025).**




Another important post-training technique is knowledge distillation and model compression. The idea is to take the large “teacher” model (the full DeepSeek) and compress its knowledge into one or several smaller, more practical “student” models. This has been shown in figure 1.8.

**Figure 1.8 The concept of knowledge distillation. A large, powerful "teacher" model (like DeepSeek-R1) is used to generate training data to teach a much smaller, more efficient "student" model, transferring its capabilities without the high computational cost.**




The DeepSeek R1 model was based on DeepSeek v3, which had around 671 billion parameters. When the DeepSeek paper was released, the team also released distilled models that were as low as 1.5 billion parameters in size. In particular, DeepSeek open-sourced 1.5B, 7B, 8B, 14B, 32B, and 70B checkpoints based on Qwen2.5 and Llama3 series to the community. These smaller models are highly performant and efficient.

In summary, DeepSeek's roadmap consists of innovations at multiple levels:

- Novel architectural components (MLA and MoE)

- A smarter training objective (MTP) with cutting-edge precision techniques (FP8)
- An efficient large-scale training pipeline (overlapping computations and communications, etc.)
- Post-training (RL-based reasoning skills and model distillation).

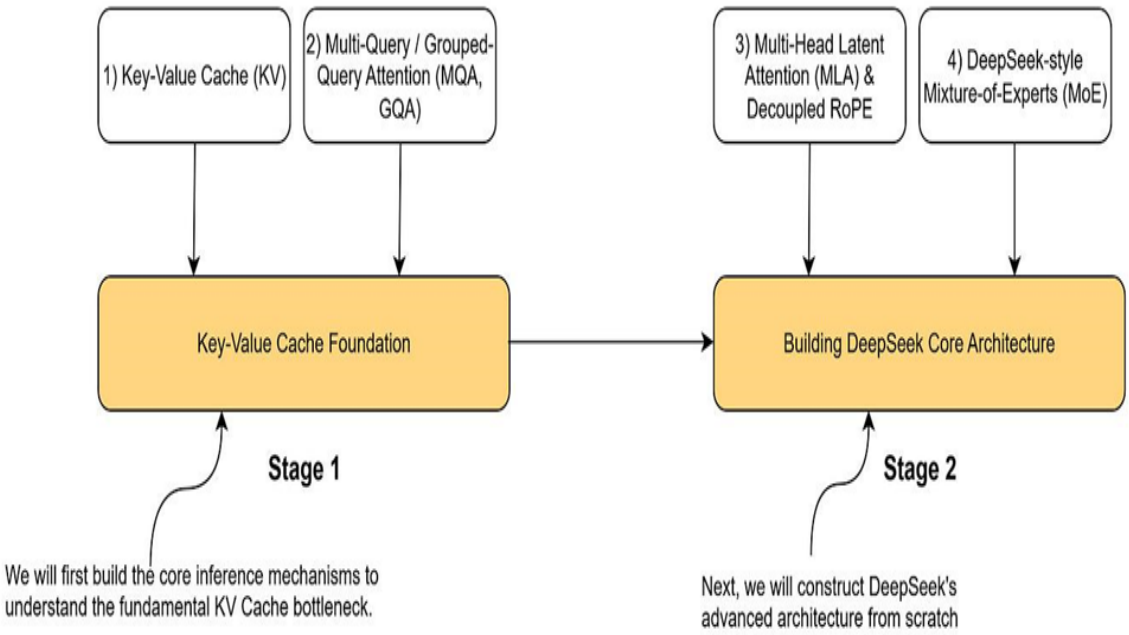
In the coming chapters, we will build each of these components step by step and demonstrate how they come together into a cohesive *mini-DeepSeek* model.

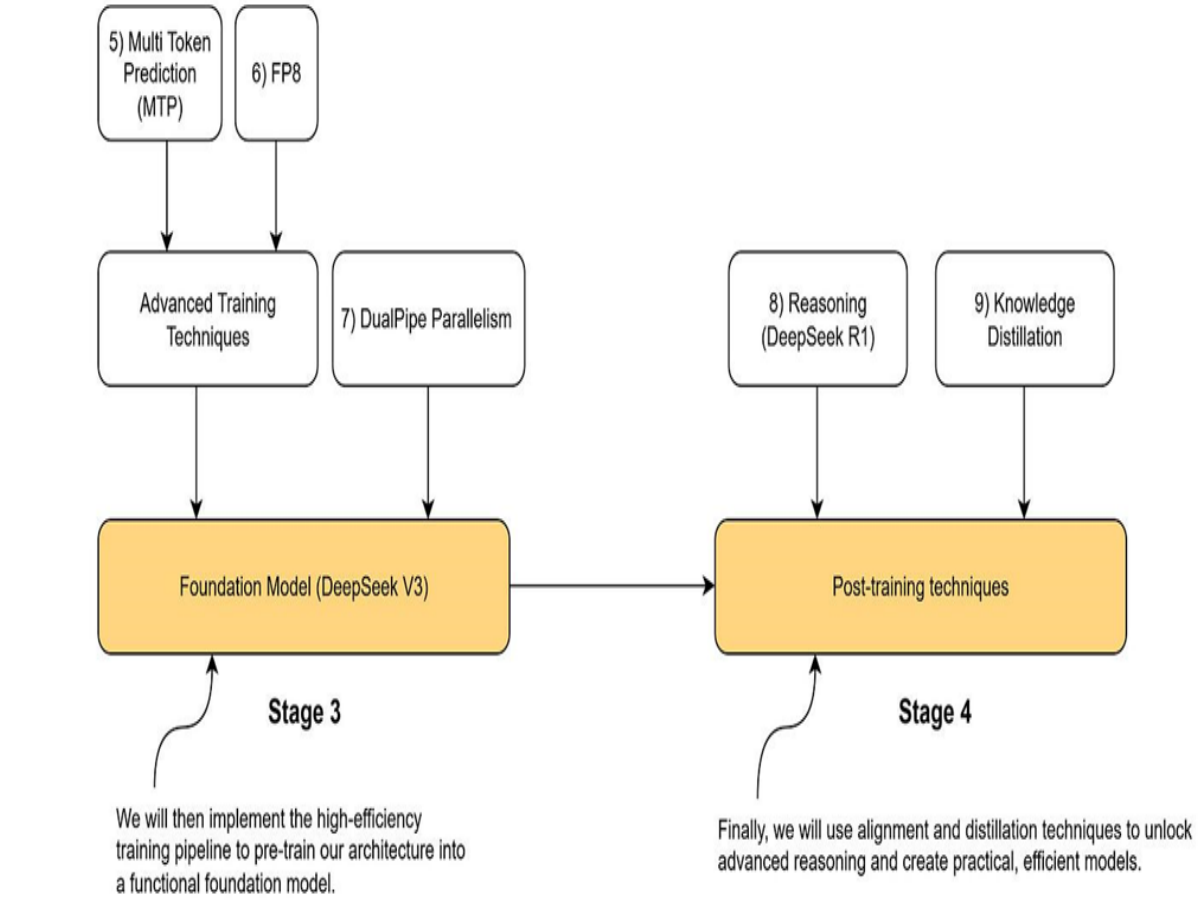
## 1.3 Book structure and scope

We have organized the book into a clear, four-stage roadmap. This structure is designed to be progressive, with each stage building directly upon the knowledge and code from the previous one. We will begin with the essential building blocks of modern LLM inference, move through the core architectural innovations of DeepSeek, and then explore the advanced training and post-training techniques that give the model its power.

Figure 1.9 provides a high-level overview of this entire process. It visualizes the four distinct stages and lists the key technical concepts we will implement within each one. Think of this as the master blueprint for our project and the table of contents for your learning journey.

**Figure 1.9 The four-stage roadmap for building a mini-DeepSeek model in this book. We will progress from foundational concepts (Stage 1) and core architecture (Stage 2) to advanced training (Stage 3) and post-training techniques (Stage 4), implementing each key innovation along the way.**


1) Key-Value Cache (KV)
2) Multi-Query / Grouped-Query Attention (MQA, GQA)
3) Multi-Head Latent Attention (MLA) & Decoupled RoPE
4) DeepSeek-style Mixture-of-Experts (MoE)
Key-Value Cache Foundation
Building DeepSeek Core Architecture
Stage 1
Stage 2
We will first build the core inference mechanisms to understand the fundamental KV Cache bottleneck.
Next, we will construct DeepSeek's advanced architecture from scratch


5) Multi Token Prediction (MTP)
6) FP8
Advanced Training Techniques
7) DualPipe Parallelism
8) Reasoning (DeepSeek R1)
9) Knowledge Distillation
Foundation Model (DeepSeek V3)
Post-training techniques
Stage 3
Stage 4
We will then implement the high-efficiency training pipeline to pre-train our architecture into a functional foundation model.
Finally, we will use alignment and distillation techniques to unlock advanced reasoning and create practical, efficient models.

Stages 1 and 2 are related to the architectural innovations in DeepSeek. Stage 3 is related to the training pipeline, and Stage 4 is related to the post-training pipeline.

In stage 1, we will understand what is meant by the Key-Value Cache (KV Cache) and why it is the foundational building block for ultimately understanding multi-head latent attention (MLA), which is one of the key innovations in the DeepSeek architecture.

In stage 2, we will look at multi-head latent attention (MLA) and mixture of experts (MoE). We will visually understand how MLA and MoE work and also code them in practice.

In Stage 3, we will implement the DeepSeek training pipeline. In this stage, we will learn about:

1. Multi-token prediction (MTP)
2. FP8 quantization
3. Dual pipe parallelism

Finally, in stage 4, we will look at post-training techniques implemented by DeepSeek:

1. Supervised Fine-tuning
2. Reinforcement Learning (RL)
3. Model distillation

# 1.4 What this book will teach you and what it won’t

This book is designed as a hands-on journey through the architectural innovations that make DeepSeek possible. We believe that the best way to understand these complex systems is to build them yourself.

What you will learn in this book spans both theoretical understanding and practical implementation.

- You will discover how Multi-Head Latent Attention (MLA) dramatically reduces memory requirements while maintaining model quality, implementing the mechanism that allows DeepSeek to run efficiently on hardware that would struggle with traditional transformers.
- You will master the intricacies of Mixture-of-Experts (MoE) architectures, understanding how to route different tokens to specialized sub-networks and balance computational load across experts.
- Through Multi-Token Prediction (MTP), you'll see how predicting multiple future tokens simultaneously can accelerate both training and inference.
- And with FP8 quantization, you'll learn to compress model weights and activations to just eight bits while preserving the model's capabilities.
- You will also learn about how DeepSeek pre-trained their model.
- Finally, you will learn the details regarding the post-training techniques that DeepSeek implemented, which include Reinforcement Learning (RL) and Distillation.

What this book won't do is reproduce DeepSeek's proprietary training data or attempt to replicate their exact model weights. We will not dive into the massive-scale distributed training infrastructure required to train models with hundreds of billions of parameters—that would require resources beyond what most readers have access to. We also will not cover production deployment concerns like serving models to millions of users or implementing safety filters and content moderation systems.

Instead, we focus on clarity and comprehension. Every concept is introduced with minimal assumptions about prior knowledge, building up from first principles. For example, when we implement MLA, we will start with standard attention, understand its limitations, and then gradually transform it into the latent version. This approach means that you will not only know how to implement these techniques but also understand them deeply enough to modify and improve upon them.

## 1.5 What you will need to follow along

Following along with this book requires a foundation in machine learning concepts, but not expertise. If you are comfortable with Python and have worked through introductory deep learning materials, you are ready to begin. Specifically, you should understand how neural networks learn through backpropagation, be familiar with the basic operations in PyTorch or a similar framework, and have some exposure to the transformer architecture, even if you haven't implemented one yourself.

In terms of hardware, we have designed all the implementations to be accessible. While training large language models typically requires massive computational resources, we will work with scaled-down versions that capture the essential ideas while remaining tractable on consumer hardware.

A laptop with a decent CPU can run most of the examples, though they will train slowly. A single consumer GPU with 8-12GB of VRAM will make the experience much smoother, allowing you to experiment more freely and see results faster. For the more ambitious experiments, particularly when working with the MoE architectures, having 24-48GB of VRAM opens up additional possibilities, though it's not required.

We will provide complete environment specifications for each chapter, ensuring you can reproduce our results exactly. We will include configurations for Google Colab and similar platforms, making it possible to follow along without any local setup. While DeepSeek was trained on trillions of tokens, we will use smaller datasets that still allow us to observe the key phenomena while keeping training times reasonable.

Let us build something amazing together!

## 1.6 Summary

- Large Language Models (LLMs) have become a dominant force in technology, but the knowledge to build them has often been confined to a few large labs.
- DeepSeek marked a pivotal moment by releasing open-source models with performance that rivaled the best proprietary systems, demonstrating that cutting-edge AI could be developed and shared openly.
- This book will guide you through a hands-on process of building a mini-DeepSeek model, focusing on its key technical innovations to provide a deep, practical understanding of modern LLM architecture and training.
- The core innovations we will implement are divided into four stages: (1) KV Cache Foundation, (2) Core Architecture (MLA & MoE), (3) Advanced Training Techniques (MTP & FP8), and (4) Post-training (RL & Distillation).
- By building these components yourself, you will gain not just theoretical knowledge but also the practical skills to implement and adapt state-of-the-art AI techniques.

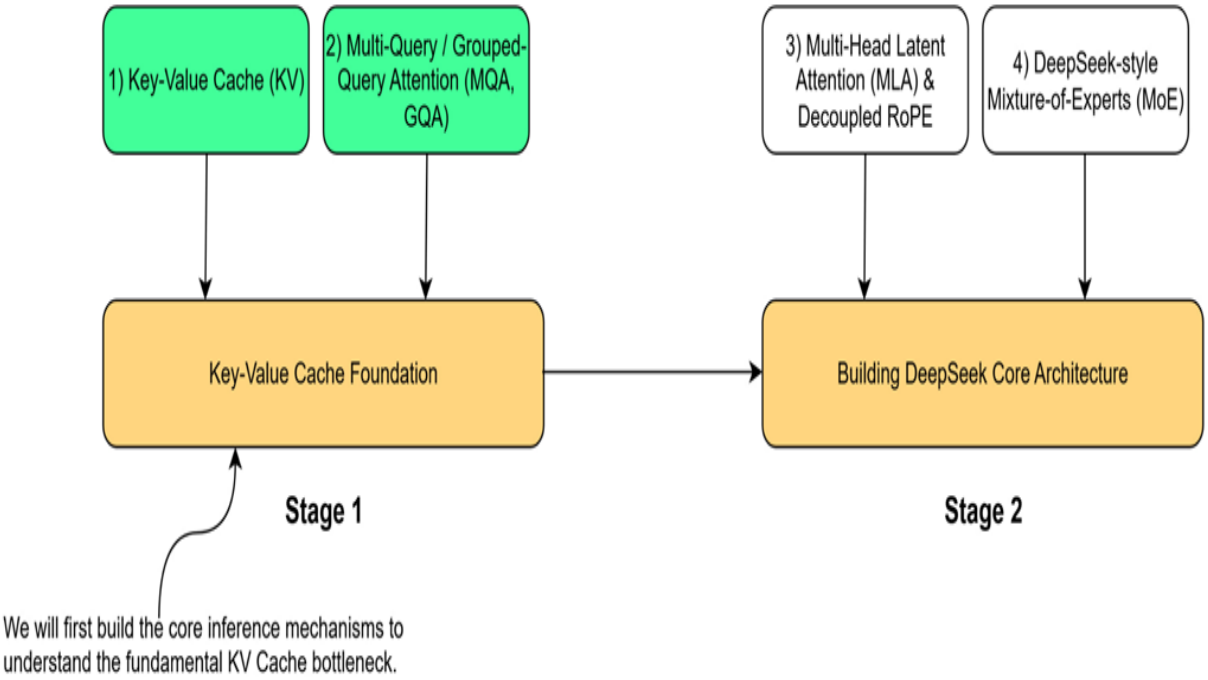
# 2 Solving the inference bottleneck with the key-value cache

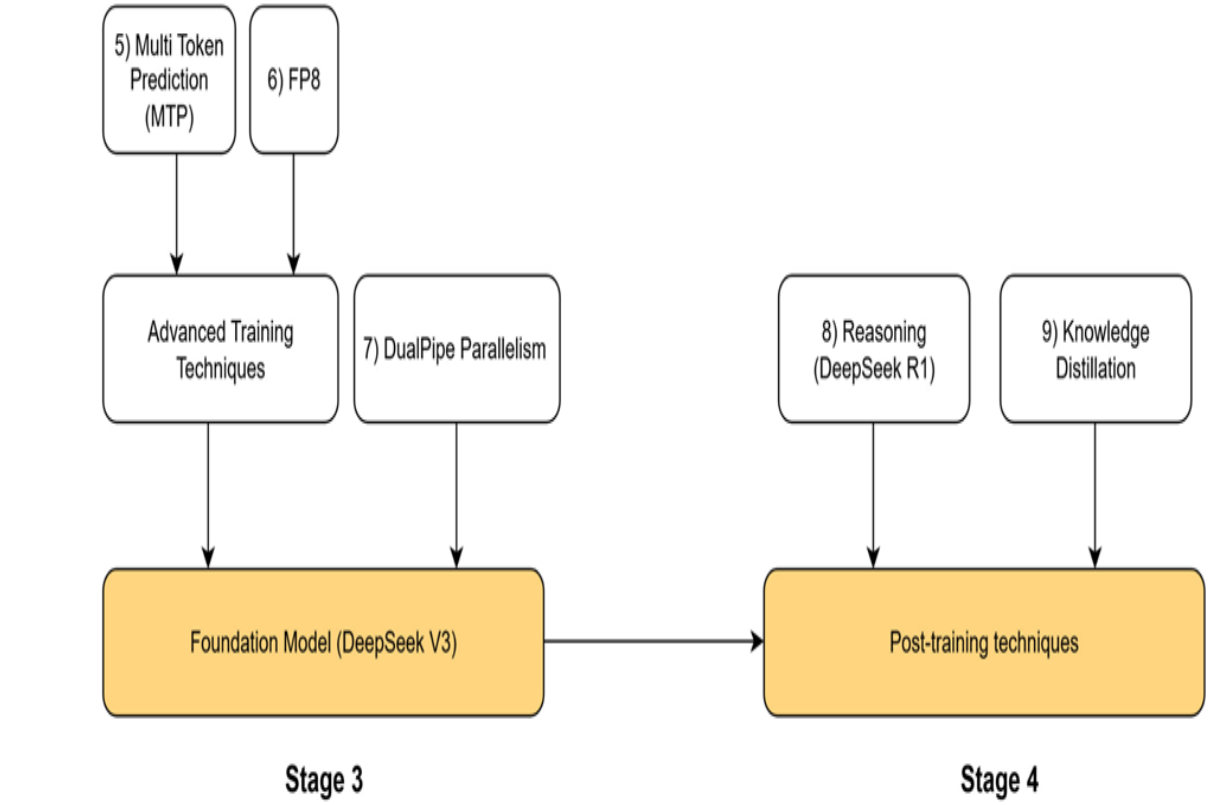
## This chapter covers

- The inefficiency of autoregressive LLM inference
- The Key-Value Cache: a solution with a cost
- MQA and GQA: First-Gen Solutions to KV Cache Memory Limits

To understand the key innovations in the DeepSeek architecture, we must begin with the technical problem they were designed to address. Our journey follows the four-stage roadmap outlined at the start of this book, and this chapter is dedicated entirely to Stage 1: The Key-Value Cache Foundation. This stage addresses the most fundamental bottleneck in modern LLM inference. Before we can appreciate advanced architectural choices like DeepSeek's Multi-Head Latent Attention (MLA) in Stage 2, we must first master the mechanisms it evolved from and the problems it was designed to solve.

**Figure 2.1 The four-stage roadmap for building the DeepSeek model. Stage 1 establishes the Key-Value Cache Foundation which we will cover in this chapter.**


1) Key-Value Cache (KV)
2) Multi-Query / Grouped-Query Attention (MQA, GQA)
3) Multi-Head Latent Attention (MLA) & Decoupled RoPE
4) DeepSeek-style Mixture-of-Experts (MoE)
Key-Value Cache Foundation
Building DeepSeek Core Architecture
Stage 1
Stage 2
We will first build the core inference mechanisms to understand the fundamental KV Cache bottleneck.


5) Multi Token Prediction (MTP)
6) FP8
Advanced Training Techniques
7) DualPipe Parallelism
8) Reasoning (DeepSeek R1)
9) Knowledge Distillation
Foundation Model (DeepSeek V3)
Post-training techniques
Stage 3
Stage 4

As the roadmap shows, this foundation is built on two core concepts: the Key-Value (KV) Cache itself and its first-generation optimizations, Multi-Query and Grouped-Query Attention (MQA & GQA). These techniques form the bedrock upon which more advanced architectures are built. Stage 2 of our roadmap introduces the core architectural innovations from DeepSeek-V2: Multi-Head Latent Attention (MLA), Decoupled RoPE and DeepSeek- Mixture-of-Experts (MoE). Before we can tackle those, we must first build a foundational understanding of the problems they were designed to solve. This chapter is divided into three parts to build this foundational understanding from the ground up:

First, we will code a complete autoregressive generation loop, visualizing how language models generate text one token at a time. This hands-on implementation will allow us to witness firsthand the computational inefficiency of the traditional approach.

Second, we will implement the KV Cache itself, the elegant optimization that solves this initial performance problem. Through our code, we will demonstrate its dramatic speedup, but also uncover its "dark side": a massive memory cost that creates a new, severe bottleneck.

Finally, we will build functional PyTorch layers for Multi-Query Attention (MQA) and Grouped-Query Attention (GQA). These are the first-generation architectural solutions designed to mitigate the KV Cache's memory problem. It's important to understand that these solutions are not a free lunch; both MQA and GQA explicitly trade off model quality and expressivity for gains in memory efficiency and inference speed. MQA represents an extreme on this spectrum, prioritizing memory savings above all, while GQA offers a more balanced compromise. By constructing these industry-standard techniques and understanding their inherent trade-offs, we will have the complete context needed to tackle DeepSeek's unique innovations, which aim to achieve the best of both worlds, in the chapters to come.

# 2.1 The LLM inference loop: Generating text one token at a time

The first and most important concept to grasp is that the KV cache is only relevant during the inference stage of a language model. This distinction is critical, so let's clarify the two main phases of an LLM's life.

### 2.1.1 Distinguishing pre-training from inference

Every large language model, from GPT-2 to DeepSeek-R1, goes through two distinct phases:

1. Training: This is the massive, computationally expensive learning phase. The model is trained on a vast dataset (trillions of tokens) to learn grammar, facts, reasoning patterns, and the statistical relationships between words. During this phase, its parameters (or weights) are adjusted. Once pre-training is complete, the model's parameters are fixed, resulting in a pre-trained LLM.
2. Inference: This is the "usage" phase. The pre-trained model, with its fixed parameters, is now used to perform tasks. When you interact with ChatGPT or use an API to ask a model to "make a travel plan for Italy," you are performing inference. The model isn't learning anymore; it's using its learned knowledge to predict the next token in a sequence.

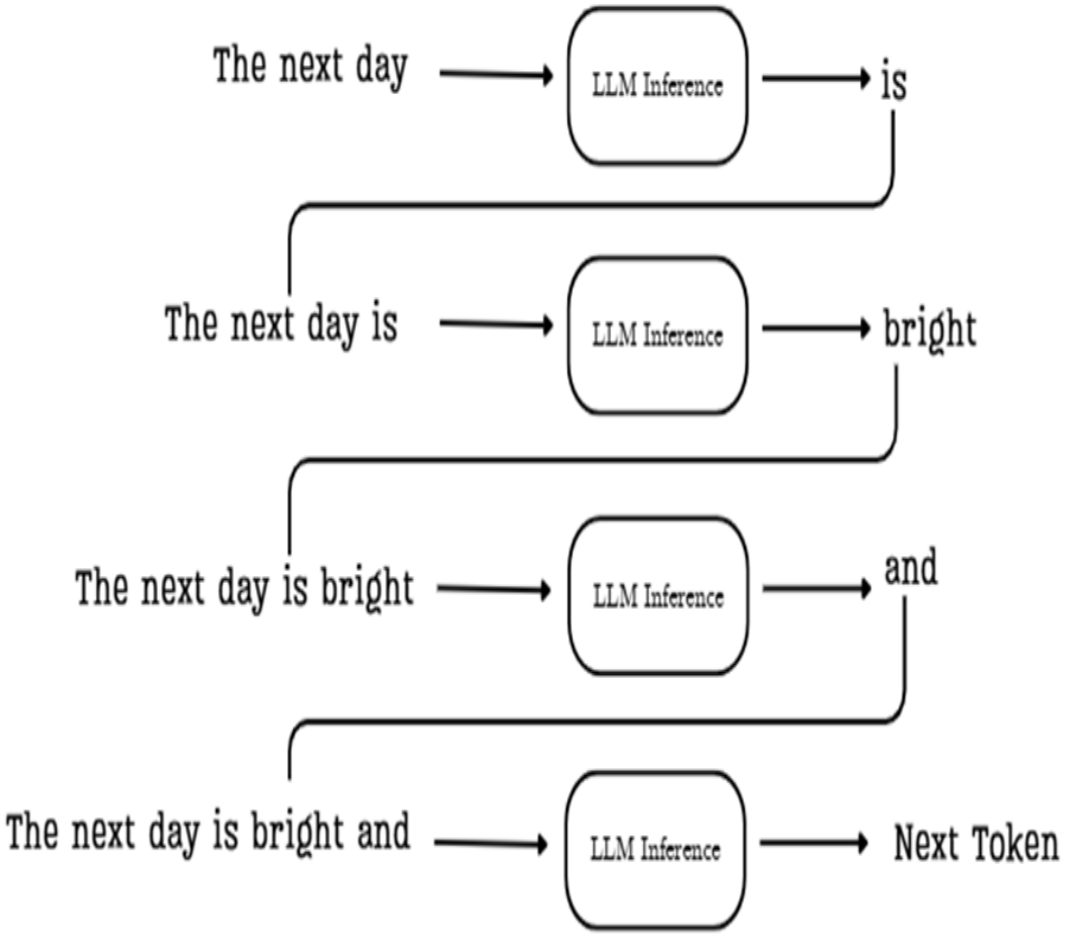
The entire discussion in this chapter applies exclusively to the inference stage. We assume we have a fully trained model, and our goal is simply to use it to generate text.

### 2.1.2 The autoregressive process: Appending tokens to build context

During inference, a language model generates text one token at a time. While a user interface like ChatGPT might make it seem like the entire response appears at once, underneath the hood, a methodical, step-by-step process is unfolding. This is called autoregressive generation.

The core idea is simple but powerful: each new token the model generates is immediately added back to the input sequence, becoming part of the context for generating the next token. This creates a feedback loop that allows the model to build coherent and contextually relevant text.

**Figure 2.2 In the autoregressive generation loop, the model's output from one step is appended to the input for the next, progressively extending the context.**




Let's trace the flow shown in the diagram:

“The next day.”.

The model's task is to predict the most likely token to follow this sequence. Here's how the process works:

1. Initial Context: We start by providing the model with an initial prompt, such as "The next day."
2. First Prediction: This sequence is fed into the LLM inference pipeline, which processes the context and predicts the most probable next token,

in this case, "is."
3. Append and Repeat: The new token, "is," is appended to the sequence. The input for the next step is now the expanded context: "The next day is." This new, longer sequence is fed back into the model.
4. Second Prediction: The model now processes "The next day is" and predicts the next token, "bright." This process continues, with the context growing one token at a time.

This loop continues, with each newly generated token being added back to the input sequence for the next prediction step. The process stops when the model generates a special end-of-sequence token or reaches a predefined limit on the number of new tokens to generate.

This iterative, feedback-driven process is fundamental to how autoregressive LLMs like the transformer construct coherent and contextually relevant text. Keep this flow in mind, as it's the key to understanding why the KV cache is both necessary and problematic within this architectural paradigm.

### 2.1.3 Visualizing autoregressive generation with GPT-2

The following listing demonstrates the autoregressive loop in action using the pre-trained GPT-2 model. The code starts with an initial prompt and then enters a loop. Inside this loop, it performs the core task: it passes the current sequence to the model, gets the prediction for the very next token, and immediately appends that new token to the sequence for the next iteration. This simple visualization makes it clear that the model is performing a full computational pass for every single new token it generates.

You can find this code, along with all other code listings in this book, in the official GitHub repository: [https://github.com/VizuaraAI/DeepSeek-From-Scratch](https://github.com/VizuaraAI/DeepSeek-From-Scratch).

**Listing 2.1 Visualizing autoregressive generation with GPT-2**

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import torch

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
```

```
model = GPT2LMHeadModel.from_pretrained("gpt2")

prompt = "The next day is bright"
inputs = tokenizer(prompt, return_tensors="pt")
input_ids = inputs.input_ids

print(f"Prompt: '{prompt}'", end="")

for _ in range(20):
    outputs = model(input_ids)  #A
    logits = outputs.logits

    next_token_logits = logits[:, -1, :]  #B
    next_token_id =
next_token_logits.argmax(dim=-1).unsqueeze(-1)

    input_ids = torch.cat([input_ids, next_token_id],
    ➥ dim=-1)  #C

    new_token = tokenizer.decode(next_token_id[0])
    print(new_token, end="", flush=True)

print("\n")
```

Running this code produces the following output, where the text after the initial prompt is generated token by token:

*'The next day is bright' and sunny, and the sun is shining. The sun is shining, and the moon is shining.*

This simple demonstration makes a crucial point clear: the model is performing a full computational pass through its architecture for every single new token it generates.

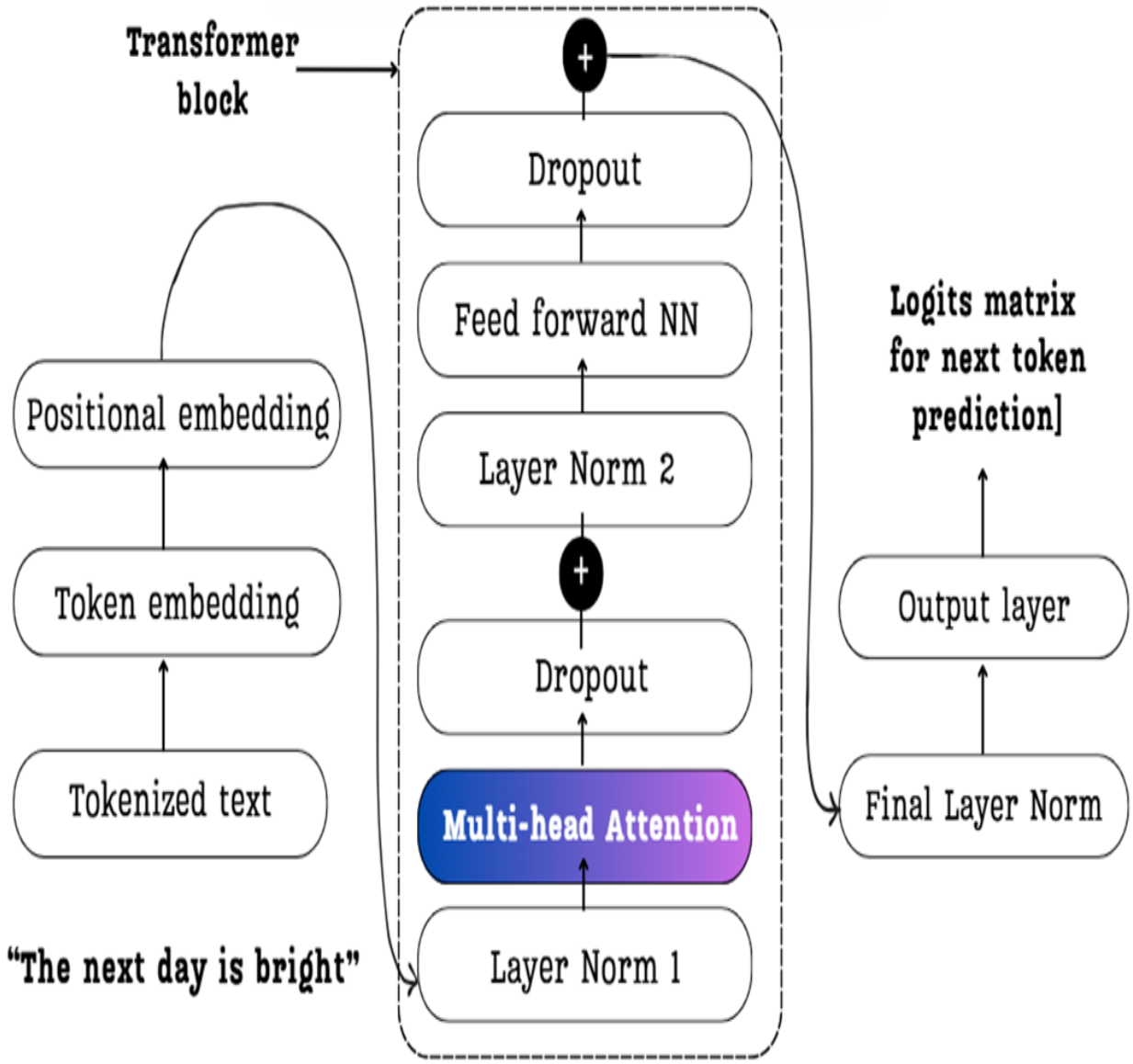
This is evident because the call to outputs = model(input_ids) happens inside the for loop, and the input_ids tensor, which contains the entire sequence so far, is passed to the model on every single iteration. This observation leads us to a critical question: what exactly is happening inside that computation, and is all of it truly necessary?

## 2.2 The core task: Predicting the next token

Now we know that the LLM performs a full computational pass for each new token. Let's peel back the layers of the architecture to understand what that computation entails. Let's focus on the heart of the Transformer block: the Multi-Head Attention mechanism. This is where the model figures out the relationships between tokens.

The diagram below provides a high-level map of this journey. It shows how our example, "The next day is bright," flows through the key components we will deconstruct in the coming sections. Keep this image in mind as it represents the entire process we are about to build.

**Figure 2.3 A high-level overview of the Transformer block architecture. This diagram illustrates the complete data flow from the initial input tokens ("The next day is bright") through embedding, multi-head attention, and feed-forward layers, ultimately resulting in the logits used for next-token prediction.**




### 2.2.1 From input embeddings to context vectors: A mathematical walkthrough

The diagram in Figure 2.3 showed us the major components, but to understand the computations we might be repeating during inference, we need to zoom in on the most critical component: the Multi-Head Attention block. This is where the model calculates the relationships between tokens and creates the enriched "context vectors" that form the basis of its understanding.

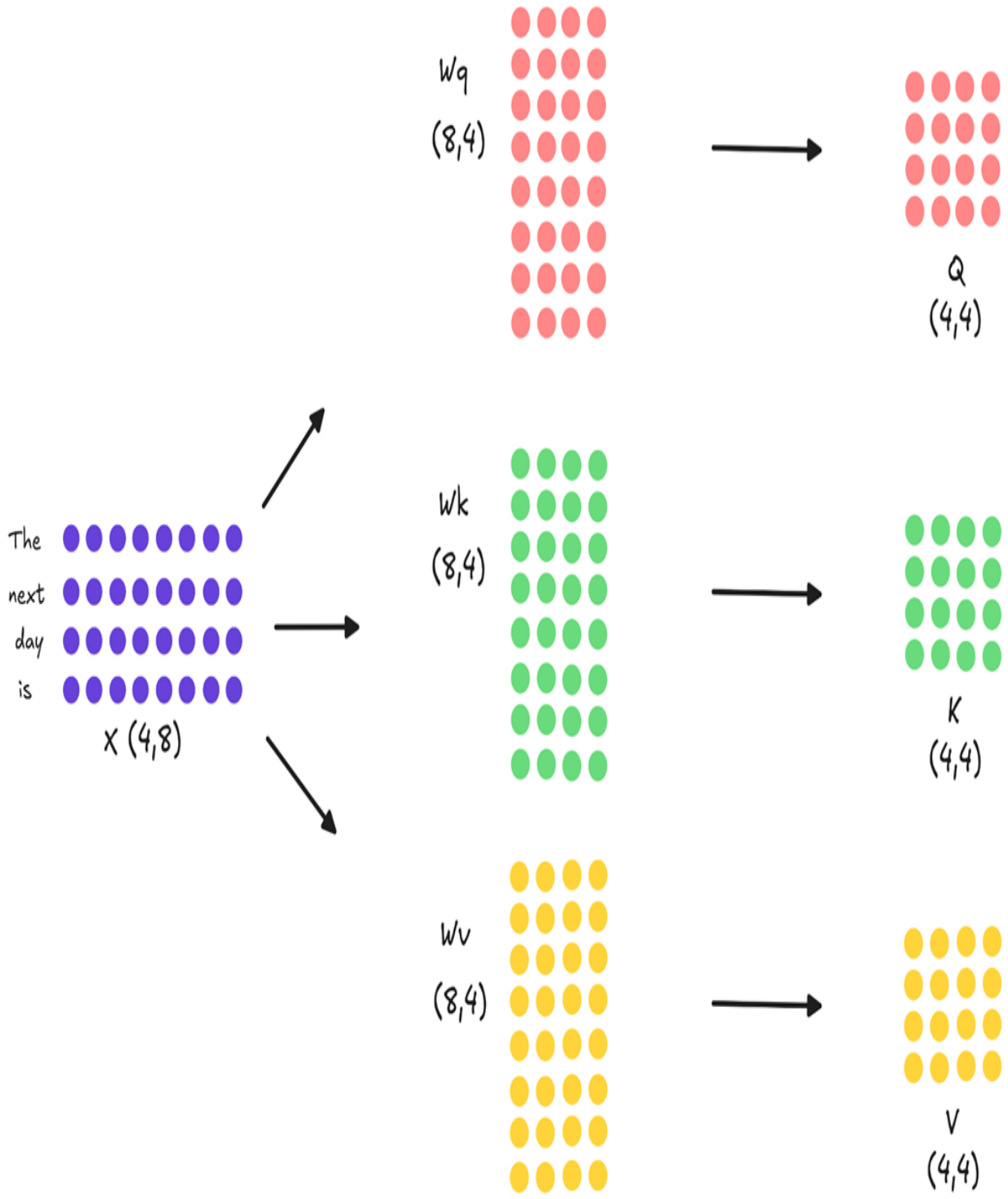
Let's trace the path of our input sequence, "The next day is," through a single attention calculation to see exactly what's happening under the hood.

Step 1: Projecting Inputs into Query, Key, and Value

After tokenization and embedding, our input is represented as a matrix, which we'll call X. For this walkthrough, we will use small, simplified dimensions to make the mathematics and diagrams easy to follow. Let's assume our matrix X has a shape of (4, 8), representing our four tokens, each with a 8-dimensional embedding. In a real model, this embedding dimension would be much larger, for example, 5120 in DeepSeek-V2 and 7168 in DeepSeek-V3, but the underlying mathematical principles remain identical.

The very first step within the attention block is to project this input matrix into three distinct representations: the Query (Q), Key (K), and Value (V) matrices. This is done by multiplying X with three separate, trainable weight matrices: Wq (for Query), Wk (for Key), and Wv (for Value).

**Figure 2.4 The input embedding matrix X is projected into three new matrices: Query, Key, and Value. Each projection is a matrix multiplication with a unique, learned weight matrix.**




As the diagram illustrates:

- The input X (shape 4x8) is multiplied by Wq (shape 8x4) to produce the Query matrix (shape 4x4).

- The input X (shape 4x8) is multiplied by Wk (shape 8x4) to produce the Key matrix (shape 4x4).
- The input X (shape 4x8) is multiplied by Wv (shape 8x4) to produce the Value matrix (shape 4x4).

These three new matrices represent our input tokens in different roles. The Query matrix represents what each token is "looking for," while the Key and Value matrices represent what each token "offers" as context.

Step 2: Calculating Attention Scores

Next, the model needs to determine how relevant each token is to every other token. This is done by calculating attention scores. We take the Query matrix and perform a matrix multiplication with the transpose of the Key matrix (Keys.T).

**Figure 2.5 The dot product between the Query and transposed Key matrices produces the Attention Scores matrix. Each element in this matrix represents the relevance of one token to another.**


The resulting Attention Scores matrix (shape 4x4) quantifies the relationship between every pair of tokens. For example, the value in the fourth row and second column would represent how much attention the token "is" should pay to the token "next."

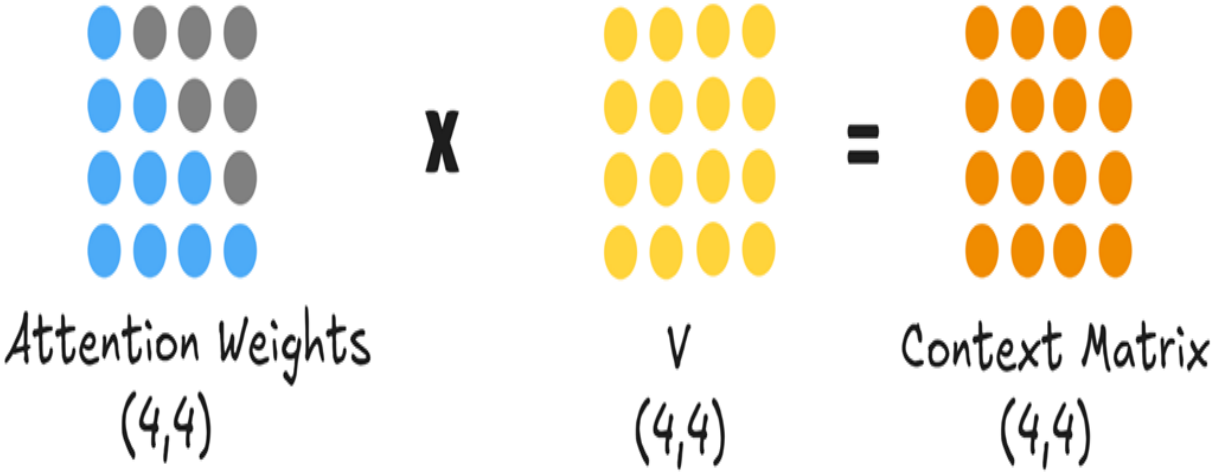
Step 3: From Scores to Context Vectors

These raw scores are then processed further. They are scaled (to stabilize training), and a causal mask is applied to ensure tokens can only gather context from previous tokens in the sequence, preventing it from "cheating" by looking ahead at tokens it is not supposed to know about yet. This mask

effectively zeros out the upper triangle of the attention scores matrix. Finally, a softmax function is applied to convert the scores into attention weights, a set of probabilities that sum to 1 for each row.

These final attention weights are then multiplied by the Value matrix.

**Figure 2.6 The Attention Weights are multiplied by the Value matrix to produce the final Context Matrix. Each row is now a context-aware representation of the original token.**



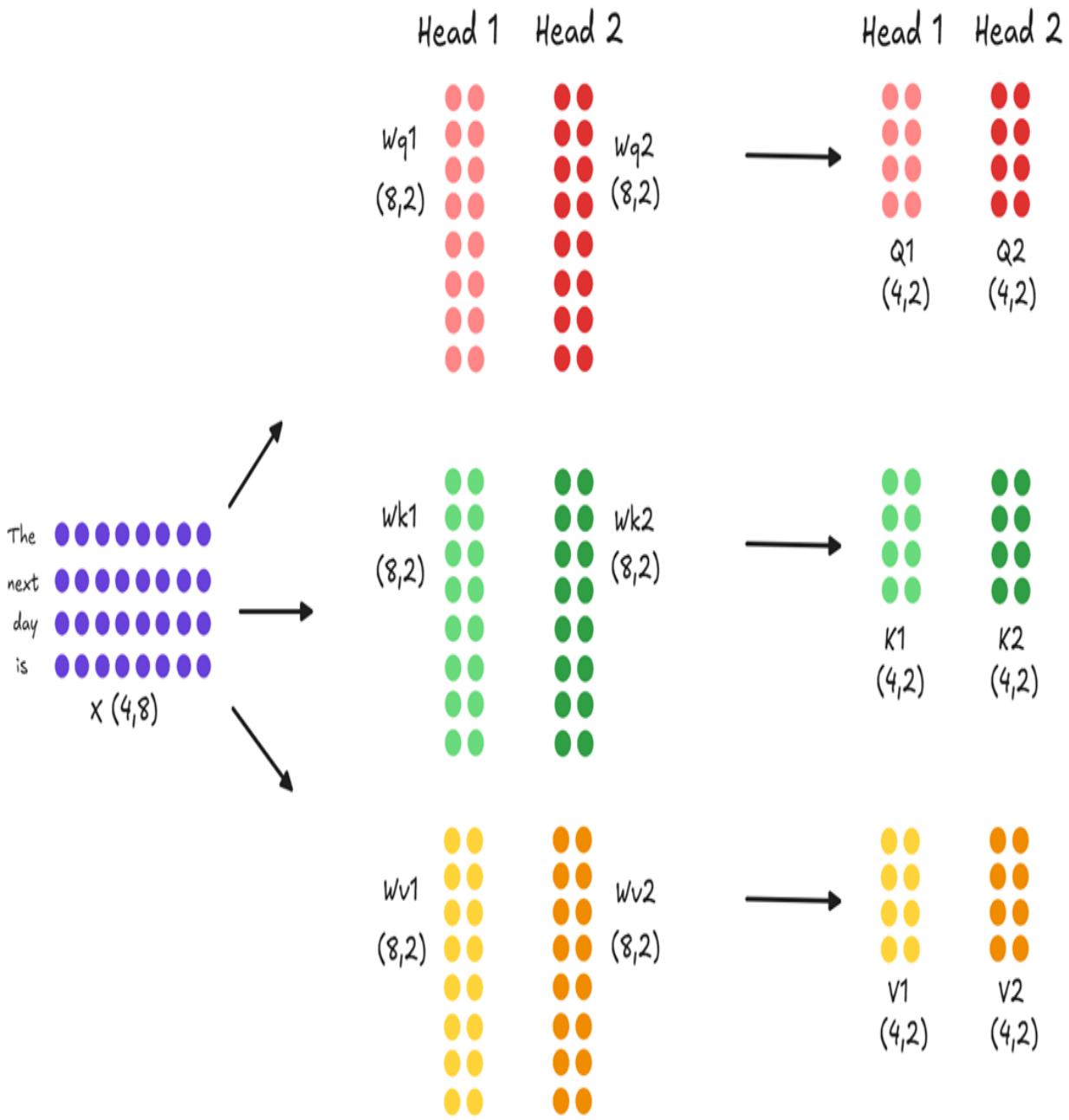

This final multiplication produces the Context Matrix (shape 4x4). Each row in this matrix is a new, enriched vector for each of our input tokens. The context vector for "is," for example, is now a weighted sum of all the Value vectors that came before it, containing rich information about the entire preceding sequence.

Step 4: Scaling to Multi-Head Attention

The process we've just described above is for a single attention calculation. However, a model might need to track different kinds of relationships simultaneously; for example, syntactic dependencies (like subject-verb agreement) and semantic relationships (like word meanings). A single attention calculation might struggle to capture this diversity.

This is where multi-head attention comes in. Instead of one large set of projection matrices (Wq, Wk, Wv), the model uses multiple, smaller, independent sets one for each "head."

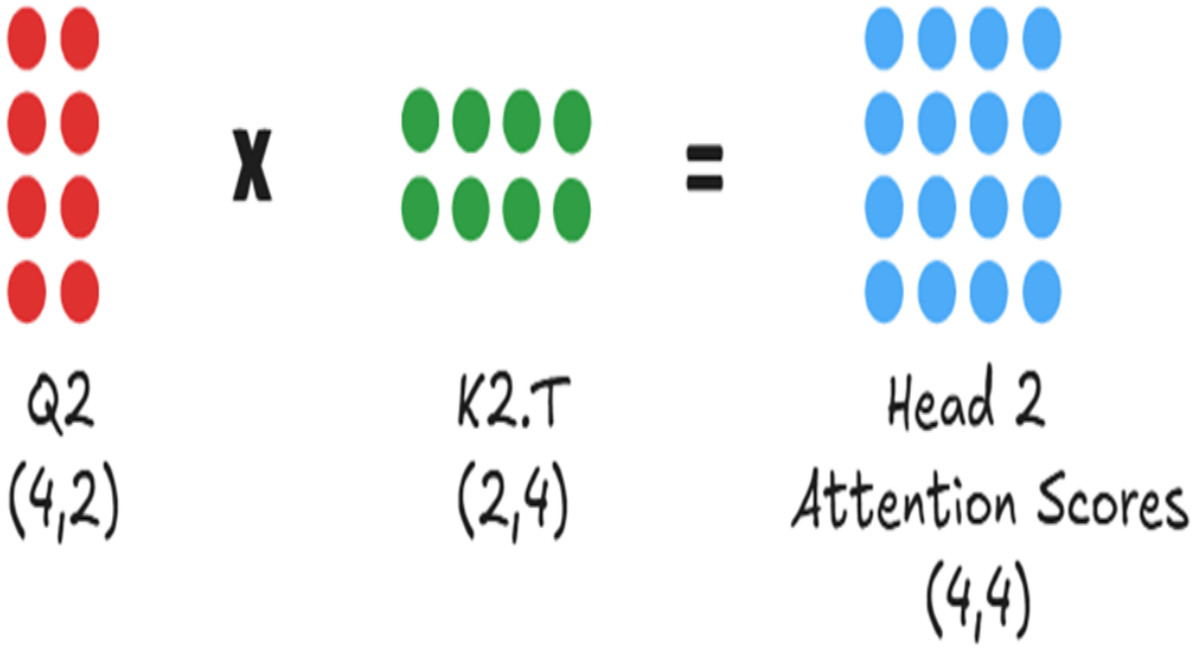
**Figure 2.7 Parallel Projections in Multi-Head Attention. The input embedding X is projected into separate Query, Key, and Value matrices for each attention head.**




As shown in figure 2.7, if our model has two heads, the initial (4, 8) input embedding is not projected into three (4, 4) matrices. Instead, it is projected in parallel into six smaller (4, 2) matrices: Q1, K1, V1 for Head 1, and Q2, K2, V2 for Head 2.

Next, each head computes its own attention scores independently and in parallel. Head 1 calculates the relevance between its queries (Q1) and keys (K1), while Head 2 does the same with Q2 and K2.

**Figure 2.8 Each attention head computes its own unique attention scores matrix in parallel.**

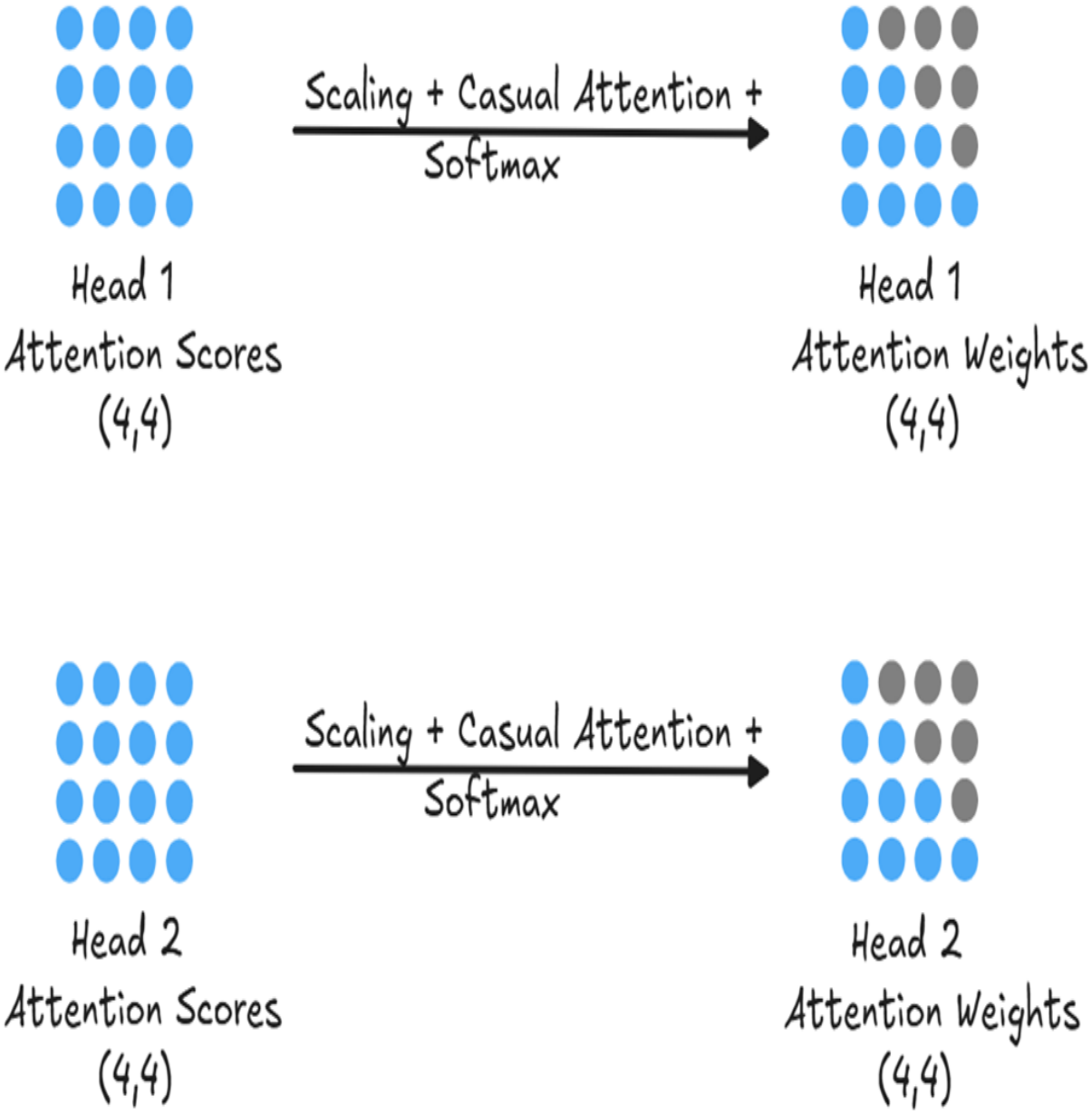




This ability to analyze the input from multiple perspectives simultaneously is the core of multi-head attention's power. By having its own unique projection matrices, each head learns to view the input in a different representational subspace. This allows for specialization: Head 1 might learn

to focus on grammatical structure, while Head 2 might focus on semantic meaning, all from the same input sequence.

After each head has independently calculated its raw attention scores, these scores represent a measure of raw, unnormalized relevance. For Head 1, its (4, 4) Attention Scores matrix tells it how strongly each of its queries connects with each of its keys. However, these raw scores are not yet in a usable format for creating a weighted average. To make them useful, each head processes its score matrix through a series of transformations, as shown in figure 2.9.

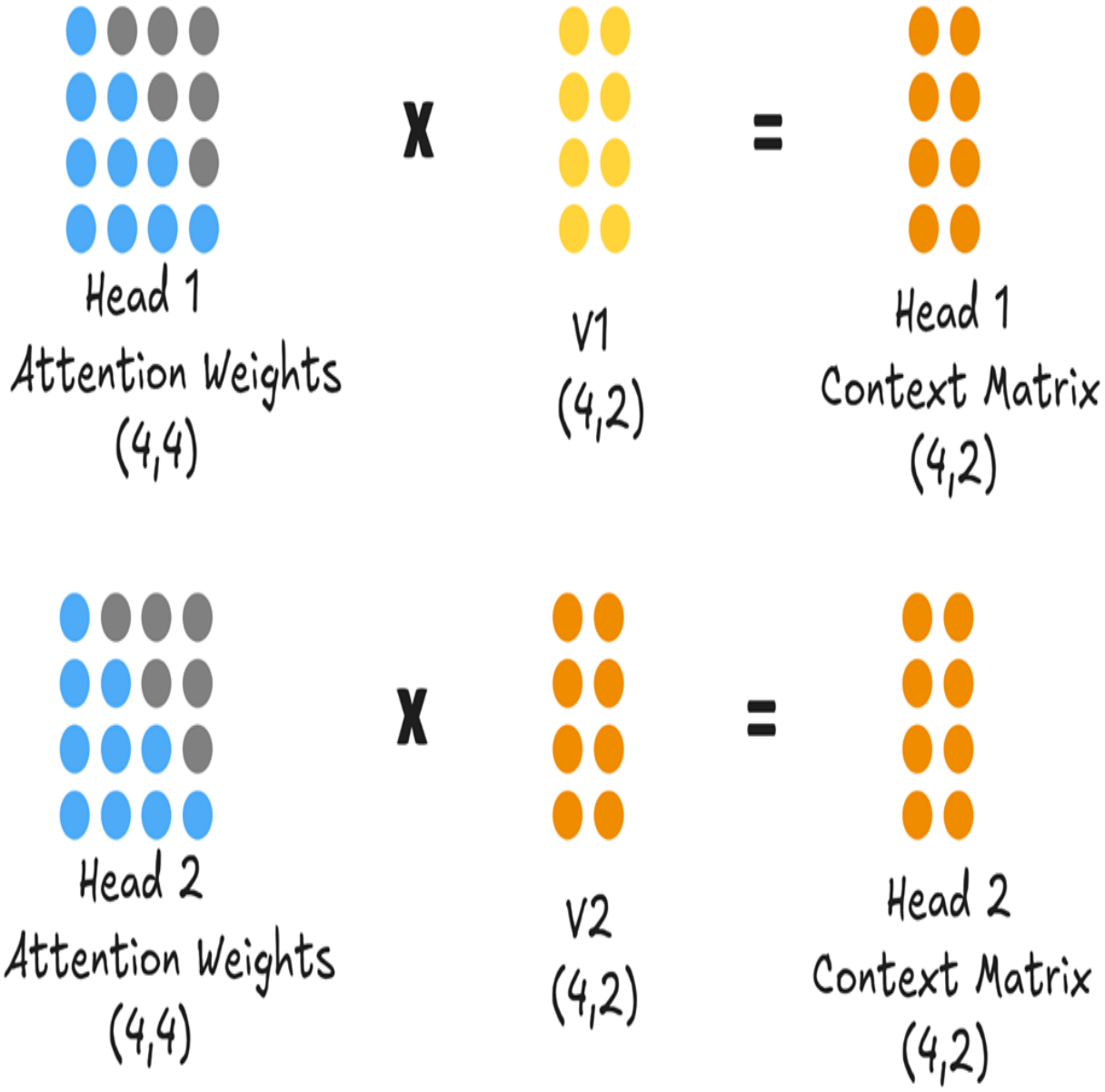
**Figure 2.9 The attention scores for each head are independently processed to create final attention weights.**




Just like in the single-head case, each head's raw attention scores are then scaled, masked, and converted into a probability distribution of attention weights via a softmax function.

With the final, normalized attention weights for each head, the model can now create its enriched output. This is achieved by multiplying each head's Attention Weights matrix with its corresponding Value matrix.

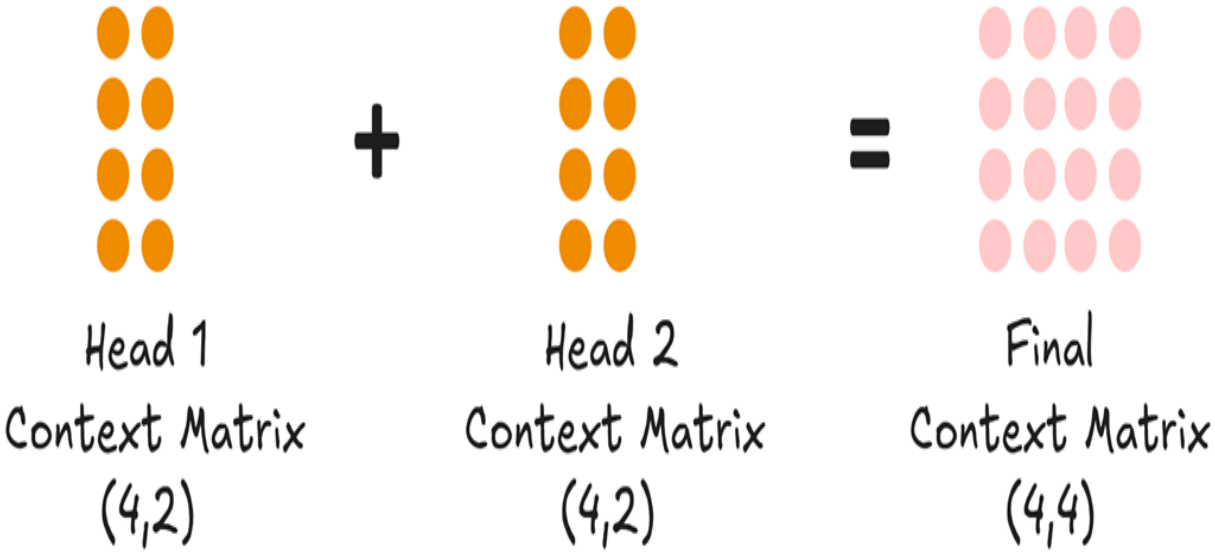
**Figure 2.10 Each head produces its own context matrix, representing its unique, context-aware view of the input sequence.**




At the end of this step, we have two separate context matrices: Head 1 Context Matrix and Head 2 Context Matrix, both of shape (4, 2). Each matrix is a different contextualized representation of the original input. The final step in the multi-head attention block is to unify these parallel streams of information back into a single representation that the next layer can use. This is done in two stages.

First, the individual context matrices from all the heads are concatenated together along their last dimension (column-wise).

**Figure 2.11 The context matrices from all heads are concatenated to form a single, richer matrix, which is then passed through a final projection layer.**




As shown in figure 2.11, concatenating the Head 1 Context Matrix (shape 4, 2) and the Head 2 Context Matrix (shape 4, 2) results in a combined matrix of shape (4, 4). This new matrix now contains the insights from both heads.

Second, this concatenated matrix is passed through one final linear layer, often called the output projection layer. This layer has its own learnable weights and is responsible for mixing the information from the different heads and projecting it back into the main model's expected dimension, producing the final Context Matrix (shape 4, 4) that exits the multi-head attention block.

This parallel, multi-faceted, and finally unified approach is what gives the Transformer architecture its expressive power. The Context Matrix it produces is what gets passed to the subsequent layers to eventually produce the logits for our next-token prediction.

## 2.2.2 From context vectors to logits

We saw how the attention mechanism processes our input embeddings and produces an enriched Context Matrix. For our input "The next day is," this is a (4, 4) matrix where each row is a new, context-aware vector for each token.

It is the culmination of the model's effort to understand the relationships between words in the sequence. Now, this matrix gets passed to the subsequent layers in the Transformer block to eventually produce the logits for our next-token prediction.

**Step 1: The Feed-Forward Network**

The Context Matrix first passes through a Feed-Forward Network (FFN) within the Transformer block. Unlike the attention mechanism, which looks across all tokens, the FFN processes each token's context vector independently. It typically consists of two linear layers with a non-linear activation function in between. This step allows the model to perform more complex calculations on each token's contextualized representation. Crucially, the FFN is designed to output a matrix of the exact same shape as its input, preserving the dimensions of our Context Matrix.

**Step 2: Iterating Through the Transformer Blocks**

The output from the FFN doesn't immediately exit the model. It first goes through the final components of the current Transformer block, which include another Layer Normalization and a residual connection that adds the block's input to its output. This entire process attention, feed-forward network, normalization, and residual connections constitutes one full Transformer block. The resulting matrix is then fed as the input to the next Transformer block. This cycle repeats for every block in the model's architecture (e.g., 12 times for GPT-2 small).

**Step 3: The Final Projection to Logits**

After the sequence has been processed by the very last Transformer block in the stack, the resulting matrix of context vectors goes through one final Layer Normalization. This normalized matrix is then passed to the Final Output Layer, which is where the model makes its prediction.

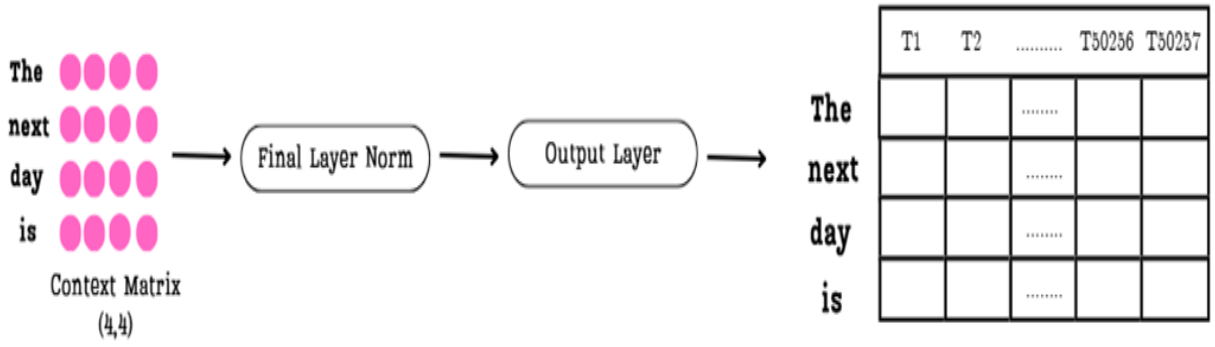
The Output Layer performs the crucial step of projecting our context vectors into the vast space of the model's vocabulary. Let's define a key term: logits.

**Definition**

What are Logits? A logit is a raw, unnormalized score. For any given position in a sequence, the model produces a logit for every single word in its vocabulary. A higher logit score for a particular word indicates a higher likelihood that that word is the correct next token.

The Output Layer is a simple linear layer whose job is to take the final Context Matrix and transform it into the Logits Matrix.

**Figure 2.12 The journey from the final Context Matrix to the Logits Matrix. The Output Layer projects each context-aware vector into a long vector of scores, one for each word in the vocabulary.**




As shown in figure 2.12, the entire Context Matrix is processed. Each of its rows is transformed into a very long vector of logits:

- **Input:** The final Context Matrix from the last Transformer block, with shape (4, 4).
- **Transformation:** The Output Layer projects each of the 4 rows into a vector of size 50,257 (the vocabulary size of GPT-2).
- **Output:** The final Logits Matrix with shape (4, 50257).

Each row in this massive matrix represents a complete prediction. The first row contains the model's scores for what token should follow "The," the second for what token should follow "next," and so on.

Now that we have this matrix of raw scores, how does the model make a final, single-token decision? This next step contains the most important insight for optimizing the entire inference process.

## 2.2.3 The key insight: Why only the last row matters

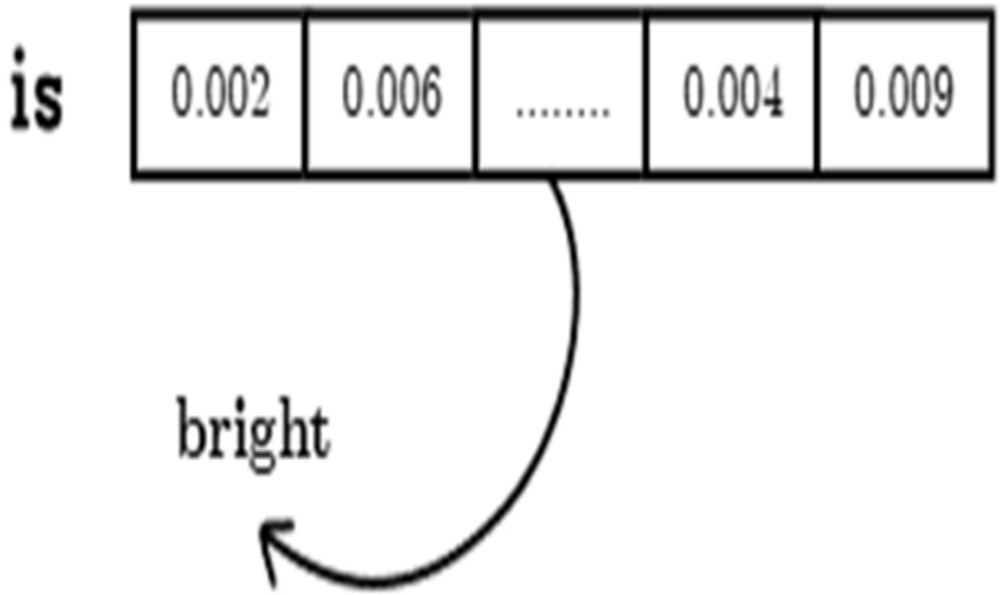
We now have our Logits Matrix, a massive tensor of shape (4, 50257) containing predictions for every position in our input sequence. However, our goal is very specific: we only want to predict the single token that comes after our complete input, "The next day is."

**This means we can discard almost all of the information in the Logits Matrix.**

- The first row (predictions for what comes after "The") is irrelevant.
- The second row (predictions for what comes after "next") is irrelevant.
- The third row (predictions for what comes after "day") is irrelevant.

The only row we care about is the last one, the logits vector corresponding to the token "is." This single vector holds the key to our next token. This is the crucial insight: since we only ever use the last row to make our prediction, re-computing the logits for all the earlier rows at every single step is incredibly wasteful. This observation is the fundamental motivation for optimizations like the KV cache and its derivatives, MQA and GQA.

**Figure 2.13 The final prediction step. The logits vector for the last token is converted into a probability distribution, and the token with the highest probability is selected as the output.**




To select the next token for our input we:

- **Isolate the Final Logits Vector:** We select only the last row from the Logits Matrix. This gives us a vector of shape (1, 50257) containing the model's unnormalized confidence scores for every possible word in its vocabulary.
- **Apply the Softmax Function:** These raw logits are converted into probabilities using the softmax function. This function transforms the entire vector into a probability distribution, where each value is between 0 and 1, and all values sum to 1. The output is a vector of probabilities, as shown in the figure with values like 0.002, 0.006, etc.
- **Select the Most Likely Token:** We now simply find the index of the highest probability in this distribution (an argmax operation). This index corresponds to a specific token in the model's vocabulary. As the diagram shows, if the highest probability points to the token "bright", then that becomes the model's final, generated output for this step.

This entire, multi-step part from raw text to a single predicted token is performed for every token the model generates. And this gives us a key insight: after all the complex, context-building work, the final prediction depends only on the context vector of the last token. It's crucial to remember that this last context vector is significant because, thanks to the self-attention mechanism, it already contains a weighted summary of information from all previous tokens in the sequence.

This should make you suspicious. If we are constantly feeding a growing sequence back into the model, are we doing a lot of unnecessary, repeated work in the attention block itself? As we'll prove mathematically in the next section, the answer is a resounding yes.

# 2.3 The problem of redundant computations

So far, we've established two critical facts about LLM inference:

1. The model generates text one token at a time in an autoregressive loop, feeding its own output back as input.
2. To predict the single next token, the model only requires the context vector of the last token in the current sequence.

Now, let's connect these two ideas. If the model is constantly re-processing a growing sequence of tokens, and yet only needs the information from the very last one to make its next decision, it seems that a huge number of computations might be performed unnecessarily. Intuitively, it feels like we might be performing the same calculations again and again.

I'm going to show you that during inference, we are indeed repeating many computations. We will then see what we can do to avoid these repetitions, which will lead us directly to the concept of the KV Cache.

## 2.3.1 Intuition: Are we calculating the same thing over and over?

Let's move from intuition to concrete mathematical proof. We'll first show the redundancy intuitively by tracing the data at each step, and then we will

quantify its performance impact by analyzing the computational complexity. Let's revisit the autoregressive loop from figure 2.2, but this time, let's focus on the data being passed into the model at each step.

Imagine we start with the prompt "The next day."

**Inference Step 1:**

- **Input:** "The next day"
- **Process:** The three tokens go through the entire LLM pipeline.
- **Output:** "is"

**Inference Step 2:**

- The new token is appended.
- **Input:** "The next day is"
- **Process:** These four tokens go through the entire LLM pipeline.
- **Output:** "bright"

**Inference Step 3:**

The new token is appended again.

- **Input:** "The next day is bright"
- **Process:** These five tokens go through the entire LLM pipeline.
- **Output:** "and"

Notice the pattern. In Step 2, we are re-processing the tokens "The," "next," and "day." We already processed them in Step 1. In Step 3, we are re-processing "The," "next," "day," and "is," all of which have been processed in previous steps. It seems we are passing the same tokens through the entire architecture again and again, just to add one new token at the end.

This process feels inefficient. It's like re-reading the first nine chapters of a book every time you want to read a new chapter. If reading each chapter takes a fixed amount of time, then reading up to chapter n requires 1 + 2 + ... + n units of work scaling quadratically, $O(n^2)$.

The main drawback of these repeated computations is their explosive cost: for every additional token, the GPU must re-process and store increasingly large amounts of data, making both time and memory requirements grow rapidly with sequence length.

This intuitive sense of repetition is, in fact, correct. Let’s take a hands-on example and prove mathematically that we are repeating the exact same calculations within the attention mechanism at every single step of inference.

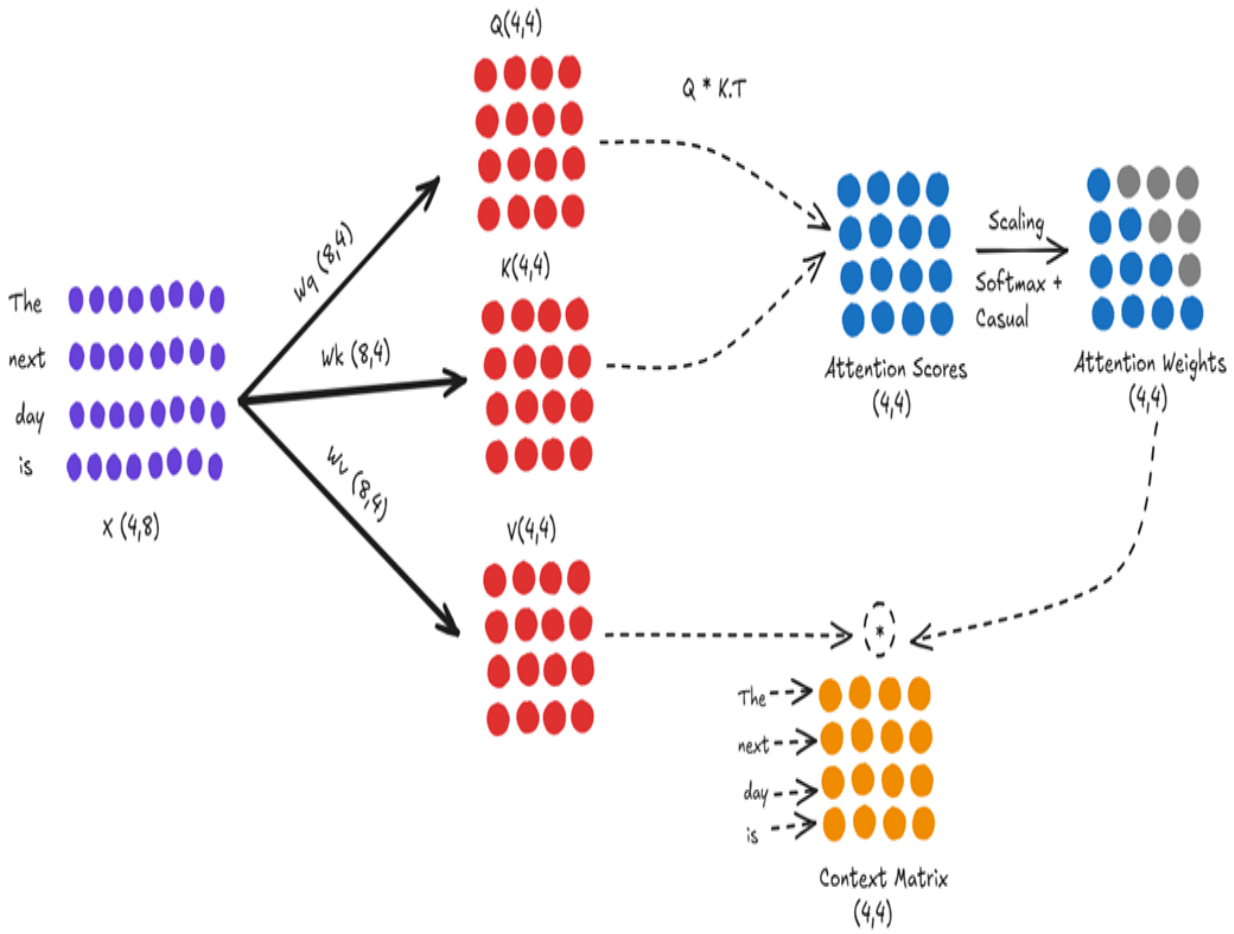
## 2.3.2 A mathematical proof: Visualizing repeated calculations

Our intuition suggests we are performing redundant work. Now, let's prove it by walking through the attention mechanism for two consecutive inference steps. We will see with our own eyes that we are calculating the exact same values multiple times.

### Step A: Inference at Time T=4 (Input: "The next day is")

First, let's consider the state of our model at the moment it has processed the input "The next day is" and is about to predict the next token. This is an input sequence with 4 tokens. As shown in figure 2.14, this sequence is about to be processed by a single attention head.

**Figure 2.14 The full attention calculation for the input sequence "The next day is."**




Let's trace the data flow in figure 2.14 step-by-step:

1. **Input Embeddings (X):** We start on the far left with our input embedding matrix X, which has a shape of (4, 8) for our four tokens.

2. **Projection:** This X matrix is multiplied by the fixed, pre-trained weight matrices Wq, Wk, and Wv. This projection creates the Query (Q), Key (K), and Value (V) matrices, which in this example all have the shape (4, 4).

3. **Attention Scores:** Next, the model calculates the raw relevance between tokens. The Query matrix is multiplied by the transpose of the Key matrix (Q * K.T), resulting in the (4, 4) Attention Scores matrix.

4. **Attention Weights:** This raw score matrix is then processed (scaled and passed through a causal softmax) to produce the final (4, 4) Attention

Weights matrix. The grayed-out dots represent future positions that have been masked to zero.

5. **Context Matrix:** Finally, the Attention Weights are multiplied by the Value matrix to produce the (4, 4) Context Matrix.

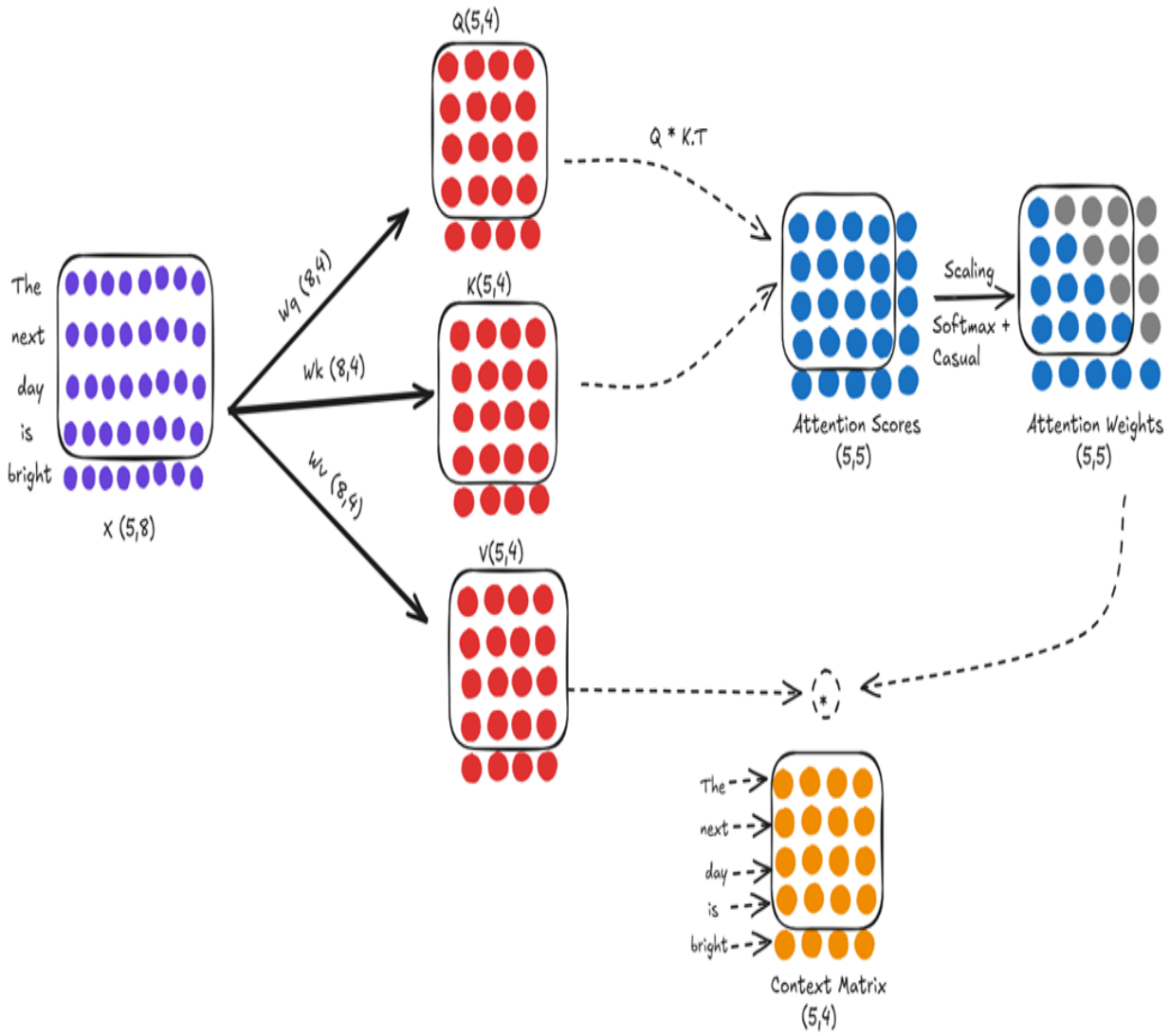
This Context Matrix then travels through the rest of the Transformer architecture. As we established, only the last row of this matrix the context vector for "is" is used to generate the final logits and predict the next token. Let's say the model correctly predicts the token "bright."

Now, we move to the next step in the autoregressive loop, and this is where the inefficiency becomes glaringly obvious.

**Step B: Inference at Time T=5 (Input: "The next day is bright")**

Following the autoregressive loop, the newly predicted token, "bright," is appended to our sequence. The new input for the model is now "The next day is bright," a sequence of 5 tokens. This new, longer sequence is fed back into the exact same attention mechanism, with the exact same learned weight matrices (Wq, Wk, Wv).

**Figure 2.15 The full attention calculation for this new, 5-token input.**




At first glance, this looks like a completely new calculation. But let's compare it closely to the computation we just performed in figure 2.15.

The first four rows of our new input matrix X (shape 5, 8) are identical to the entire input matrix from the previous step. Since the weight matrices (Wq, Wk, Wv) are fixed during inference, this means the first four rows of our new Query, Key, and Value matrices (all now shape 5, 4) are identical to the entire Query, Key, and Value matrices from the previous step.

This redundancy cascades directly into the attention score calculation. The score at any position (i, j) is the dot product of the i-th query vector and the j-th key vector. Since the first four query vectors and the first four key

vectors are identical to the previous step, the entire top-left (4, 4) sub-block of our new (5, 5) Attention Scores matrix is also identical to the entire score matrix we calculated just a moment ago.

We are performing a massive amount of redundant computation. We are re-calculating the projections and the attention scores for the entire history of the sequence at every single step. This is a huge waste of computational resources. And the most inefficient part? As we've established, after all of this redundant work, the only piece of information we are actually going to use to predict the next token is the context vector derived from the new token, "bright."

We are re-calculating an entire history of interactions just to compute one new row, and then we throw away most of the old rows' final outputs anyway. This is the core problem that inference optimization techniques must solve.

### 2.3.3 The performance impact: From quadratic to linear complexity

The redundant calculations we've identified are not just theoretically inefficient; they have a severe impact on performance, especially as the input sequence grows longer. This impact is best understood through the lens of computational complexity.

**Note**

It is important to clarify that this discussion is strictly about the inference stage.

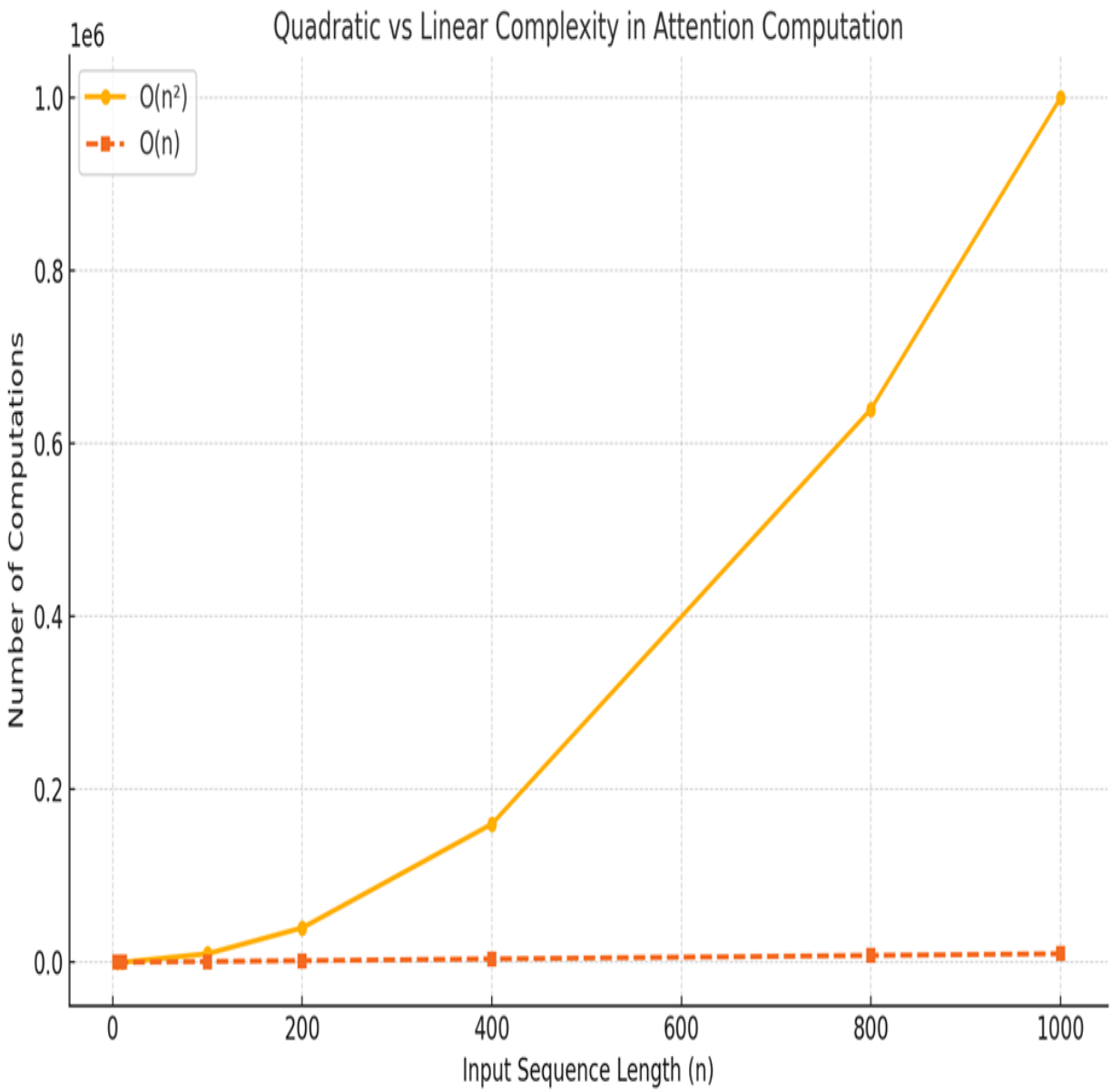
Without any optimization, the core of the attention mechanism is quadratic in nature. For each layer and for each attention head, the number of calculations needed to compute the attention scores scales quadratically with the length of the input sequence (n), often denoted as $O(n^2)$. While the total number of computations is multiplied by constants like the number of layers (L) and heads (H), it's the quadratic relationship with the sequence length n that dominates performance.

Why quadratic? Think about the Attention Scores matrix.

- For an input of 4 tokens, we compute a 4x4 matrix (16 scores).
- For an input of 5 tokens, we compute a 5x5 matrix (25 scores).
- For an input of 1,000 tokens, we would compute a 1,000 x 1,000 matrix (1,000,000 scores).

At each step of autoregressive generation, we are re-calculating this entire n x n matrix, performing $O(n^2)$ work repeatedly. As the sequence length n grows, the number of computations explodes. This quadratic complexity is the primary reason why uncached inference for long sequences is computationally expensive and extremely slow. Each new token becomes progressively slower to generate than the last because the model has to do more and more historical re-calculation.

**Figure 2.16 A plot comparing the quadratic ($O(n^2)$) growth of computations in uncached autoregressive inference versus the ideal linear ($O(n)$) growth.**




The goal of inference optimization is to transform this quadratic process into a linear one (O(n)). In a linear complexity scenario, the amount of computation required to generate a new token grows linearly with the length of the sequence, not quadratically. This means that while generating a new token for a long sequence still requires more computation than for a short one, the increase is far more manageable. For instance, doubling the context length would roughly double the work for the next token, rather than quadrupling it, preventing the explosive growth of the uncached approach.

This is precisely what caching allows us to achieve. By storing the results of past computations instead of repeating them, we can break free from the quadratic trap. As we saw visually, the only new computations we need to perform for a new token are related to that token itself. The computations for all previous tokens can be retrieved from memory.

This shift from quadratic to linear complexity is the fundamental reason why caching is not just an optimization but a foundational requirement for making LLMs with large context windows practical. It explains the dramatic speedup we will demonstrate in code:

- **Without Caching (Quadratic):** Generating the 100th token is much slower than generating the 10th token.
- **With Caching (Linear):** Generating the 100th token takes roughly the same amount of time as generating the 10th token.

Having established the dire need for a solution, we are now ready to build it.

# 2.4 The solution: Caching for efficiency

The solution to this problem is both elegant and intuitive: if we are repeatedly calculating the same values, why not just calculate them once and store them for future use? This is the core principle of caching.

By storing the results of past computations, we can avoid re-doing the work for tokens we've already seen. This allows us to break free from the quadratic trap and achieve a much more efficient, linear computation time. This powerful optimization is known as the Key-Value Cache, or KV Cache.

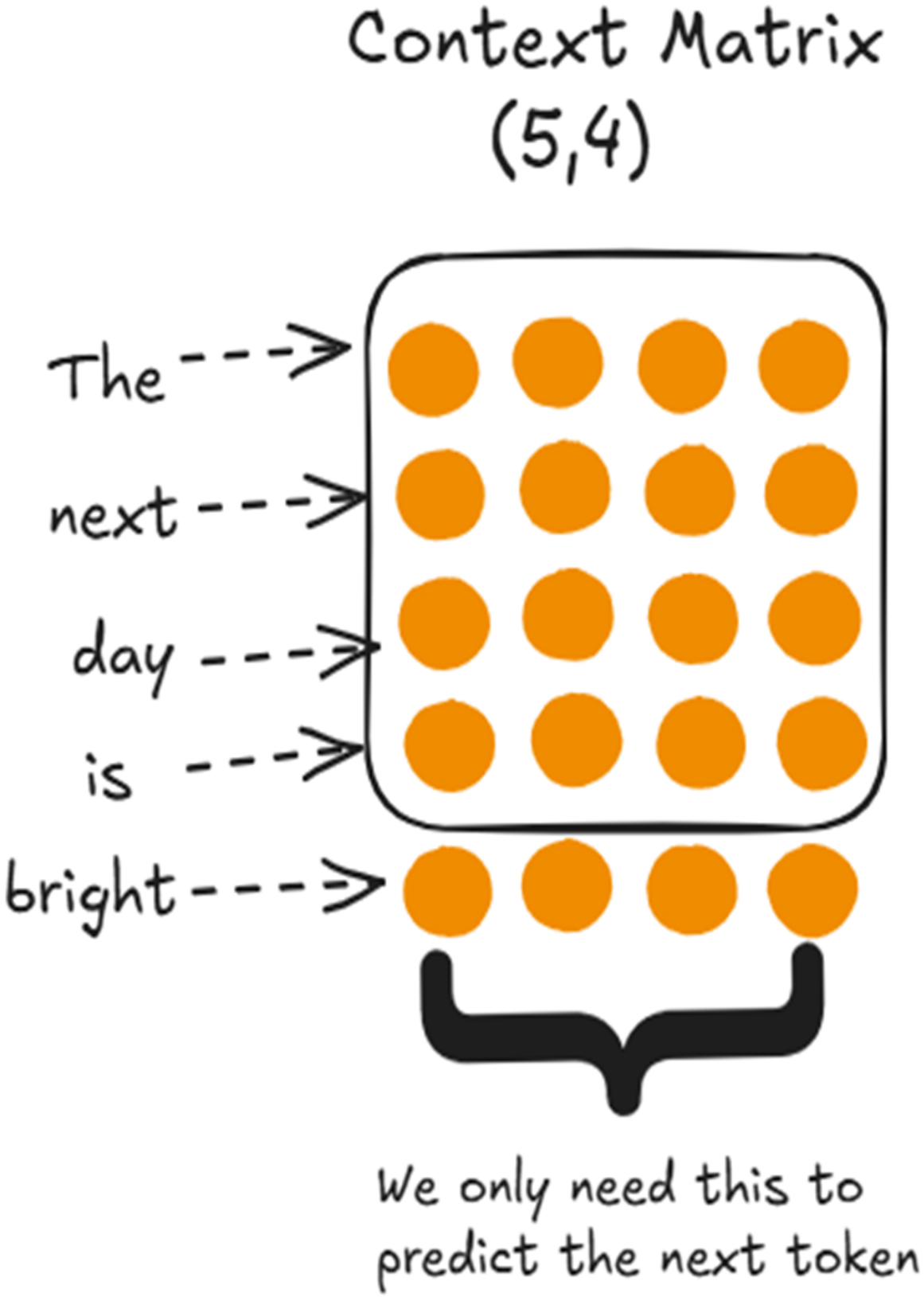
## 2.4.1 What to cache? A step-by-step derivation

To understand what we need to cache, we must start with our end goal and work backward. As we established in section 2.2.3, our entire objective at each inference step is to produce the single context vector for the most recent token.

Let's take our example where the model has just processed "The next day is" and generated "bright." The new input sequence is now 5 tokens long. To predict the next token, we only need the context vector for "bright."

The values shown in the box are from previous steps. Since these values remain unchanged across steps, we cache them. The same principle applies to the subsequent images in this section.

**Figure 2.17 Out of the entire context matrix, only the final row, corresponding to the newest token, is required to predict the next token in the sequence.**


Context Matrix
(5,4)
The
next
day
is
bright
We only need this to
predict the next token

So, our immediate goal is to compute this one vector. Let's backtrack to figure out the minimal set of computations required to produce it.

### How is the Context Vector for "bright" Calculated?

From our previous exploration of the attention mechanism, we know that the context vector is the result of multiplying the attention weights by the Value matrix. Since we only need the context vector for "bright," we only need to perform the calculation for that specific row.

**Figure 2.18 The context vector for "bright" is calculated by multiplying the attention weights for "bright" with the full Value matrix.**

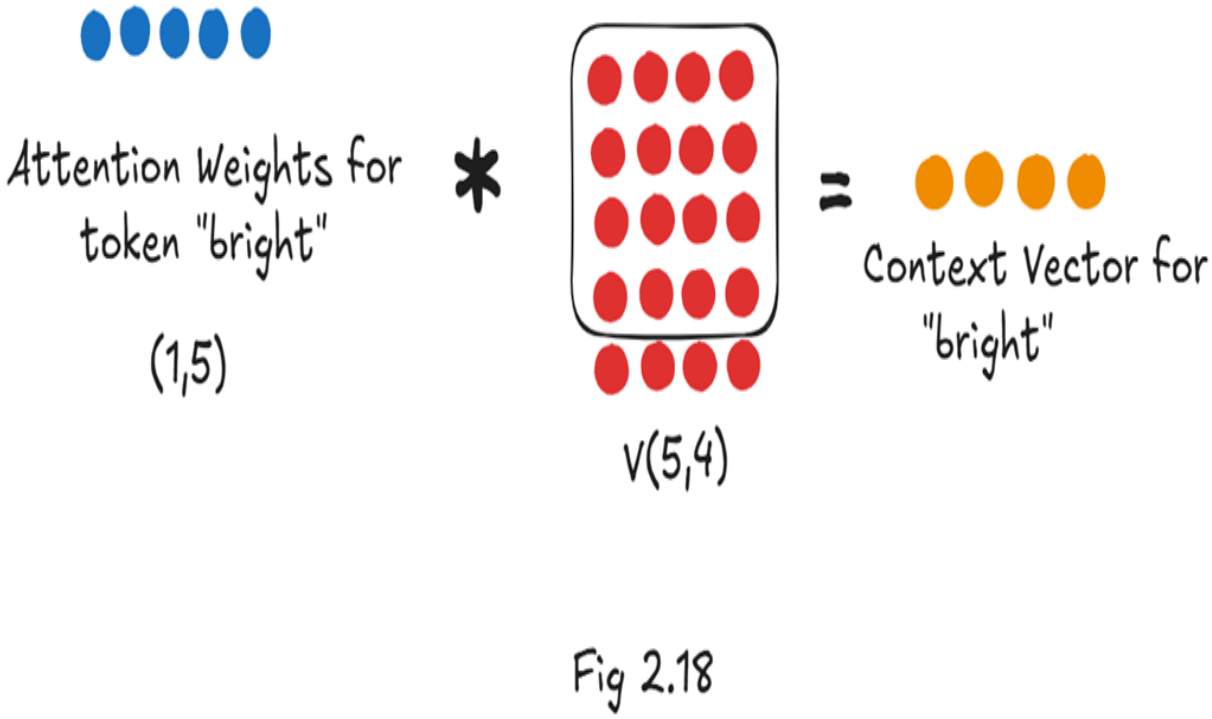



Fig 2.18

As figure 2.18 shows, to get our target vector, we need two components:

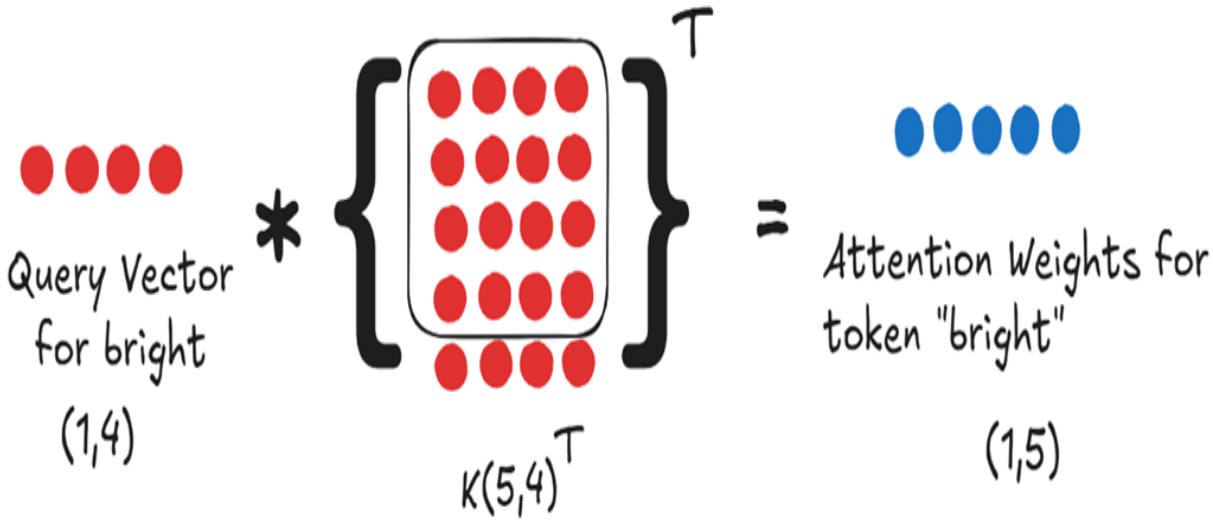
- **The Attention Weights for "bright":** This is a single row vector (shape 1x5) that tells us how much "bright" should attend to every token in the sequence (including itself).
- **The full Value matrix:** This is a matrix of shape (5x4) that contains the "content" representation for every token in the sequence.

Let's continue backtracking. How do we get these two components?

**How are the Attention Weights for "bright" Calculated?**

The attention weights are simply the softmax-normalized attention scores. So, to get the weights for "bright," we first need the raw attention scores for "bright." These scores are calculated by taking the dot product of the Query vector with the transpose of the full Key matrix.

**Figure 2.19 The attention weights for "bright" are derived from its Query vector and the Key vectors of all tokens in the sequence.**




To get the attention weights, we fundamentally need:

- The **Query vector for our new token.**
- The **full Key matrix**, which contains the Key vectors for all five tokens in the sequence.

So, our list of requirements has now grown. To get the single context vector for "bright," we need:

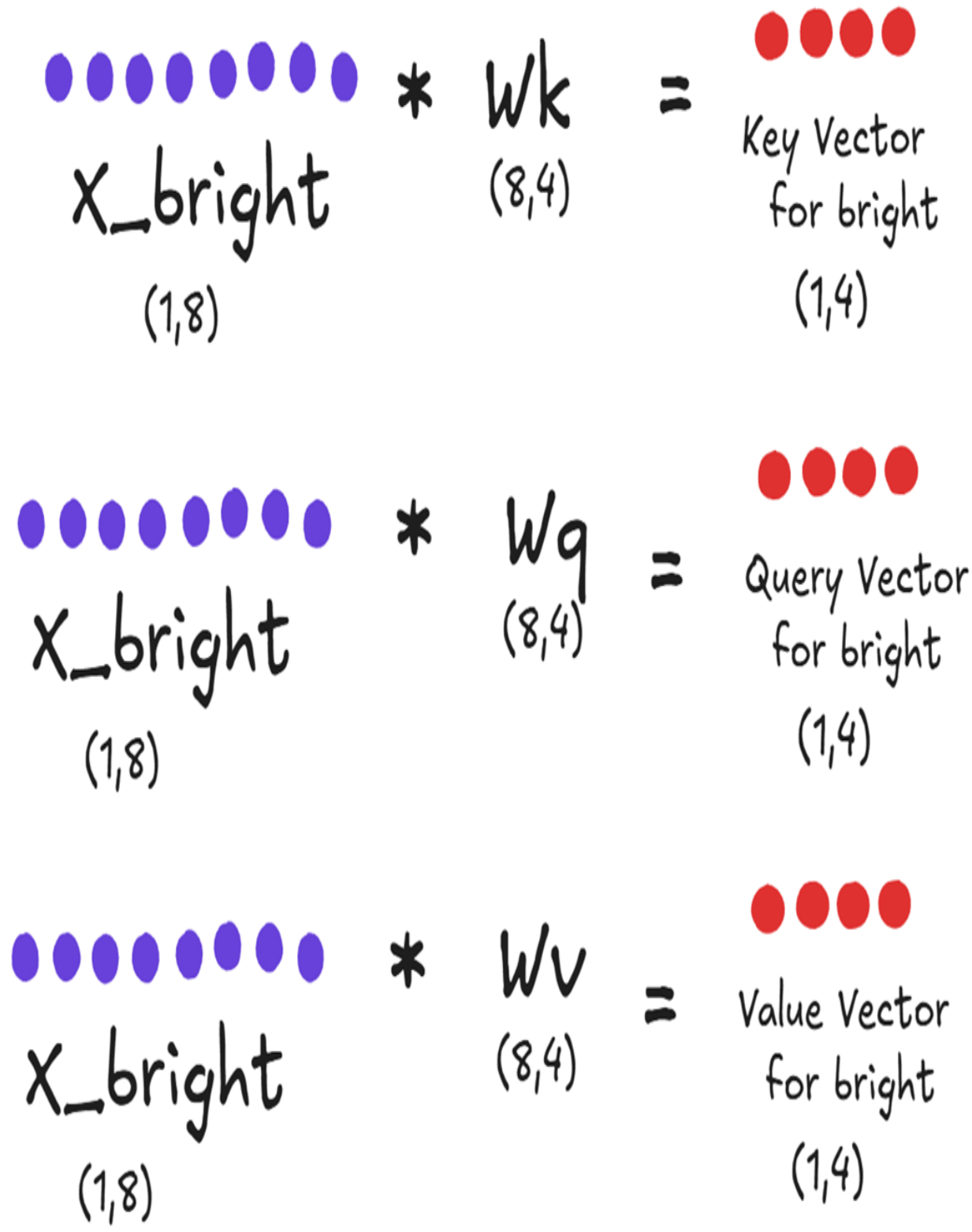
- The **Query vector for "bright" (q_bright).**
- The full Key matrix (K).
- The full Value matrix (V).

This finally brings us to the source: how are these Query, Key, and Value vectors created?

The Query, Key, and Value vectors are all created by projecting the input embeddings through the learned weight matrices Wq, Wk, and Wv. Here is where we can separate what is new from what is old.

When the new token "bright" arrives, we must compute its specific Query, Key, and Value vectors. This is done via three simple matrix multiplications using its input embedding, X_bright.

**Figure 2.20 The three essential computations for a new token. The input embedding for "bright" is projected to create its unique Query, Key, and Value vectors.**




As shown in figure 2.20, these are the only new projections required:

- We compute the **Query vector** for “bright.”
- We compute the **Key vector** for “bright.”
- We compute the **Value vector** for “bright.”

Now we can finally answer the central question: what about the Key and Value vectors for all the previous tokens ("The", "next", "day", "is")?

In the inefficient, uncached approach we saw in section 2.3, we would have re-calculated them from scratch. But now we understand that this is wasteful. Those previous Key and Value vectors have already been computed in prior inference steps. They don't change.

This is precisely where the concept of caching comes into play. Instead of re-computing them, we can simply store the Key and Value matrices from the previous step in memory. This stored data is the Key-Value (KV) Cache. This leads us to the final, efficient workflow and answers why we only cache Keys and Values, but not Queries:

1. Compute for the New Token: When the token "bright" arrives, we perform the three essential multiplications shown in figure 2.20 to get its Query, Key, and Value vectors.
2. Assemble the full Key & Value Matrices:
    - We retrieve the Key matrix for "The next day is" (a 4x4 matrix) from our Key Cache.
    - We append the new Key vector for "bright" to it, creating the full 5x4 Key matrix.
    - We do the exact same thing for the Value matrix, retrieving the old Value matrix from the Value Cache and appending the new Value vector.
3. Calculate Attention: We use the new **Query vecto**r for "bright" and the full, updated **Key and Value matrices** to perform the attention calculation and get the single context vector we need.
4. Update the Cache: We update our cache by storing the new, larger 5x4 Key and Value matrices, ready for the next token.

This is the essence of the Key-Value Cache. We only perform the expensive projection calculations for the single new token at each step. All historical information is preserved in the cache. We don't need to cache the Query

vectors because, as we've established, we only ever need the query for the current token, which must always be computed fresh. This simple yet powerful technique of caching the Keys and Values is what transforms the attention calculation from a cripplingly slow quadratic operation into an efficient linear one.

### 2.4.2 The new inference loop with KV caching

With our understanding of what to cache, we can now define a new, highly efficient workflow for autoregressive generation. At each step, instead of re-calculating the entire history, we leverage our stored Key and Value matrices.

Let's summarize the new process when generating a token:

1. Receive New Token: The model receives the embedding for the single, new token.
2. Compute New Projections: Perform the three essential matrix multiplications to get the Query, Key, and Value vectors for only this new token.
3. Retrieve from Cache: Load the existing Key and Value matrices for all previous tokens from the KV Cache.
4. Append to Cache: Append the new Key and Value vectors to the cached matrices to form the full, updated Key and Value matrices for the entire sequence.
5. Calculate Attention: Multiply the new Query vector with the transpose of the full, updated Key matrix to produce the raw attention scores, which are then converted into attention weights.
6. Compute Context Vector: Multiply the attention weights by the full, updated Value matrix to get the single context vector for the new token.
7. Predict Next Token: Pass this context vector through the rest of the model's layers to predict the next token.
8. Update Cache: Save the updated Key and Value matrices back into the KV Cache for the next iteration.

This loop avoids the massive redundancy of the naive approach. The heavy lifting of matrix multiplications for past tokens is done only once and their results are simply reused. It's important to remember that this caching

happens independently at each level of the architecture: the KV cache is maintained on a per-layer and per-head basis. This means each Transformer layer in the model keeps its own distinct set of Key and Value caches for each of its attention heads, ensuring that the specialized context learned at each layer is preserved.

### 2.4.3 Demonstrating the speedup of KV caching

The theoretical benefit of moving from a quadratic to a linear process is clear, but the real-world impact is even more striking. We can demonstrate this with a simple test using the pre-trained GPT-2 model from Hugging Face, which allows us to enable or disable the KV cache with a simple flag (use_cache).

The following code will generate 100 new tokens from a prompt, first with the KV cache enabled, and then with it disabled, timing both processes.

**Listing 2.2 Demonstrating the Speedup of KV Caching**

```
prompt = "The next day is bright"
inputs = tokenizer(prompt, return_tensors="pt")
input_ids = inputs.input_ids
attention_mask = inputs.attention_mask

# Timing without KV cache
start_time_without_cache = time.time()
output_without_cache = model.generate(input_ids,
    ➥ max_new_tokens=100,
    ➥ use_cache=False,                          #A
    ➥ attention_mask=attention_mask)
end_time_without_cache = time.time()
duration_without_cache = end_time_without_cache -
start_time_without_cache
print(f"Time without KV Cache: {duration_without_cache:.4f}
seconds")

# Timing with KV cache
start_time_with_cache = time.time()
output_with_cache = model.generate(input_ids,
    ➥ max_new_tokens=100,
    ➥ use_cache=True,                           #B
    ➥ attention_mask=attention_mask)
```

```
end_time_with_cache = time.time()
duration_with_cache = end_time_with_cache -
start_time_with_cache
print(f"Time with KV Cache: {duration_with_cache:.4f} seconds")

# Calculate and print the speedup
speedup = duration_without_cache / duration_with_cache
print(f"\nKV Cache Speedup: {speedup:.2f}x")
```

Running this code on a standard machine reveals the efficiency of this technique.

```
Time without KV Cache: 30.9818 seconds
Time with KV Cache: 6.1630 seconds

KV Cache Speedup: 5.03x
```

The results are unambiguous. By simply enabling the KV cache, we achieve a more than 5x speedup in generating 100 tokens. For very large models and much longer sequences, this speedup factor can be even more dramatic, often reaching 6x or more. This is the incredible advantage of the KV cache: it makes real-time, interactive generation feasible by eliminating the crippling cost of repeated computations.

However, this incredible speed comes at a price. Caching isn't free. Storing these Key and Value matrices in memory introduces its own significant challenge, the "dark side" of the KV Cache

# 2.5 The dark side of the KV cache: The memory cost

Now we have seen the incredible speedup the KV Cache provides. By eliminating redundant computations, it makes interactive, long-sequence generation possible. However, this efficiency comes at a steep, non-negotiable price: memory.

This isn't just a matter of storage capacity; the inference process becomes memory-bandwidth bound. At every single generation step, the massive Key

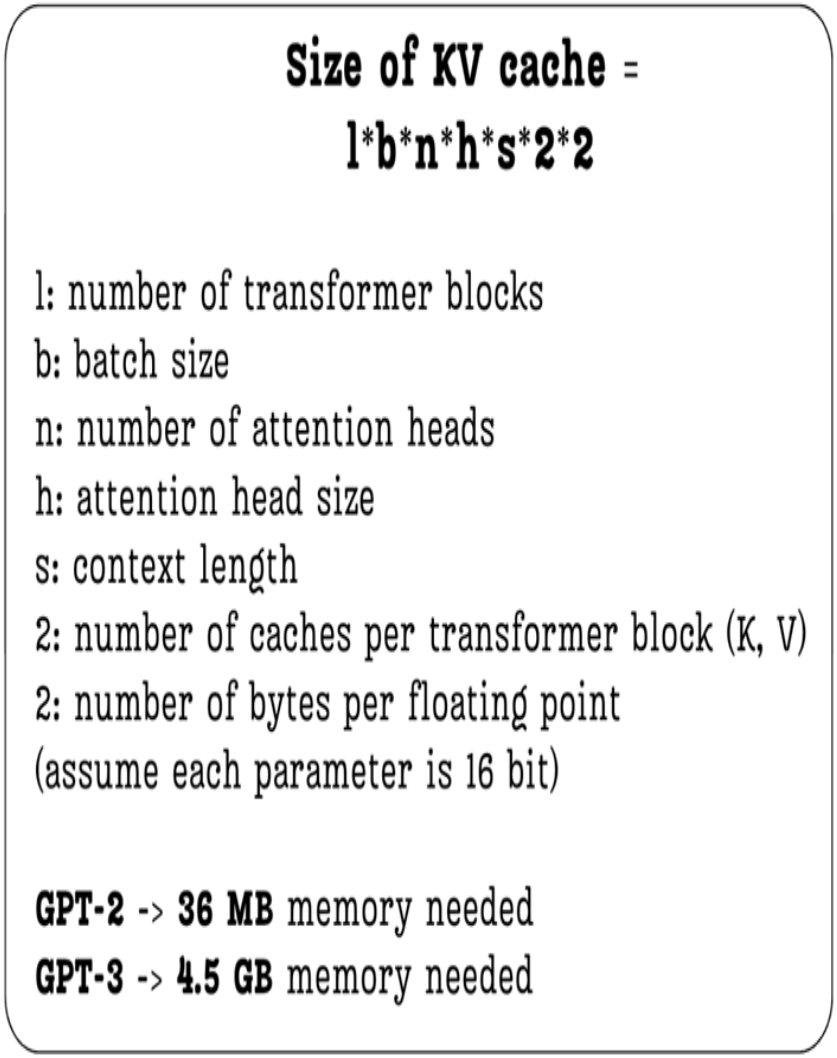
and Value matrices for all previous tokens must be read from the GPU's main memory (HBM) into its faster on-chip compute cores. This constant data movement becomes the new performance bottleneck, which is why modern GPUs designed for AI prioritize higher-capacity HBM and greater memory bandwidth, often more so than raw computational power (FLOPs).

Caching, by its very nature, is a trade-off. We are trading memory space to save computation time. While we avoid re-calculating the Key and Value matrices, we must now store them in the GPU's memory. For large models with long context windows, this memory footprint can become the new primary bottleneck.

### 2.5.1 The KV cache formula: Deconstructing the size

We can precisely calculate the memory required for the KV cache using a straightforward formula. Figure 2.21 breaks down each component of this calculation.

**Figure 2.21 The formula for calculating the KV cache size and its application to well-known models.**




**KV Cache speeds things, but takes space!**

Let's walk through the formula:

$$Size = l * b * n * h * s * 2 * 2$$

- l (layers): The total number of Transformer blocks in the model. We need a separate cache for each layer.
- b (batch size): The number of sequences we process in parallel.
- n (heads): The number of attention heads per layer.
- h (head size): The dimension of each attention head's Key and Value vectors.

- s (sequence length): The number of tokens in the context. This is a critical factor.
- First *2 We need to cache two matrices: one for Keys and one for Values.
- Second *2 This represents the number of bytes per parameter. A standard 16-bit floating-point number (like float16 or bfloat16) takes up 2 bytes of memory.

The formula makes the trade-off explicit. Every time we want to increase the model's context length (s), or use a larger model with more layers (l) and heads (n), the memory required for the KV cache grows proportionally.

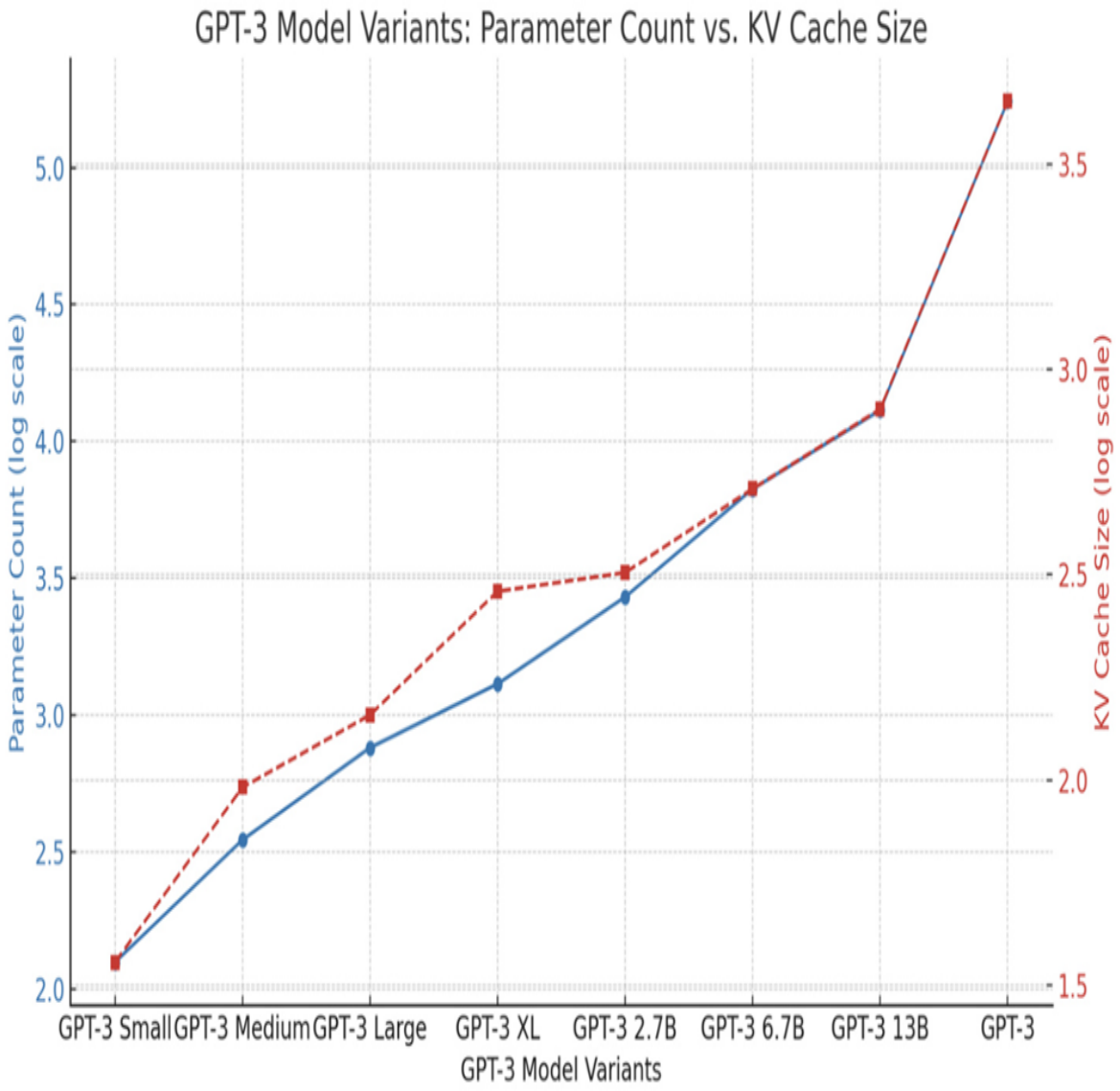
The examples in the figure highlight the real-world impact:

- The original GPT-2 (128M) model required a relatively modest 36 MB for its KV cache.
- GPT-3, a much larger and more powerful model, requires a staggering 4.5 GB of memory for the same purpose over 100 times more!

## 2.5.2 The scaling problem in practice

This exponential growth in memory consumption is a fundamental challenge in scaling Transformer models. As models grow larger and support longer context windows, the KV cache often becomes the primary limiting factor for deployment.

**Figure 2.22 A comparison of model parameter count versus KV cache size for different GPT-3 model variants. The dashed line shows that as models become larger, their memory requirements for the KV cache grow at a similar, steep rate.**




As figure 2.22 illustrates, there is a strong correlation between the size of a model and the size of its KV cache. This memory burden restricts the number of sequences we can process in a single batch and puts a hard cap on the maximum context length a model can support on a given piece of hardware.

Let's consider two modern examples based on our notes:

For a large 30B parameter model with 48 layers, 7168 total head dimensions (n*h), and a context length of 1024, the KV cache for a batch size of 128

would be a massive 180 GB. This exceeds the memory capacity of even the most powerful modern GPUs.

For a model with the architectural scale of DeepSeek-V3 (61 layers, 128 heads of size 128) and a huge context length of 100,000 tokens, the KV cache for a single sequence would be approximately 400 GB.

This is the dark side of the KV cache. It speeds things up, but it takes up an enormous amount of space. This memory pressure is the direct reason why API providers like OpenAI charge significantly more for models with larger context windows; the hardware cost to support that memory is substantial.

This bottleneck is what motivated researchers to find a better way. How can we get the speed benefits of caching without paying such a high price in memory? This question leads us to the first generation of architectural solutions: Multi-Query and Grouped-Query Attention.

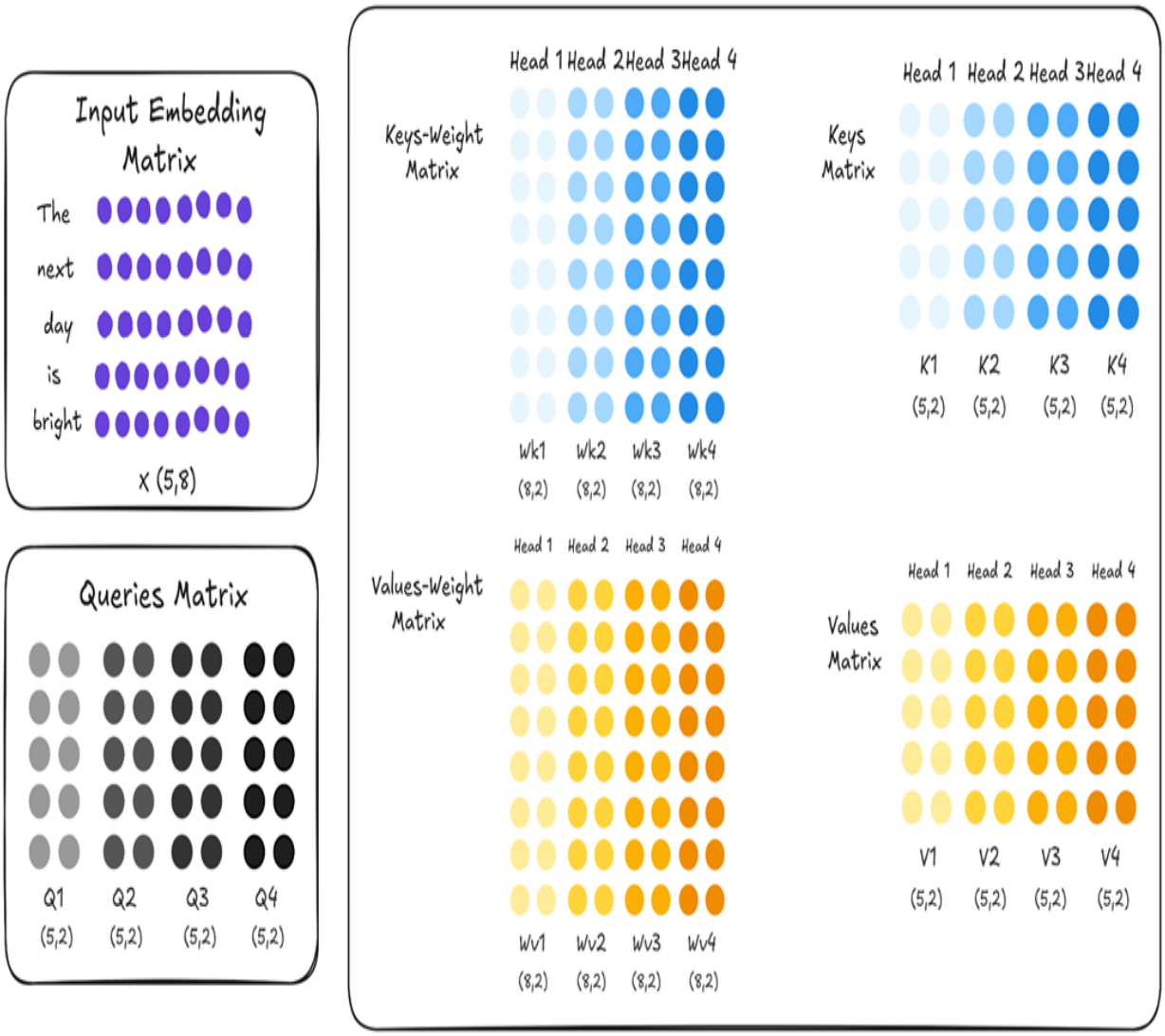
# 2.6 The memory-first approach: Multi-Query Attention (MQA)

What is the simplest, most direct thing one could do to solve the KV Cache memory problem? Multi-Query Attention (MQA) answers this question with a radical proposal: what if all the attention heads simply share the same Key and Value matrices?

## 2.6.1 The core idea: Sharing a single key and value

To understand MQA's innovation, we must first recall how standard Multi-Head Attention (MHA) works. In a standard Transformer, within each layer, each attention head acts as an independent expert. This means it has its own unique, learned weight matrices for Keys (Wk) and Values (Wv), distinct from all other heads in that layer. This can be summarized as: in vanilla MHA, each head has distinct Wk and Wv.

**Figure 2.23 Standard Multi-Head Attention (MHA). Each of the four attention heads has its own distinct Key and Value weight matrices, indicated by the different colors. This allows each head to specialize and learn different patterns.**

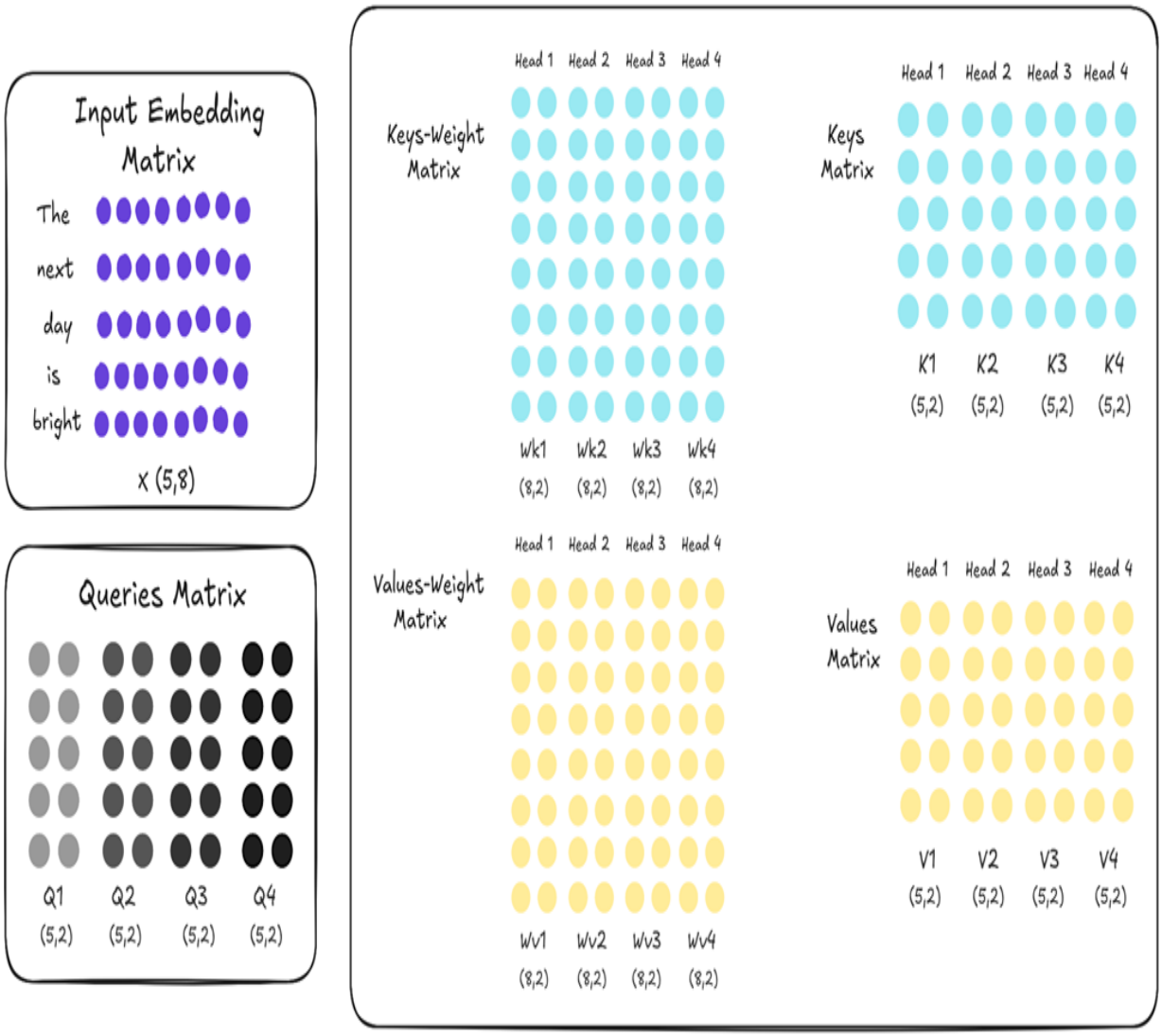



As shown in figure 2.23, if we have four attention heads, the Key weight matrix is effectively split into four unique parts: Wk1, Wk2, Wk3, and Wk4. The same is true for the Value weight matrix. Because these weights are initialized randomly and trained independently, each head learns to project the input embeddings into a different representational space. K1 is different from K2, V1 is different from V2, and so on. This diversity is the source of MHA's power; it allows the model to capture multiple perspectives simultaneously.

However, this is also the source of its memory problem. To enable fast inference, we must cache the full Key and Value matrices for every single head. Multi-Query Attention takes a direct and aggressive approach to solve

this. It proposes a simple change: while each head still gets its own unique Query projection (allowing each head to "ask" a different question), all heads are forced to share one single, common set of Key and Value projections.

**Figure 2.24 Multi-Query Attention (MQA). All four heads still have unique Query projections, but they now share a single, common Key and Value projection, indicated by the uniform light blue and yellow colors.**




Look closely at the difference in figure 2.24. The Key weight matrices for all heads (Wk1 through Wk4) are now identical, and the same is true for the Value weight matrices. This means that when the input embedding is

projected, the resulting K1, K2, K3, and K4 matrices are all exact copies of each other. The same applies to the Value matrices.

The implication for caching is immediate and profound. Instead of needing to store four separate Key matrices and four separate Value matrices in our cache, we only need to store one Key matrix and one Value matrix. During inference, each of the four query heads will simply attend to this single, shared set of keys and values.

This simple architectural tweak is the core idea behind Multi-Query Attention. It prioritizes memory savings above all else. In the next sections, we will explore the dramatic impact this has on the KV Cache formula and the inevitable trade-off it creates for model performance.

### 2.6.2 The impact on the KV cache formula

The architectural change from MHA to MQA has a dramatic and direct impact on the size of the KV Cache. Let's revisit the formula we established in Section 2.5.1:

$$Size\_MHA = l * b * n * h * s * 2 * 2$$

The key variable here is n, the number of attention heads. In MHA, because every head has its own unique Key and Value matrices, the total memory required scales linearly with the number of heads.

In Multi-Query Attention, since all heads share the same single Key and Value pair, we no longer need to store n different versions. We only need to store one. The formula becomes:

$$Size\_MQA = l * b * 1 * h * s * 2 * 2$$

In this revised formula, the n term is effectively replaced with 1, where n is the total number of attention heads, eliminating the linear scaling with the number of heads. This reduces the size of the KV Cache by a factor of n. The impact of this reduction is staggering for large models:

GPT-3 (175B): This model has 96 attention heads (n=96). Using MQA would reduce its KV Cache size by a factor of 96, from 4.5 GB down to a mere 48 MB.

DeepSeek-V3 (671B): This model has 128 attention heads (n=128). MQA would reduce its theoretical KV Cache size by a factor of 128, from ~400 GB down to just over 3 GB.

This is an incredible reduction in memory footprint, and it directly translates to faster inference (as we will see in the code) because less data needs to be loaded from memory at each step. So, if MQA is so effective at solving the memory problem, why doesn't every model use it?

## 2.6.3 The performance trade-off: Loss of expressivity

The dramatic memory savings of MQA seem almost too good to be true, and in a way, they are. This efficiency comes at a significant cost: a degradation in the model's performance and its ability to understand complex language. To understand this trade-off, we must revisit the fundamental reason we use multi-head attention in the first place.

Let's consider the following ambiguous sentence:

> *"The artist painted the portrait of a woman with a brush."*

This sentence has at least two possible interpretations:

1. Interpretation A (Instrument): The artist used a brush to paint the portrait. (painted with a brush)
2. Interpretation B (Attribute): The portrait is of a woman who is holding a brush. (woman with a brush)

A sophisticated language model needs to be able to understand and disentangle both of these potential relationships simultaneously. This is precisely what Multi-Head Attention (MHA) was designed to do.

**How MHA Handles Ambiguity?**

In a standard MHA block, each attention head is an independent "expert analyst" with its own set of learned weights (Wk and Wv). This independence allows them to specialize. During training, the model might learn the following:

- **Head 1** could specialize in syntactic relationships. Its learned weights might cause its Key and Value vectors to focus on verb-instrument pairings. When its Query for the token "painted" looks at the Key for "brush", it would calculate a very high attention score, effectively capturing the meaning: "The action of painting was done using a brush."
- **Head 2**, on the other hand, could specialize in semantic or descriptive relationships. Its unique weights might cause its Key and Value vectors to focus on noun-attribute pairings. When its Query for "woman" looks at the Key for "brush", it might calculate a high score, capturing the alternative meaning: "The woman in the portrait is associated with a brush."

Because K1 is different from K2, and V1 is different from V2, the model can process both interpretations in parallel. The final context vectors contain a rich, blended understanding of all the different relationships detected by all the heads. This is the source of MHA's expressive power.

**How MQA Loses This Power?**

Now, let's consider what happens in Multi-Query Attention. MQA forces all heads to share a single, common Key and Value matrix. K1 is now identical to K2, and V1 is identical to V2.

This creates a critical problem. The single, shared Key matrix can no longer specialize. It must try to be a jack-of-all-trades, encoding a generic representation of the sentence's information. It cannot be an expert at understanding two different kinds of relationships at the same time. For example, it struggles to precisely capture both that the brush is a tool for painting and that it is an object held by the woman.

When Head 1 (the syntax expert) and Head 2 (the semantics expert) both send out their unique queries, they are both looking at the exact same,

generic set of keys. The shared Key for "brush" cannot effectively signal both "I am an instrument for painting" and "I am an object held by a woman" at the same time. One of these nuances will likely be weakened or lost entirely.

This is the fundamental drawback of MQA:

> By forcing all heads to share the same Key and Value representations, MQA severely restricts their ability to specialize. The model loses a significant amount of its capacity to capture diverse and subtle relationships within the text, leading to a degradation in overall performance.

While MQA is a brilliant solution for the memory problem, it achieves this by fundamentally compromising the core strength of the multi-head design. This is why it's often seen as a "memory-first" approach. This significant performance trade-off is what led researchers to seek a more balanced middle ground, which we will explore later with Grouped-Query Attention. But first, let's see how we can implement this memory-saving MQA architecture in code.

## 2.6.4 Implementing an MQA layer from scratch

Implementing Multi-Query Attention in PyTorch is straightforward. The core logic of the attention calculation remains the same; the only change is in how the Key and Value projections are handled. Instead of creating n_heads different projections, we create only one and then repeat it for all heads.

The following code defines a MultiQueryAttention module. Pay close attention to the __init__ method, where the architectural difference is most apparent.

**Listing 2.3 Implementing an MQA layer from scratch**

```
import torch
import torch.nn as nn

class MultiQueryAttention(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.0):
```

```
        super().__init__()
        assert d_model % num_heads == 0, \
            "d_model must be divisible by num_heads"

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_head = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)     #A
        self.W_k = nn.Linear(d_model, self.d_head) #B
        self.W_v = nn.Linear(d_model, self.d_head) #B
        self.W_o = nn.Linear(d_model, d_model)

        self.dropout = nn.Dropout(dropout)
        self.register_buffer('mask', torch.triu(
        ➥ torch.ones(1, 1, 1024, 1024), diagonal=1))

    def forward(self, x):
        batch_size, seq_len, _ = x.shape

        q = self.W_q(x).view(batch_size, seq_len,
self.num_heads,
        ➥ self.d_head).transpose(1, 2)

        k = self.W_k(x).view(batch_size, seq_len, 1,
        ➥ self.d_head).transpose(1, 2)
        v = self.W_v(x).view(batch_size, seq_len, 1,
        ➥ self.d_head).transpose(1, 2)

        k = k.repeat(1, self.num_heads, 1, 1)  #C
        v = v.repeat(1, self.num_heads, 1, 1)  #C

        attn_scores = (q @ k.transpose(-2, -1)) / (self.d_head
** 0.5)

        attn_scores = attn_scores.masked_fill(
        ➥ self.mask[:,:,:seq_len,:seq_len] == 0, float('-inf'))

        attn_weights = torch.softmax(attn_scores, dim=-1)
        attn_weights = self.dropout(attn_weights)

        context_vector = (attn_weights @ v).transpose(1, 2) \
        ➥ .contiguous().view(batch_size, seq_len, self.d_model)

        output = self.W_o(context_vector)
        return output
```

Let's break down the key differences between this MultiQueryAttention module and a standard MultiHeadAttention module:

- **Key and Value Projections:** In a standard MHA, the output dimension of W_k and W_v would be d_model. Here, they project down to d_head, the size of a single head. This is because we are only creating one projection, not num_heads projections that are later split.
- **Repeating K and V:** The magic of MQA happens in the forward pass. After computing the single Key and Value projections, we use the .repeat() method. This doesn't actually copy data in memory in the same way as a full matrix would; instead, it creates a "view" of the data where the same Key and Value tensors are presented to each of the num_heads Query heads. This is how "sharing" is implemented efficiently.
- **Efficiency Gain:** The primary gain comes from the reduced size of the Key and Value caches. In an MHA implementation, we would need to cache tensors of shape (batch_size, num_heads, seq_len, d_head) for both Keys and Values. In MQA, we only need to cache tensors of shape (batch_size, 1, seq_len, d_head), drastically reducing the memory footprint.

With this implementation, we have a functional attention layer that aggressively optimizes for memory, albeit at the cost of the performance trade-offs we've discussed. This sets the stage perfectly for exploring a more balanced solution.

# 2.7 The middle ground: Grouped-Query Attention (GQA)

This trade-off sacrificing model expressivity for memory efficiency is not ideal. It led researchers to seek a more balanced approach, a technique that could offer substantial memory savings without completely dismantling the power of the multi-head design. This solution is Grouped-Query Attention (GQA).

GQA provides a pragmatic compromise between the high expressivity of MHA and the significant memory efficiency of MQA. It lies somewhere in

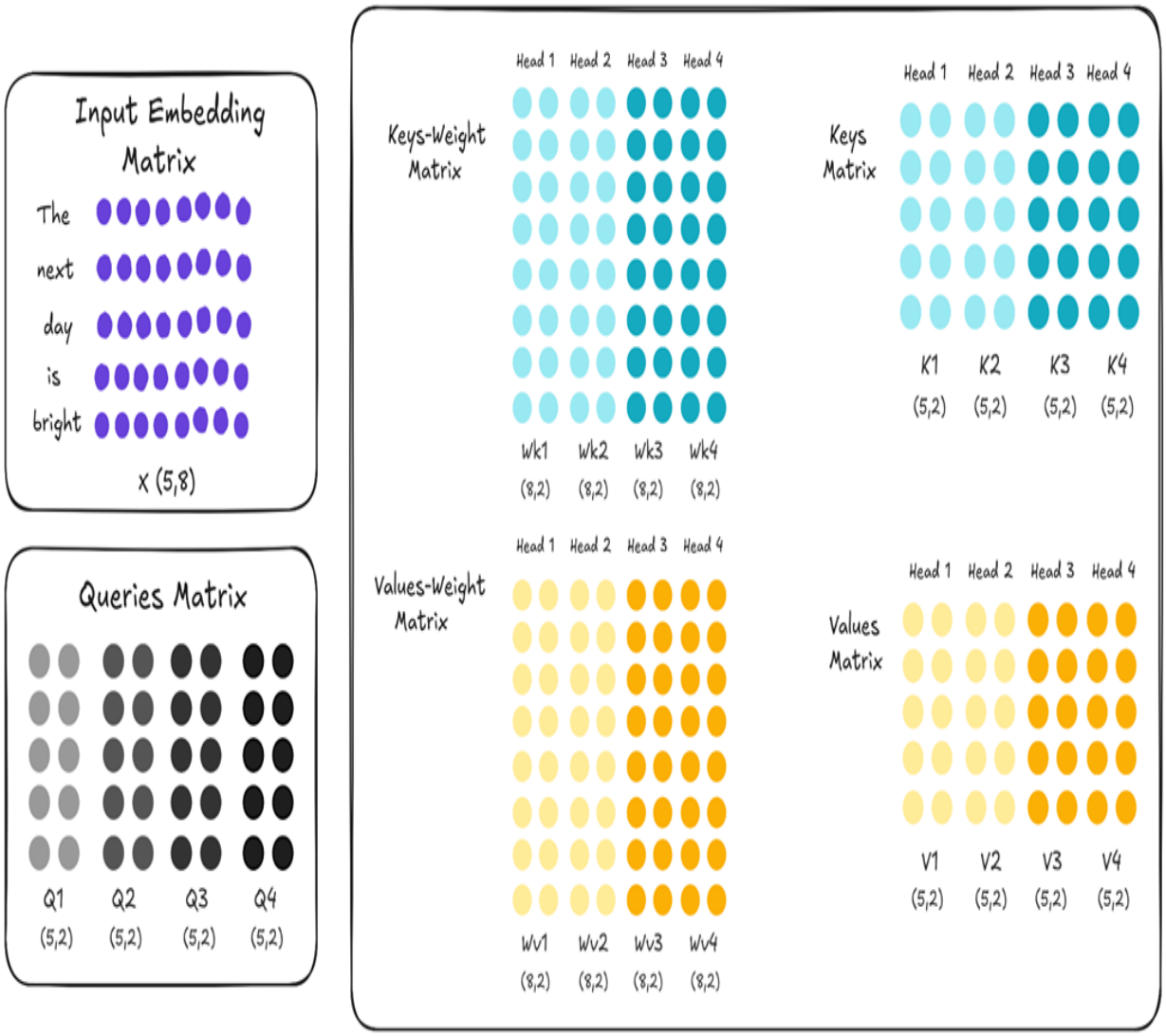
the middle, offering a tunable knob to balance these competing priorities.

### 2.7.1 The core idea: Sharing keys and values within groups

The core idea of Grouped-Query Attention is simple yet effective: instead of forcing all attention heads to share the same Key and Value matrices, what if we create groups of attention heads and only share the Keys and Values within those groups?

Let's visualize what this means. In our four-head example, instead of treating all four heads as one single unit (like in MQA), we can partition them into two groups.

**Figure 2.25 Grouped-Query Attention (GQA). The four attention heads are divided into two groups. Within Group 1 (light blue/light yellow), Head 1 and Head 2 share the same Key and Value projections. Within Group 2 (dark blue/dark yellow), Head 3 and Head 4 share a different, distinct set of Key and Value projections.**




As figure 2.25 illustrates, we've created a hybrid model:

**Within Group 1:** Head 1 and Head 2 share the same Wk and Wv matrices. Their resulting K1 and K2 are identical, and V1 and V2 are identical.

**Within Group 2:** Similarly, Head 3 and Head 4 share their own set of Wk and Wv matrices, making K3 identical to K4, and V3 identical to V4.

**Between Groups:** Crucially, the Key/Value pair for Group 1 is different from the Key/Value pair for Group 2. The light blue K1/K2 matrices are distinct from the dark blue K3/K4 matrices.

This grouping strategy elegantly solves the main drawback of MQA. We are no longer forcing all heads to look at the same information. Now, Head 1 (in Group 1) and Head 3 (in Group 2) have different Key and Value matrices, allowing them to specialize and capture different perspectives, just like in standard MHA. We have reintroduced diversity into the system.

At the same time, we are still saving a significant amount of memory compared to MHA. Instead of caching four unique Key matrices, we only need to cache two: one for Group 1 and one for Group 2. GQA provides a middle ground, allowing us to find a sweet spot between model performance and memory cost.

### 2.7.2 The tunable knob: Balancing memory and performance

The introduction of groups in GQA provides a powerful "tunable knob" to balance the trade-off between memory efficiency and model expressivity. The number of groups, which we'll call g, directly controls this balance.

Let's revisit the KV Cache size formula.

- For **MHA**, the size scaled with n (the total number of attention heads).
- For **MQA**, the size scaled with 1 (a single shared K/V pair).
- For **GQA**, the size now scales with g (the number of unique groups).

The formula becomes:

Size_GQA = l * b * g * h * s * 2 * 2

This gives us a spectrum of possibilities:

- If we set the number of groups equal to the number of heads (g = n), GQA becomes identical to MHA. We have maximum performance and maximum memory usage.
- If we set the number of groups to one (g = 1), GQA becomes identical to MQA. We have maximum memory savings and the lowest performance.
- By choosing a value for g between 1 and n, we can find a practical middle ground.

For example, a model like Llama 3 8B has 32 total attention heads. Instead of the extremes of MHA (32 unique K/V pairs) or MQA (1 unique K/V pair), it uses GQA with 8 groups. This means that every 4 query heads share a single key/value head.

This reduces the KV cache size by a factor of 4 (from 32 to 8), offering a significant memory saving while retaining much more of the expressive power than MQA would. This balanced approach has made GQA a very popular choice in modern, open-source LLMs. It offers a practical way to manage the KV cache bottleneck without a crippling hit to the model's performance.

However, it is still fundamentally a compromise. We are trading some amount of model expressivity for a reduction in memory. While GQA is a clever and effective optimization, it doesn't solve the core tension between performance and memory; it just allows us to choose a better point on the trade-off curve.

This led the DeepSeek team to ask a different, more profound question: can we fundamentally change the nature of this trade-off? Is it possible to keep the full expressive power of having unique projections for every head (like in MHA) while also achieving significant memory reduction?

The answer to that question is yes, and the solution is Multi-Head Latent Attention. But before we start into that groundbreaking technique, let's solidify our understanding of GQA by implementing it from scratch.

## 2.7.3 Implementing a GQA layer from scratch

Implementing Grouped-Query Attention is a natural extension of our MQA code. The key difference is that instead of having one shared projection for Keys and Values, we now have num_groups of them. We then ensure that the query heads within each group attend to the corresponding key/value group.

The following listing implements a GroupedQueryAttention module. The key variable to watch is num_groups, which acts as the "tunable knob.” It directly controls the number of Key and Value projections, allowing us to balance memory savings and model performance.

**Listing 2.4 Implementing a GQA layer from scratch**

```
import torch
import torch.nn as nn

class GroupedQueryAttention(nn.Module):
    def __init__(self, d_model, num_heads, num_groups,
    ➥ dropout=0.0, max_seq_len: int = 0):
        super().__init__()
        assert d_model % num_heads == 0, \
            "d_model must be divisible by num_heads"
         assert num_heads % num_groups == 0, \
             "num_heads must be divisible by num_groups"

        self.d_model = d_model
        self.num_heads = num_heads
        self.num_groups = num_groups
        self.d_head = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model,
        ➥ self.num_groups * self.d_head) #A
        self.W_v = nn.Linear(d_model,
        ➥ self.num_groups * self.d_head) #A
        self.W_o = nn.Linear(d_model, d_model)

        self.dropout = nn.Dropout(dropout)
        # Optional causal mask pre-allocation logic...
        self._register_mask_buffer(max_seq_len)

    def forward(self, x):
        B, T, _ = x.shape

        q = self.W_q(x).view(B, T, self.num_heads,
        ➥ self.d_head).transpose(1, 2)
        k = self.W_k(x).view(B, T, self.num_groups,
        ➥ self.d_head).transpose(1, 2) #B
        v = self.W_v(x).view(B, T, self.num_groups,
        ➥ self.d_head).transpose(1, 2) #B

        heads_per_group = self.num_heads // self.num_groups
     k = k.repeat_interleave(heads_per_group, dim=1) #C
     v = v.repeat_interleave(heads_per_group, dim=1) #C

        # ... rest of attention calculation ...
        attn_scores = (q @ k.transpose(-2, -1)) *
(self.d_head**-0.5)
```

```
        causal_mask = self._get_causal_mask(T, x.device)
        attn_scores = attn_scores.masked_fill(causal_mask,
float("-inf"))
        attn_weights = torch.softmax(attn_scores, dim=-1)
        attn_weights = self.dropout(attn_weights)

        context = (attn_weights @ v).transpose(1,
2).contiguous() \
        ➥ .view(B, T, self.d_model)

        return self.W_o(context)

    # Helper methods for mask management
    def _register_mask_buffer(self, max_seq_len):
        if max_seq_len > 0:
            mask = torch.triu(torch.ones(1, 1, max_seq_len,
max_seq_len,
            ➥ dtype=torch.bool), diagonal=1)
            self.register_buffer("causal_mask", mask,
persistent=False)
        else:
            self.causal_mask = None

    def _get_causal_mask(self, seq_len, device):
        if self.causal_mask is not None and \
        ➥ self.causal_mask.size(-1) >= seq_len:
            return self.causal_mask[:, :, :seq_len, :seq_len]
        return torch.triu(torch.ones(1, 1, seq_len, seq_len,
        ➥ dtype=torch.bool, device=device), diagonal=1)
```

This implementation provides the "tunable knob" we discussed. By simply changing the num_groups argument, we can move seamlessly along the spectrum from MQA-like behavior (num_groups=1) to MHA-like behavior (num_groups=num_heads).

# 2.8 The performance vs. memory trade-off

We have now explored the first generation of solutions to the KV Cache memory crisis: Multi-Query Attention (MQA) and Grouped-Query Attention (GQA). Both techniques offer a significant reduction in the memory footprint of the KV cache, making it possible to run larger models with longer context lengths on existing hardware.

However, they both operate on the same fundamental principle: they save memory by reducing the number of unique Key and Value projections.

- **MQA** is the most extreme case, collapsing n unique K/V heads down to just one.
- **GQA** offers a more moderate compromise, collapsing n heads into g groups.

While effective, this is ultimately a trade-off. We are sacrificing the expressive power that comes from having fully independent, specialized attention heads in order to save memory. GQA allows us to choose a more palatable point on the performance-vs-memory curve, but it doesn't change the curve itself. We are still forced to choose between maximum performance (MHA) and maximum memory efficiency (MQA), or a compromise in between (GQA).

This unresolved tension is what makes the DeepSeek architecture so innovative. The developers asked a different question: instead of reducing the number of heads, can we make the information within each head more compact? Can we compress the Key and Value matrices themselves?

This shift in thinking from **reducing heads** to **compressing information** is the conceptual leap that leads directly to Multi-Head Latent Attention, which we'll discuss in chapter 3. It represents a fundamentally new approach to solving the KV Cache bottleneck, one that aims to preserve the full expressive power of MHA while still achieving dramatic memory savings.

# 2.9 Summary

- Autoregressive generation, where each new token is appended to the input, results in the re-processing of the entire sequence at every step in a naive implementation.
- This repeated computation leads to a quadratic ($O(n^2)$) complexity problem, which makes generating long sequences of text computationally impractical.
- The Key-Value (KV) Cache optimizes inference by storing the Key and Value matrices of past tokens, avoiding redundant calculations and

transforming the process into a linear-time (O(n)) operation.

- While the KV Cache dramatically accelerates computation, it introduces a severe memory bottleneck, as its size grows proportionally with sequence length, number of layers, and attention heads.
- Multi-Query Attention (MQA) drastically reduces the KV cache's memory footprint by forcing all attention heads to share a single Key and Value projection, but this significantly degrades model performance by preventing head specialization.
- Grouped-Query Attention (GQA) provides a tunable compromise by having groups of attention heads share Key and Value projections, allowing a balance between memory savings and model expressivity.
- Architectures like MQA and GQA fundamentally operate by reducing the number of unique Key/Value pairs, establishing an inherent trade-off between memory efficiency and the expressive power of the model.

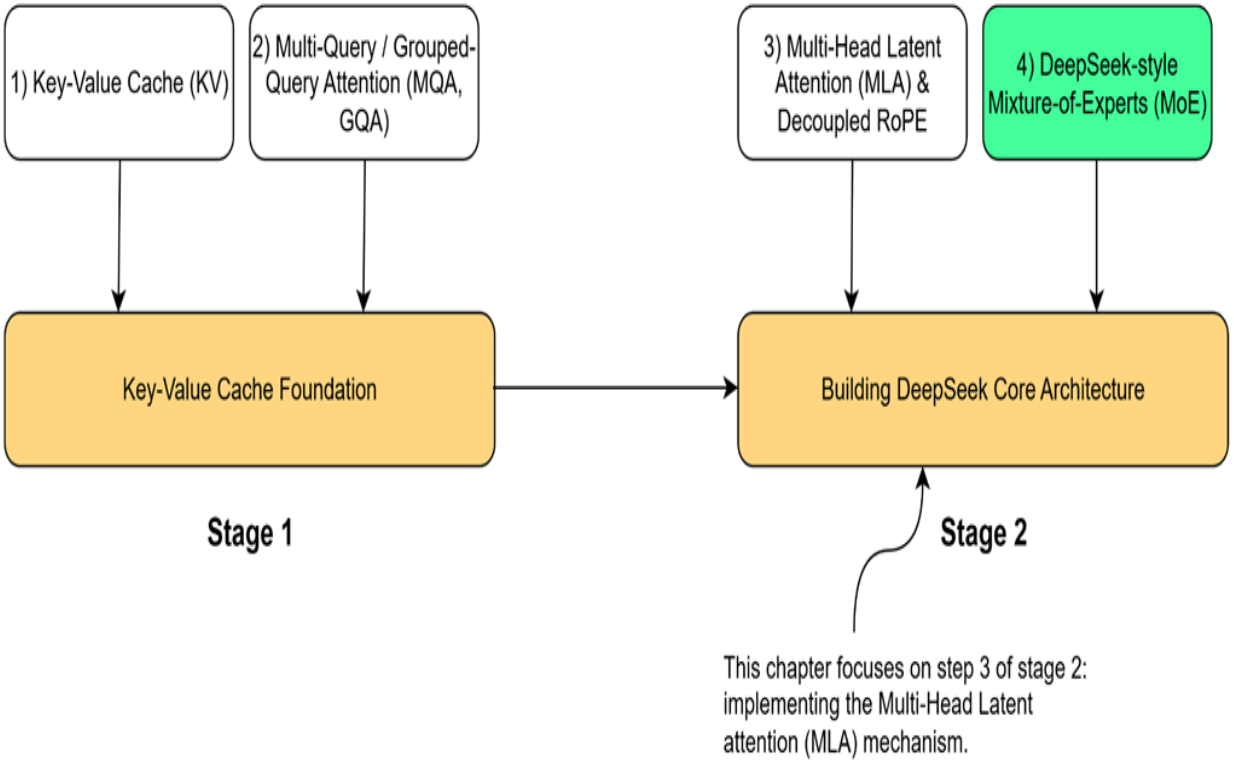
# 3 The DeepSeek breakthrough: Multi-Head Latent Attention (MLA)

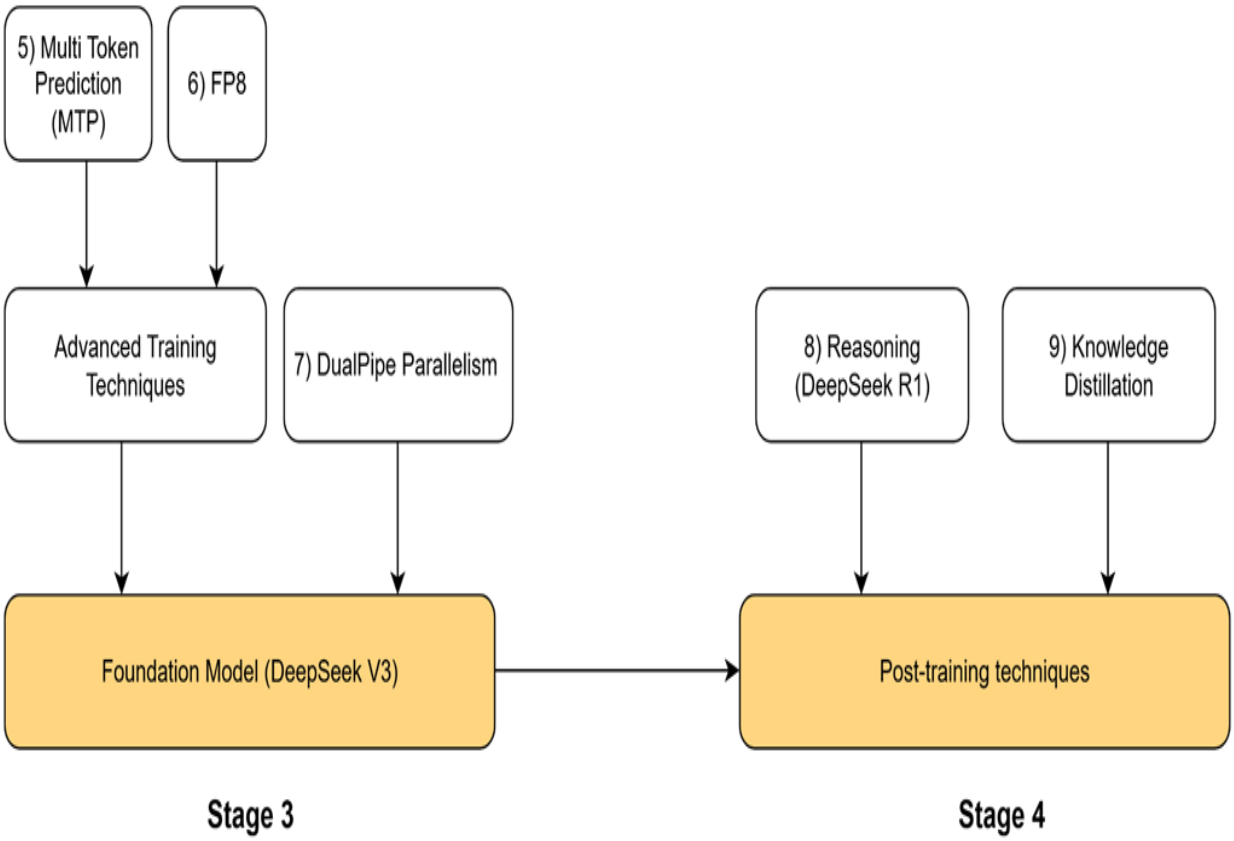
## This chapter covers

- Compressing the KV Cache with Multi-Head Latent Attention (MLA)
- Injecting positional awareness with Rotary Positional Encoding (RoPE)
- Fusing MLA and RoPE with a decoupled architecture

In our last chapter, we completed Stage 1 of our journey by building a solid foundation in efficient LLM inference. We began with the problem of repeated calculations, which we solved with the KV Cache. However, we then saw the dark side of the KV Cache: its massive memory cost. We explored the first-generation solutions, MQA and GQA, which help with memory usage but introduce a painful trade-off by sacrificing the expressive power of Multi-Head Attention (MHA). This left us with an unresolved tension between performance and efficiency.

**Figure 3.1 Our four-stage journey to build the DeepSeek model. Having completed the Key-Value Cache Foundation (Stage 1), we now begin Stage2. This chapter focuses on the highlighted component, Multi-Head Latent Attention (MLA) & Decoupled RoPE, the first major innovation in the core architecture.**


1) Key-Value Cache (KV)
2) Multi-Query / Grouped-Query Attention (MQA, GQA)
3) Multi-Head Latent Attention (MLA) & Decoupled RoPE
4) DeepSeek-style Mixture-of-Experts (MoE)
Key-Value Cache Foundation
Building DeepSeek Core Architecture
Stage 1
Stage 2
This chapter focuses on step 3 of stage 2: implementing the Multi-Head Latent attention (MLA) mechanism.


5) Multi Token Prediction (MTP)
6) FP8
Advanced Training Techniques
7) DualPipe Parallelism
8) Reasoning (DeepSeek R1)
9) Knowledge Distillation
Foundation Model (DeepSeek V3)
Post-training techniques
Stage 3
Stage 4

As highlighted in our roadmap figure 3.1, the first architectural piece we will build is Multi-Head Latent Attention (MLA). This is the core innovation DeepSeek pioneered to break the trade-off between performance and memory. First introduced in DeepSeek-V2 and carried forward to its successors, MLA proved highly effective at dramatically reducing the KV Cache footprint. However, solving the memory problem is only half the battle. To understand context, our model also needs positional awareness, which we will provide using the state-of-the-art technique, Rotary Positional Encoding (RoPE).

This sets up the central challenge of this chapter: standard RoPE is fundamentally incompatible with MLA. To resolve this conflict, we will build a complete, production-ready attention block from the ground up, implementing the key DeepSeek innovations step-by-step.

**First,** we will implement Multi-Head Latent Attention (MLA). By coding the down-projection and up-projection layers, we will demonstrate how MLA dramatically reduces the KV Cache's memory footprint while preserving the expressive power of standard Multi-Head Attention.

**Second,** we will tackle positional awareness by building the modern solution, Rotary Positional Encoding (RoPE). We'll start by exploring the flaws in simpler approaches to understand why RoPE is effective, then implement its rotational mechanism from scratch.

**Finally,** we will combine these two concepts by implementing DeepSeek's Decoupled RoPE architecture. This involves constructing two parallel processing paths one for content and one for position and then adding their outputs. This final implementation will provide a functional PyTorch module that balances high performance with memory efficiency, forming the core of the DeepSeek model.

# 3.1 MLA: The best of both worlds

The state of attention mechanisms before DeepSeek presented a difficult choice. We were forced to navigate a trade-off curve: on one end, the maximum performance of Multi-Head Attention with its massive memory

cost; on the other, the memory efficiency of Multi-Query Attention with its compromised expressivity.

This led the DeepSeek team to ask a more profound question: **Can we break the trade-off itself?**

Is it possible to design an attention mechanism that delivers:

1. **Low Cache Size:** A memory footprint comparable to the highly efficient MQA or GQA.
2. **High Performance:** The full expressive power of MHA, where every attention head has a unique, specialized perspective.

It seems impossible. To reduce the cache size, it feels like we must reduce the amount of unique information we store. How could we possibly have different values for every head's Keys and Values without caching them all?

The answer lies in a beautiful trick, a new way of thinking about the problem. The core innovation of MLA is to shift the focus from reducing the number of heads to **compressing the information** within them.

The insight is this: what if we don't have to cache the Key and Value matrices separately? What if we could first project our input into a single, combined, and much smaller matrix, a **latent matrix** and cache only that?

This is the central idea of Multi-Head Latent Attention. Instead of caching two large, full-dimensional matrices (K and V), we cache one smaller, lower-dimensional matrix (cKV). This single latent matrix becomes our new, highly efficient cache. Then, when we need the full Key and Value matrices for the attention calculation, we can reconstruct them on the fly from this compressed latent representation.

This is the magic of MLA:

- **For Caching**: We store a single, small, compressed matrix.
- **For Calculation**: We decompress it to get full-sized, unique Key and Value matrices for every head.

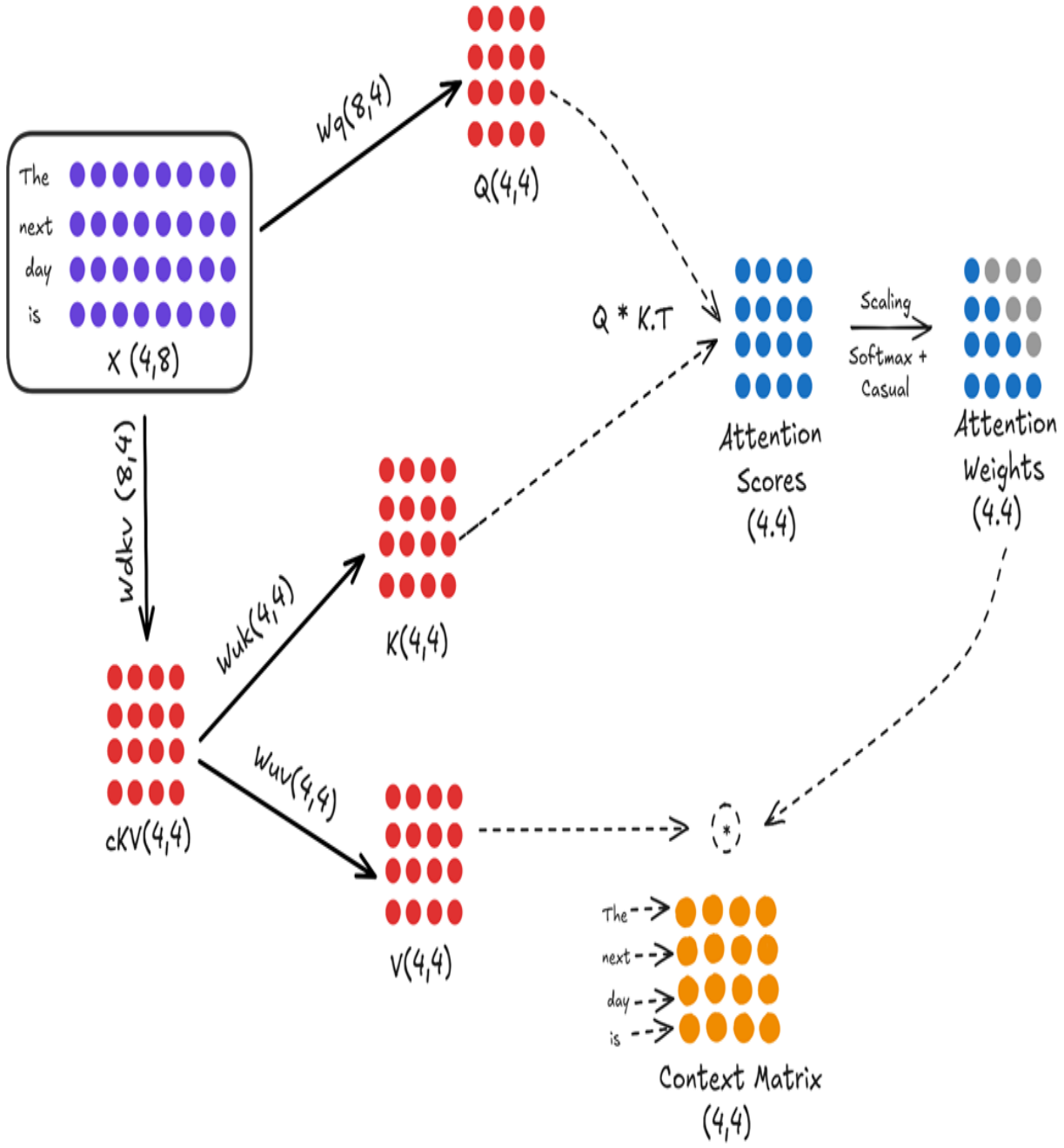
This approach promises to deliver the best of both worlds. It achieves this by cleverly **factorizing** the Key and Value projections into a two-step process: a compression to a smaller latent space for efficient caching, followed by a **reconstruction** back to the full-dimensional space for the attention calculation. To understand how it works, we must first look at the new architecture that makes this compression and decompression possible.

## 3.2 The MLA architecture: A visual walkthrough

To achieve this "compress for storage, decompress for use" strategy, Multi-Head Latent Attention introduces a new set of projection layers and modifies the data flow within the attention block. Instead of projecting the input directly into Keys and Values, it first projects the input into a compressed latent space.

Let's examine the complete workflow, as illustrated in figure 3.2.

**Figure 3.2 The full architectural data flow of Multi-Head Latent Attention (MLA).**




This diagram looks complex at first, but it's a logical extension of what we already know. Let's break it down into its two main paths: the query path and the key/value path.

### 3.2.1 The query path (unchanged)

The query path in this simplified version of MLA remains the same as in standard Multi-Head Attention. The goal is to create a full-sized query matrix that can "ask questions" of the entire context.

- The input embedding matrix `X` (shape 4, 8) is multiplied by the `Wq` weight matrix (shape 8, 4).
- This produces the final `Query` matrix `Q` (shape 4, 4), ready for the attention calculation. In this example, the head dimension (d_head) is 4.

(Note: More advanced versions of MLA also compress the query, but for understanding the core KV cache innovation, we can consider the query path to be standard).

## 3.2.2 The key/value path (the innovation)

This is where the magic happens. Instead of two separate projections for keys and values, there is now a two-step process involving a new, intermediate latent matrix.

### Step 1: Down-projection to the latent space

The input embedding matrix X is first multiplied by a new, learnable weight matrix called the **KV Down-Projector** `(Wdkv)`.

- X (shape 4, 8) is multiplied by Wd-kv (shape 8, 4).
- This produces a single, smaller, compressed matrix called the **Latent KV Matrix** (`cKV`, with a shape of (4, 4).

#### Note

In our example, the **latent dimension** (4) happens to be the same as the number of tokens. This is purely a coincidence for the sake of simple diagrams. In practice, these dimensions are completely independent and much different. For a model like **DeepSeek-V3**, the **model dimension** is **7168** and the **latent dimension** is **512**.

So, the KV Down-Projector (Wdkv) would have a shape of (7168, 512), compressing a large 7168-dimensional vector into a much smaller 512-

dimensional one. The numbers in our example are kept small only to build intuition.

This `cKV` matrix is the **only thing that gets cached** during inference. Notice two things immediately:

- We are only caching **one matrix**, not two.
- The dimension of this matrix (4 in this example, but 576 in the real DeepSeek model) is **much smaller than the full dimension of the Keys and Values combined.**

**Step 2: Up-projection from the latent space**

Now that we have our compressed cKV matrix, we can reconstruct the full Key and Value matrices from it whenever we need them for the attention calculation. This is done with two new **Up-Projection** matrices:

- The `cKV` matrix (shape 4, 4) is multiplied by the **Key Up-Projector** (`Wuk` (shape 4, 4)) to produce the final Key matrix K (shape 4, 4).
- The cKV matrix (shape 4, 4) is also multiplied by the **Value Up-Projector** (`Wuv` (shape 4, 4)) to produce the final `Value` matrix `V` (shape 4, 4).

After this two-step process, we have our three required matrices: `Q` (from the standard path), and `K` and `V` (reconstructed from the latent matrix). From this point forward, the rest of the attention calculation, computing scores, applying softmax, and calculating the context matrix proceeds exactly as it does in standard Multi-Head Attention.

The efficiency of this method comes from a two-step process. First, we store the historical Key and Value information in a single, compressed matrix to save memory. Second, we decompress this matrix on the fly to reconstruct the full, unique Key and Value matrices for each head right when they are needed for calculation. This raises a key question: how can adding projection and reconstruction steps actually make the process more efficient? The solution is found in a specific application of matrix multiplication that we'll refer to as the "absorption trick."

# 3.3 The mathematical magic: How the latent matrix helps

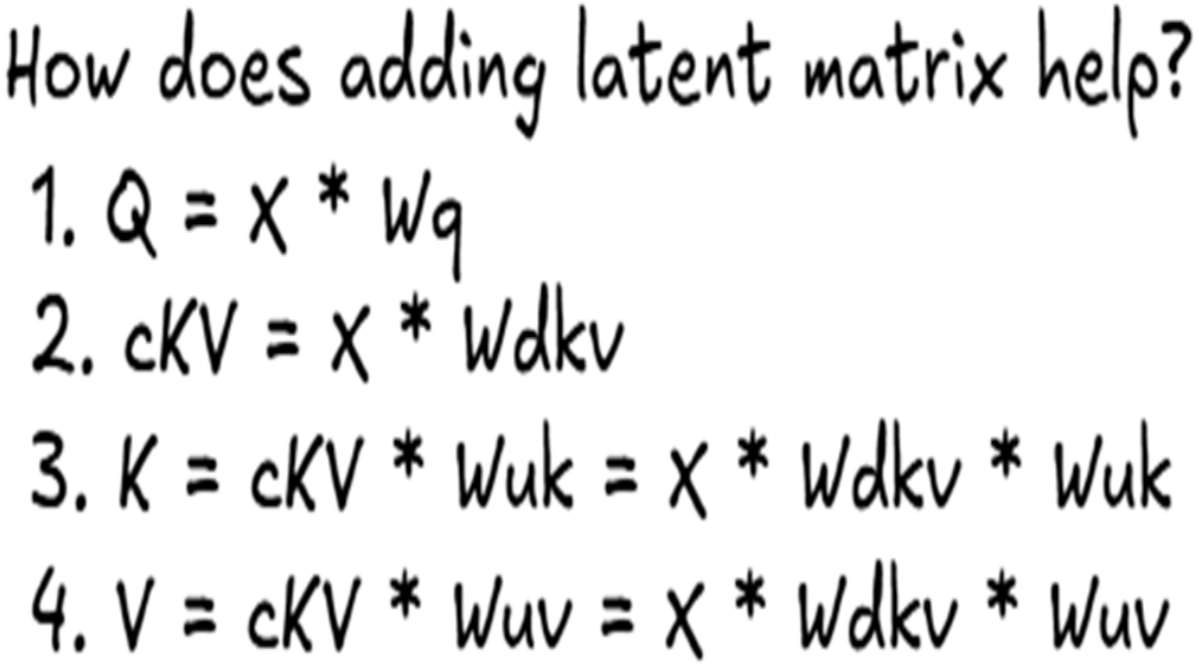
We have seen the new architecture of MLA, which introduces a latent `cKV` matrix. At first glance, this seems to add complexity. How does introducing an extra matrix and two extra projection steps (`Wd-kv` and `Wuk/Wuv`) actually lead to a more efficient system?

The answer lies in a property of matrix multiplication that the DeepSeek team exploited, a technique we'll call the "absorption trick." By cleverly rearranging the attention formula, we can show that the latent matrix is the only piece of historical information we need to cache, allowing us to have both a small cache and fully unique, expressive heads. To understand this, we need to write out the full sequence of calculations mathematically.

## 3.3.1 A Step-by-Step Derivation of Q, K, and V in MLA

Let's formalize the steps we saw in the architectural diagram. We will use X to represent our input embedding matrix.

**Figure 3.3 A step-by-step mathematical derivation of the Query, Key, and Value matrices in the MLA architecture.**




Following the steps in figure 3.3:

1. **The Query Matrix** (Q): The query calculation remains standard. It is a direct projection of the input embeddings.

$$Q = X * Wq$$

2. **The Latent KV Matrix** (cKV): This is the first new step. The input embeddings are projected down into the compressed latent space. This cKV matrix is what we will eventually cache.

$$cKV = X * Wdkv$$

3. **The Key Matrix** (K) The final Key matrix is no longer a direct projection of X. Instead, it's an up-projection of the latent matrix cKV.

$$K = cKV * Wuk$$

If we substitute the definition of cKV from step 2, we can see the full transformation from the original input:

$$K = (X * Wdkv) * Wuk$$

4. **The Value Matrix** (V) Similarly, the final Value matrix is also an up-projection of the same latent matrix cKV.

$$V = cKV * Wuv$$

And again, substituting the definition of cKV:

$$V = (X * Wdkv) * Wuv$$

So far, it still looks like we've just added extra steps. The key to understanding the efficiency gain comes when we plug these new definitions for K and V into the main attention score formula. This is where the magic begins.

### 3.3.2 The absorption trick: How attention scores are calculated

Now that we have the new mathematical definitions for our Query, Key, and Value matrices, let's see what happens when we plug them into the core formula for attention scores:

$$Attention\ Scores = Q * K^T$$

We start by substituting our new definitions for Q and K:

$$Q * K^T = (X * Wq) * ((X * Wdkv) * Wuk)^T$$

Using the rule of matrix transposition $(AB)^T = B^T A^T$, we can expand the transposed term:

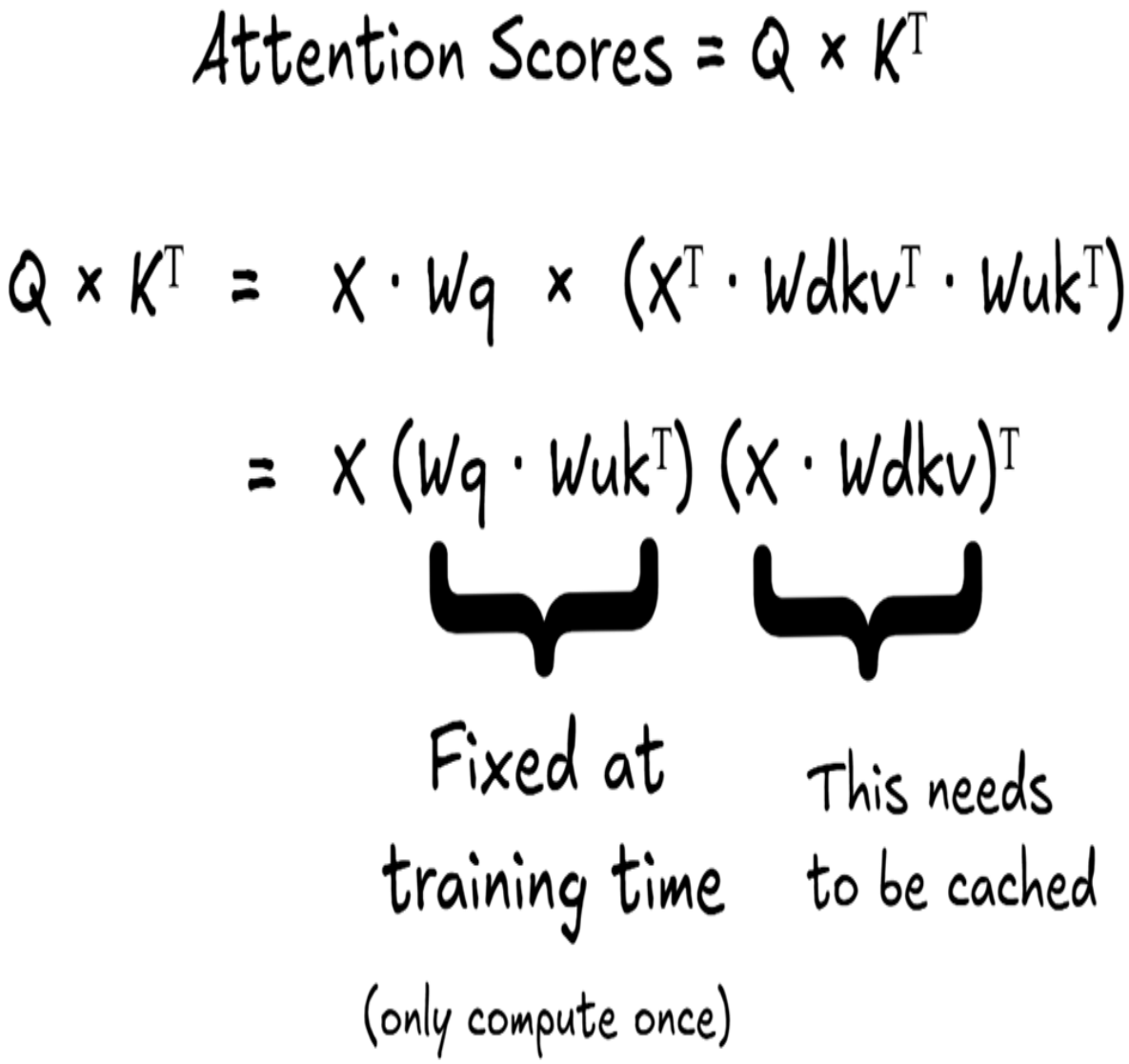
$$Q * K^{T} = (X * Wq) * (Wuk^{T} * (X * Wdkv)^{T})$$

And applying the rule again:

$$Q * K^{T} = (X * Wq) * (Wuk^{T} * Wdkv^{T} * X^{T})$$

This equation looks complicated, but it reveals something incredible. Notice that the trainable weight matrices (Wq, $Wuk^{T}$, $Wdkv^{T}$) are all grouped together in the middle. Because matrix multiplication is associative, we can re-group the terms.

**Figure 3.4 The mathematical rearrangement of the attention score calculation in MLA.**




As shown in figure 3.4, we can rearrange the equation as follows:

$$Q * K^{T} = X * (Wq * Wuk^{T}) * (X * Wdkv)^{T}$$

Let's analyze the two main components of this rearranged formula:

1. This is the product of two of our learned weight matrices. Since `Wq` and `Wuk` are **fixed during inference** (they are learned during pre-training),

this entire product is also a fixed matrix. We can pre-compute it once and reuse it. It doesn't need to be calculated at every step.
2. This term should look familiar. `X * Wdkv` is exactly the definition of our **latent KV matrix,** `cKV`.

This is the "absorption trick." The up-projection matrix for the Keys, `Wuk`, has been mathematically rearranged to be next to the query projection matrix, `Wq`. Because both `Wq` and `Wuk` are fixed, learned matrices from pre-training, their product ($Wq * Wuk^T$) is also just a single, fixed matrix that we don't need to re-calculate during inference.

This simplifies the process. To get our attention scores, the original formula $Q * K^T$ now effectively becomes:

$$(Input\ X) * (A\ fixed, pre-computed\ matrix) * (The\ transpose\ of\ our\ latent\ cKV\ matrix)$$

This is a profound change. We started with a formula that required a full Query matrix and a full Key matrix. After this mathematical rearrangement, we see that the calculation only requires two things:

1. The original input `X`.
2. The latent matrix `cKV = X * Wdkv`.

Crucially, as we generate new tokens and our input sequence grows, the **matrix X representing this growing history also expands.** Consequently, the latent matrix cKV that summarizes this history must also expand by appending the new **token's latent vector.** This proves that the cKV matrix is the only piece of historical information that needs to be updated and stored to compute the attention scores. We have successfully reformulated the attention score calculation to depend only on this single, compact, cached matrix, eliminating the need to cache the full K and V matrices entirely.

### 3.3.3 The final step: Calculating the context vector

We have successfully reformulated the attention score calculation to rely on a single, cached latent matrix, cKV. Now, let's see if this efficiency holds for

the final step: creating the `Context Vector Matrix`.

The standard formula is:

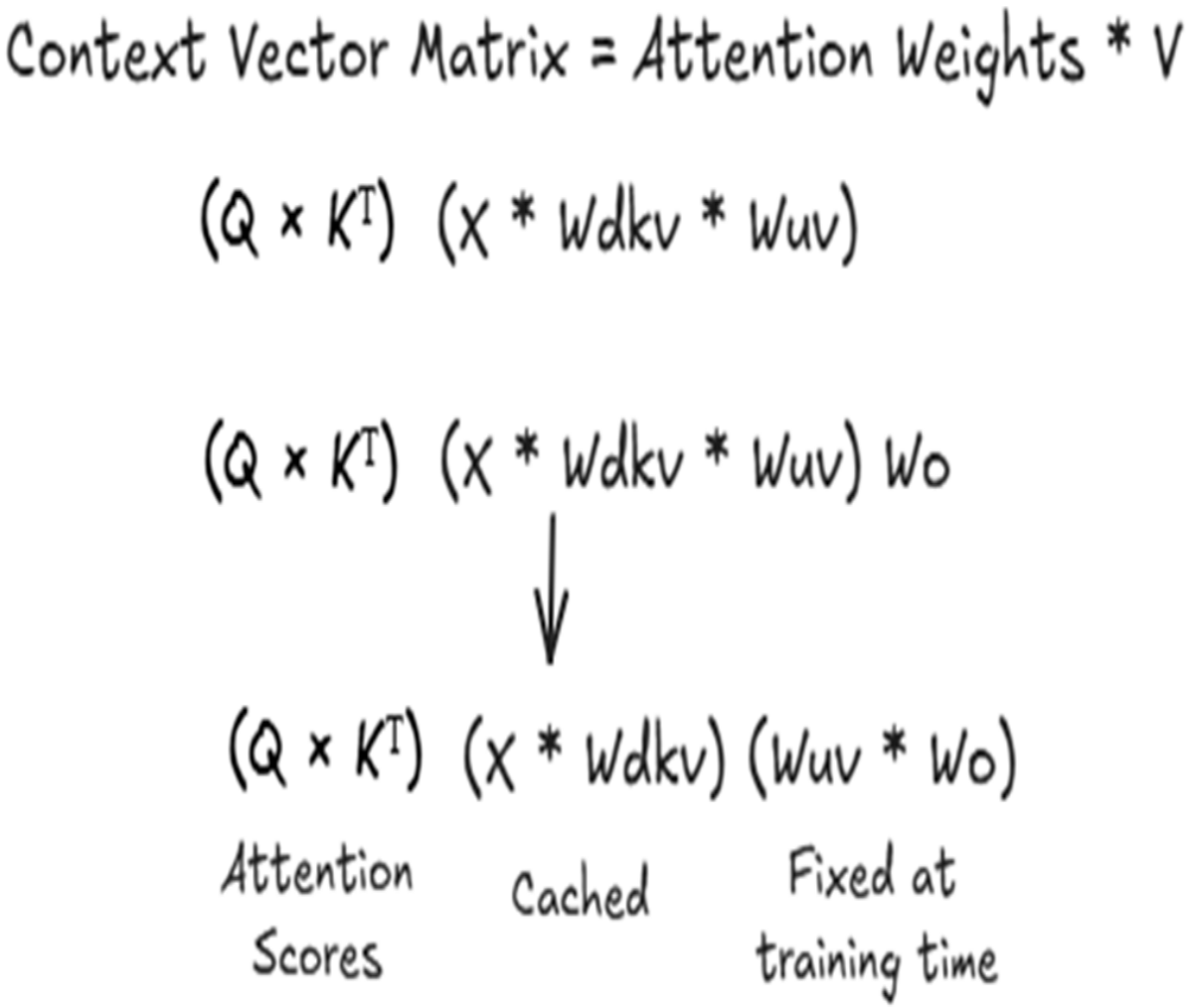
$$Context\ Vector\ Matrix\ =\ Attention\ Weights\ *\ V$$

We know that `Attention Weights` are derived from the attention scores ($Q * K^T$), which we just simplified. And we have our new definition for the `Value` matrix:

$$V\ =\ (X\ *\ Wdkv)\ *\ Wuv$$

Let's plug everything together.

**Figure 3.5 The mathematical rearrangement for calculating the final output in MLA.**




As figure 3.5 illustrates, the full calculation for the context vector, before it's passed to the final output projection layer (Wo), looks like this:

$$(Attention\ Weights) * ((X * Wd - kv) * Wuv)$$

Again, we can recognize the term (`X * Wdkv`). This is our **cached latent matrix.**

And just like before, we can use the associative property of matrix multiplication. The `Wuv` matrix can be "absorbed" by the final output projection layer, `Wo`. The product (`Wuv * Wo`) is just another single, fixed matrix that is learned during pre-training.

This confirms our central thesis: **we only need to cache the latent matrix.**

Let's summarize the beauty of this workflow:

1. When a new token arrives, we compute its contribution to the cKV cache.
2. We use the full, updated cKV cache to help compute the attention scores.
3. We also use that same, updated cKV cache to help compute the final context vector.

By introducing this intermediate latent space, the DeepSeek team designed a system where a single, compact cached matrix can be efficiently used for both the key and value sides of the attention calculation. This is how they achieved the best of both worlds: a dramatically smaller memory footprint for the cache, without sacrificing the performance benefit of having unique, expressive projections for every attention head.

# 3.4 The new inference loop with MLA

Now that we understand the mathematical magic behind MLA, let's put it all together and define the new, highly efficient workflow for generating a single new token. This process brilliantly leverages the cached latent matrix to avoid re-calculating the entire history.

We will trace the journey of a new token, "`bright,`" as it enters the system, assuming the model has already processed "`The next day is`" and has their latent representations stored in the cKV cache.

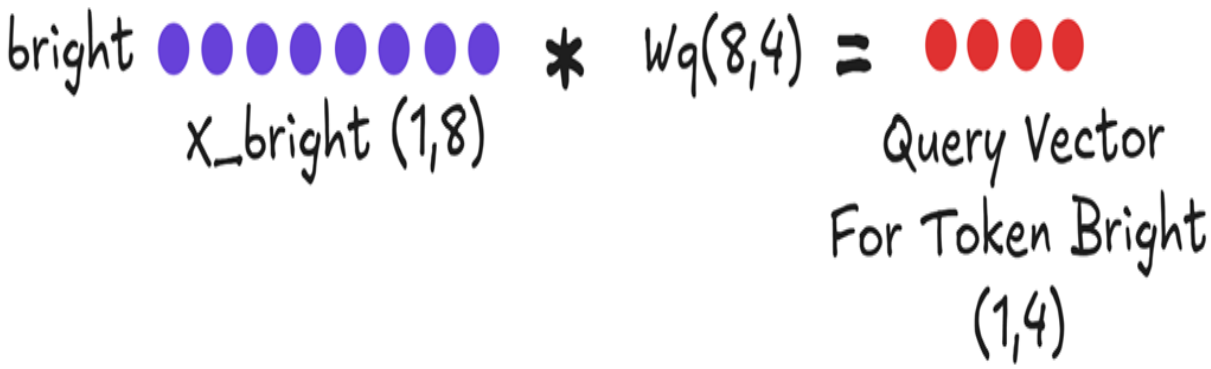
## 3.4.1 What happens when a new token arrives?

The entire process is designed to be incredibly efficient, performing the absolute minimum number of computations necessary. When a new token like "`bright`" arrives, the model follows these steps:

**Step 1: Compute the new projections**

The very first step is to compute the three essential vectors for our new token. We take the input embedding for "`bright`," which we'll call `X_bright`, and perform three parallel multiplications.

**Figure 3.6 Computing the Query vector for the new token.**

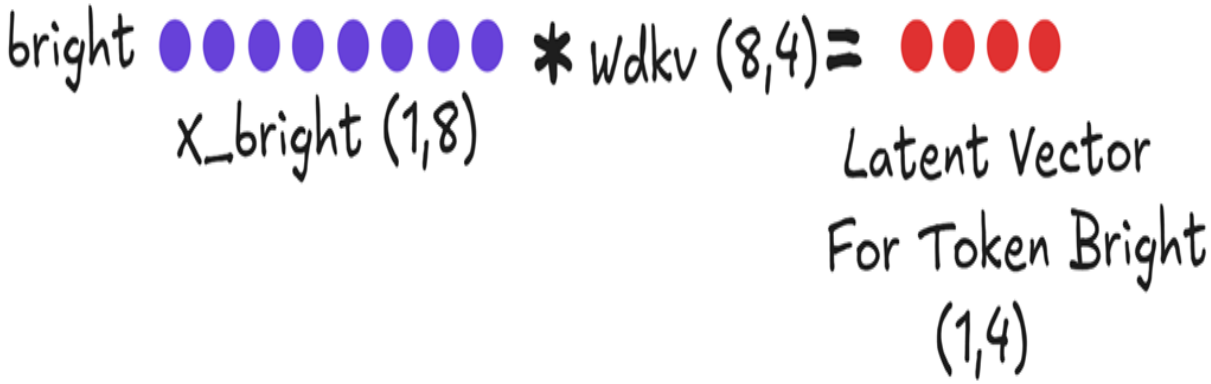



First, we compute the **Query Vector** for "`bright`". This is a standard projection and is always calculated fresh for the current token.

```
Query_bright = X_bright * Wq
```

Next, and most importantly, we compute the token's contribution to our new, compressed cache.

**Figure 3.7 Computing the new Latent Vector cKV for "bright."**



We multiply the input embedding `X_bright` by the **KV Down-Projector** (`Wdkv`). This produces a single, small, compressed **Latent Vector for Token Bright.** This tiny vector contains all the necessary Key and Value information for this token in a compressed format.

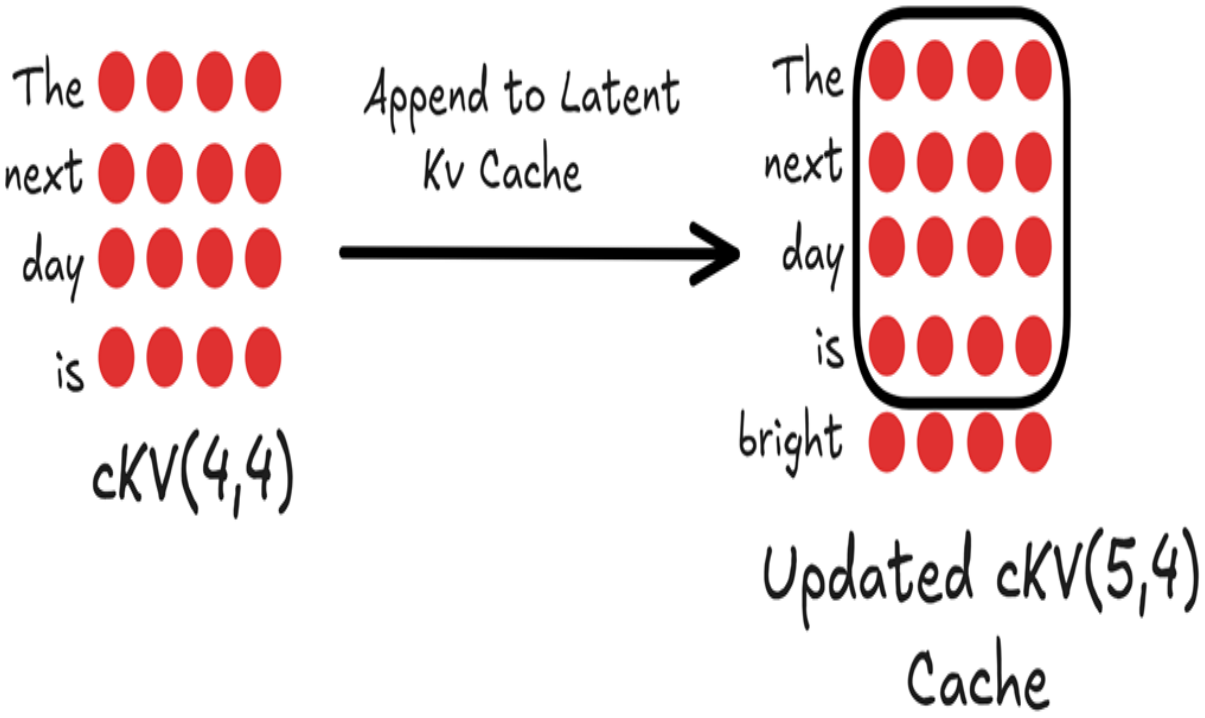
These are the only new projections we need to perform. We do *not* compute the full Key and Value vectors directly from the input. Instead, we will use our cache.

## 3.4.2 Caching the latent vector: The only thing we store

Now that we have the new `Latent Vector for Token Bright`, the next step is to update our cache. In the previous inference step (for the input "The next day is"), we would have already computed and stored the latent vectors for those tokens in our cKV cache.

Our task now is to simply append our new latent vector to this existing cache.

**Figure 3.8 Updating the Latent KV Cache. The newly computed latent vector for "bright" is appended to the existing cache for the previous tokens.**




As shown in figure 3.8, our original `cKV` cache had a shape of (4, 4), containing the compressed history. By appending the new (1, 4) latent vector, we now have an **Updated latent matrix** with a shape of (5, 4).

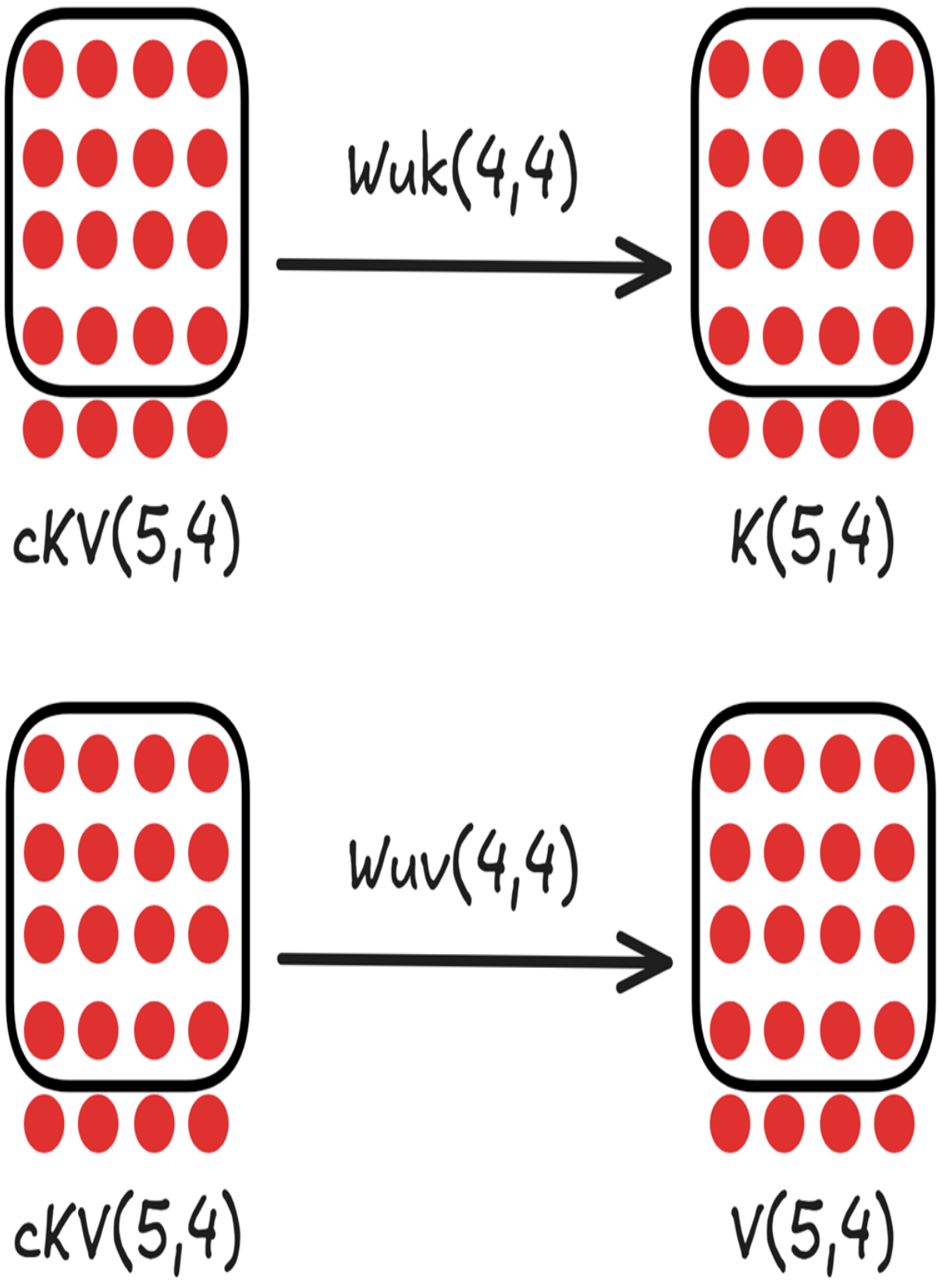
This single, compact matrix is the **only piece of historical information** we store in memory between steps. We do not need a separate cache for Keys and a separate cache for Values. This is the source of MLA's memory efficiency.

### 3.4.3 Decompressing the cache and calculating attention

With our updated cache and the fresh Query vector for "`bright`" we now have everything we need to perform the attention calculation.

First, we must "`decompress`" our latent cache back into the full-dimensional Key and Value matrices. This is done on the fly, just for this single calculation.

**Figure 3.9 Decompressing the updated latent cache into the full Key and Value matrices using the up-projection matrices.**


Wuk(4,4)
cKV(5,4)
K(5,4)
Wuv(4,4)
cKV(5,4)
V(5,4)

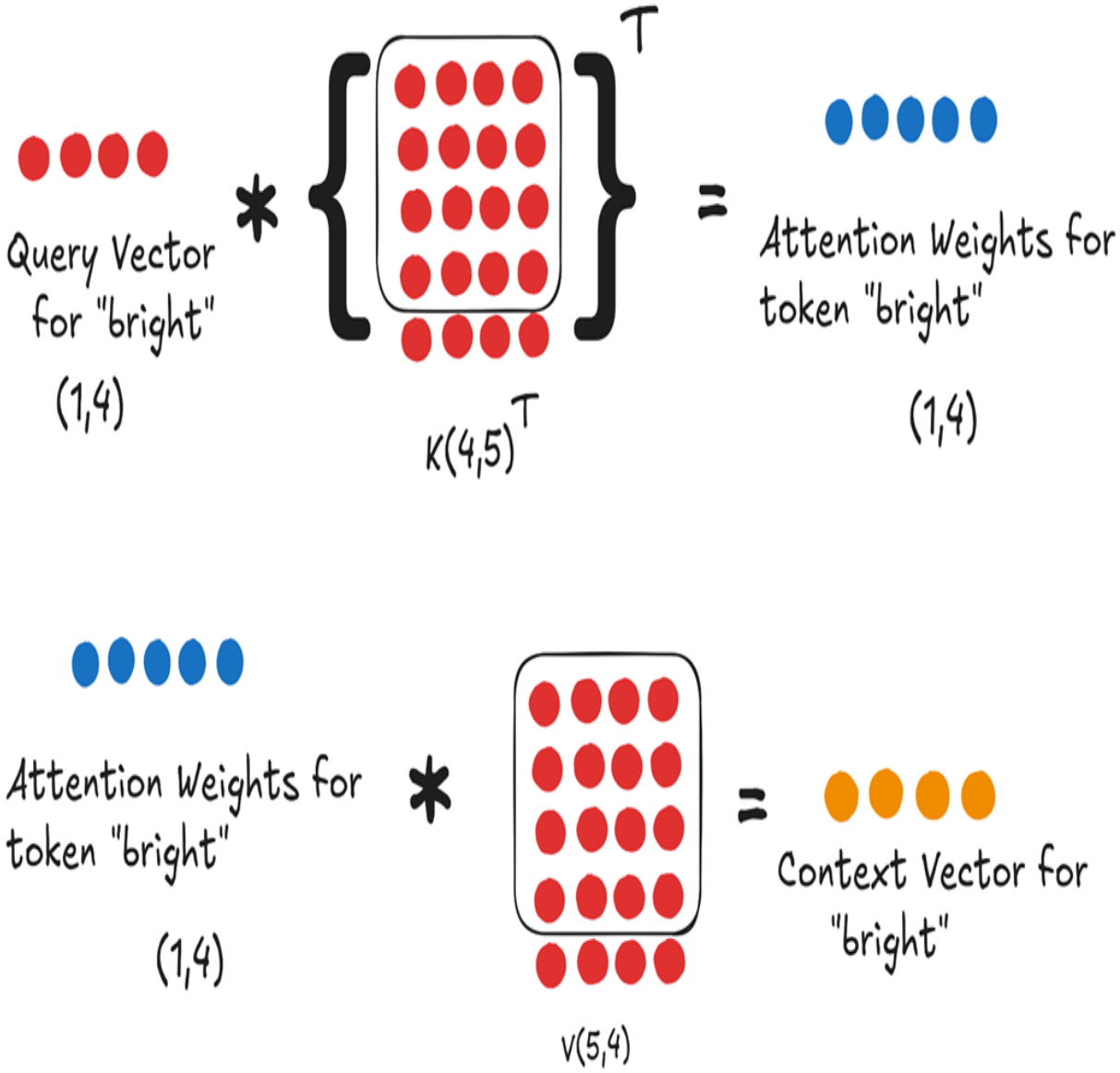
As shown in figure 3.9, we perform two parallel multiplications:

1. The `Updated cKV Cache` (shape 5, 4) is multiplied by the `Key Up-Projector` (Wuk) to produce the full Key Matrix K (shape 5, 4).
2. The same `Updated cKV Cache` (shape 5, 4) is multiplied by the `Value Up-Projector` (Wuv) to produce the full Value Matrix V (shape 5, 4).

Now, we have everything a standard attention mechanism needs: a fresh Query vector for our current token, and the full Key and Value matrices for the entire history, faithfully reconstructed from our compact cache.

The final steps proceed just as we've seen before.

**Figure 3.10 The final attention calculation using the fresh Query and the decompressed Key and Value matrices.**




1. **Calculate Attention Weights**: We multiply the `Query Vector for "bright"` by the transpose of the full, updated `Key Matrix K`. The result, after scaling and softmax, is the `Attention Weights for token "bright"`. This is the only row we need because, as we established in chapter 2, the prediction for the next token depends only on the context vector of the last token in the sequence.
2. **Compute Context Vector**: We multiply these `Attention Weights` by the full, updated `Value Matrix V` to produce the single, `final Context Vector for token "bright"`.

This single context vector is then passed through the rest of the Transformer's layers to predict the next token. We have successfully generated our output while only ever storing the small, latent `cKV` matrix in our cache.

# 3.5 Quantifying the gains

The multi-step process of MLA compressing, caching, and decompressing is a brilliant piece of engineering. But what does it achieve in practice? Let's quantify the two main benefits: the reduction in cache size and the preservation of performance.

## 3.5.1 The new KV cache formula: A 64x reduction

In chapter 2, we established the formula for the size of a standard MHA KV cache:

$$Size\_MHA = l * b * (n * h) * s * 2 * 2$$

The critical term was `2 * (n * h)`, representing the two separate caches (Keys and Values), each with a dimension equal to the number of heads (`n`) times the head dimension (`h`).

With MLA, this term is completely replaced. We no longer store two caches, and the dimension we store is not `n * h`, but the much smaller latent dimension, `d_latent`.

The new formula becomes:

$$Size\_MLA = l * b * d\_latent * s * 1 * 2$$

Let's plug in the numbers for a model at the scale of DeepSeek-V3:

- n (heads) = 128
- h (head dimension) = 128
- n * h (total dimension) = 16,384

- d_latent (latent dimension, as used in DeepSeek's paper) ≈ 512

Let's compare the core factors:

- **MHA Cache Dimension:** 2 * (n * h) = 2 * 16,384 = 32,768
- **MLA Cache Dimension:** d_latent = 512

The reduction in the size of the cached data is: 32,768 / 512 = 64 times.

This is an incredible achievement. By caching a single, compressed latent matrix, MLA reduces the memory footprint of the KV cache by a factor of 64. The theoretical 400 GB cache size we calculated for a DeepSeek-scale model collapses to a much more manageable **~7 GB**.

### 3.5.2 Preserving performance: Why head diversity is maintained

More importantly, MLA achieves this massive memory reduction without the performance degradation seen in MQA and GQA. Why?

The answer lies in the up-projection matrices, Wuk and Wuv. In the DeepSeek architecture, these up-projection matrices are themselves multi-headed. This means that there is a unique, learned Wuk and Wuv for every single attention head.

When we decompress the shared latent cache cKV, each head uses its own special up-projector. This means that:

- Head 1 reconstructs `K1` and `V1`.
- Head 2 reconstructs a different `K2` and `V2`.
- ...and so on for all 128 heads.

At the moment of the attention calculation, every head is working with its own unique Key and Value matrices, just like in standard MHA. We have not shared any content across the heads. We have preserved the full diversity and expressive power of the multi-head design. This is how MLA achieves the best of both worlds.

# 3.6 Building an MLA module from scratch

Now that we have a solid grasp of the theory and the mathematical workflow of MLA, we are ready to implement it in code. We will build a PyTorch module that encapsulates the entire logic: the down-projection, the up-projection, and the final attention calculation.

For this implementation, we will focus on the core MLA mechanism itself. We will handle the caching logic outside of this module in our main inference loop, and we will address positional encodings further in this chapter.

**Listing 3.1 Building MLA From Scratch**

```
import torch
import torch.nn as nn

class MultiHeadLatentAttention(nn.Module):
    def __init__(self, d_model, num_heads, d_latent,
    ➥ dropout=0.0):
        super().__init__()
        assert d_model % num_heads == 0, \
            "d_model must be divisible by num_heads"

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_head = d_model // num_heads
        self.d_latent = d_latent

        self.W_q = nn.Linear(d_model, d_model)  #A
        self.W_dkv = nn.Linear(d_model, d_latent)  #B

        self.W_uk = nn.Linear(d_latent, d_model) #C
        self.W_uv = nn.Linear(d_latent, d_model) #C

        self.W_o = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer('mask', torch.triu(torch.ones(
        ➥     1, 1, 1024, 1024), diagonal=1).bool())

    def forward(self, x):
        batch_size, seq_len, _ = x.shape
```

```
        # 1. Query Path
        q = self.W_q(x).view(
        ➥         batch_size, seq_len, self.num_heads,
self.d_head
        ➥     ).transpose(1, 2)

        # 2. Key/Value Path (The MLA Innovation)
        # Down-project to the latent space
        c_kv = self.W_dkv(x)  #D

        # Up-project from latent space to get full K and V
        k = self.W_uk(c_kv).view(
        ➥         batch_size, seq_len, self.num_heads,
self.d_head
        ➥     ).transpose(1, 2)  #E
        v = self.W_uv(c_kv).view(
        ➥         batch_size, seq_len, self.num_heads,
self.d_head
        ➥     ).transpose(1, 2)  #E

        # 3. Standard Attention Calculation
        attn_scores = (q @ k.transpose(-2, -1)) / \
        ➥         (self.d_head ** 0.5)

        attn_scores = attn_scores.masked_fill(
        ➥         self.mask[:, :, :seq_len, :seq_len], float('-
inf'))

        attn_weights = torch.softmax(attn_scores, dim=-1)
        attn_weights = self.dropout(attn_weights)

        context_vector = (attn_weights @ v).transpose(1, 2) \
        ➥         .contiguous().view(batch_size, seq_len,
self.d_model)

        # 4. Final Output Projection
        output = self.W_o(context_vector)
        return output
```

While Multi-Head Latent Attention optimizes the Transformer's ability to process information efficiently, our attention module is still fundamentally "time-blind." It has no inherent sense of word order, treating sentences as a "bag of words." For LLMs to truly understand context and generate coherent text, they need to know the position of each token. This leads us to the next crucial step on our path: positional awareness.

# 3.7 The problem of order

The attention mechanism, for all its power, has a fundamental limitation: it has no built-in sense of sequence order. By default, it treats a sentence as a "bag of words," where the position of a word doesn't matter. To the raw attention mechanism, the sentences "`the dog chased the cat`" and "`cat the chased dog the`" look identical. This is a huge problem, as the order of words is critical to the meaning of a sentence.

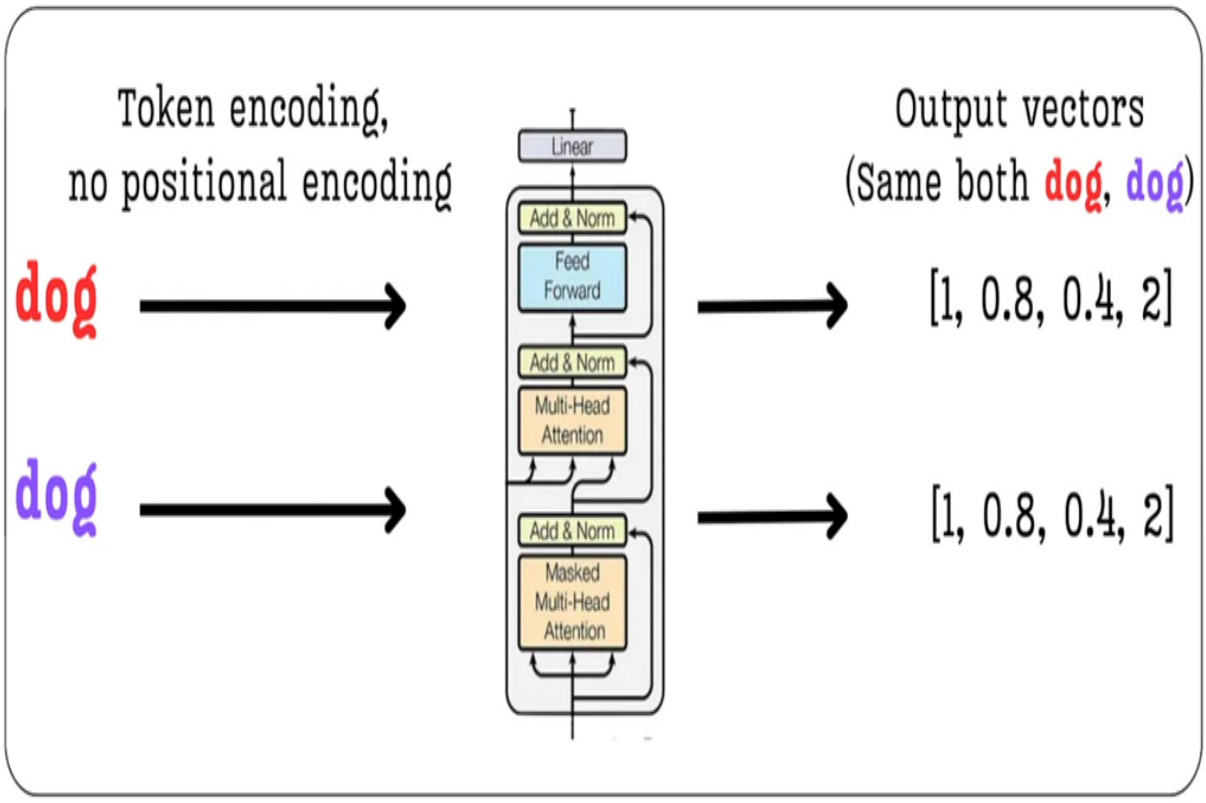
Let's consider an example:

> "The dog chased another dog."

This sentence contains two instances of the word "dog." Semantically, the token embedding for the first "dog" (the chaser) and the second "dog" (the one being chased) are identical. If we feed these token embeddings directly into the Transformer blocks without any positional information, the model has no way to distinguish between them.

As shown in figure 3.11, the input to the Transformer block for both tokens would be exactly the same. Consequently, the output context vectors produced by the attention mechanism would also be identical. The model would be blind to the fact that one dog is the subject and the other is the object. It cannot understand the true context of the sentence.

**Figure 3.11 Without positional information, the Transformer processes identical tokens in different positions in the exact same way, leading to identical context vectors and a loss of crucial contextual understanding.**




To solve this, we must find a way to tell the model where each token is located in the sequence. We need to create a distinction between the first "dog" at position 2 and the second "dog" at position 5. The mechanism for doing this is called **positional encoding**: we create a vector that represents a token's position and add it to the token's semantic embedding.

The challenge is to inject this positional information in a way that is both useful for the model and does not destroy the valuable semantic information contained in the original token embeddings. As we will see, this is a surprisingly difficult needle to thread. Let's start by exploring the most straightforward, intuitive solution one might think of first: simply using the integer position numbers themselves.

# 3.8 Attempt #1: The naive approach - integer positional encodings

Now that we've established the critical need for positional information, let's think from first principles. If you were tasked with designing a system to encode a token's position, what is the simplest, most direct method you could imagine? Most likely, you would suggest using the position number itself.

## 3.8.1 The simple idea: Using position numbers directly

This is the core idea behind Integer Positional Encoding. We simply take the integer index of a token in the sequence and use that value to create its positional embedding.

Let's return to our example: `"The dog chased another dog."`

- The first "dog" is at position 2.
- The second "dog" is at position 5.

To create a positional embedding vector that we can add to the token embedding, we need a vector of the same dimension. If our token embeddings have a dimension of 8, we would simply create a vector where every element is the position number.

- **Positional Embedding for the first "dog" (position 2):** `[2, 2, 2, 2, 2, 2, 2, 2]`
- **Positional Embedding for the second "dog" (position 5):** `[5, 5, 5, 5, 5, 5, 5, 5]`

When we add these vectors to the identical token embeddings for "dog", the resulting input embeddings fed into the Transformer will be different.

```
InputEmbedding_dog1 = TokenEmbedding_dog + [2, 2, 2, ...]
InputEmbedding_dog2 = TokenEmbedding_dog + [5, 5, 5, ...]
```

Problem solved, right? We are now successfully providing the model with a way to distinguish between the two dogs. The transformer will produce

different context vectors for them, and it can learn the different roles they play in the sentence.

However, this simple solution introduces a new, and much more severe, problem.

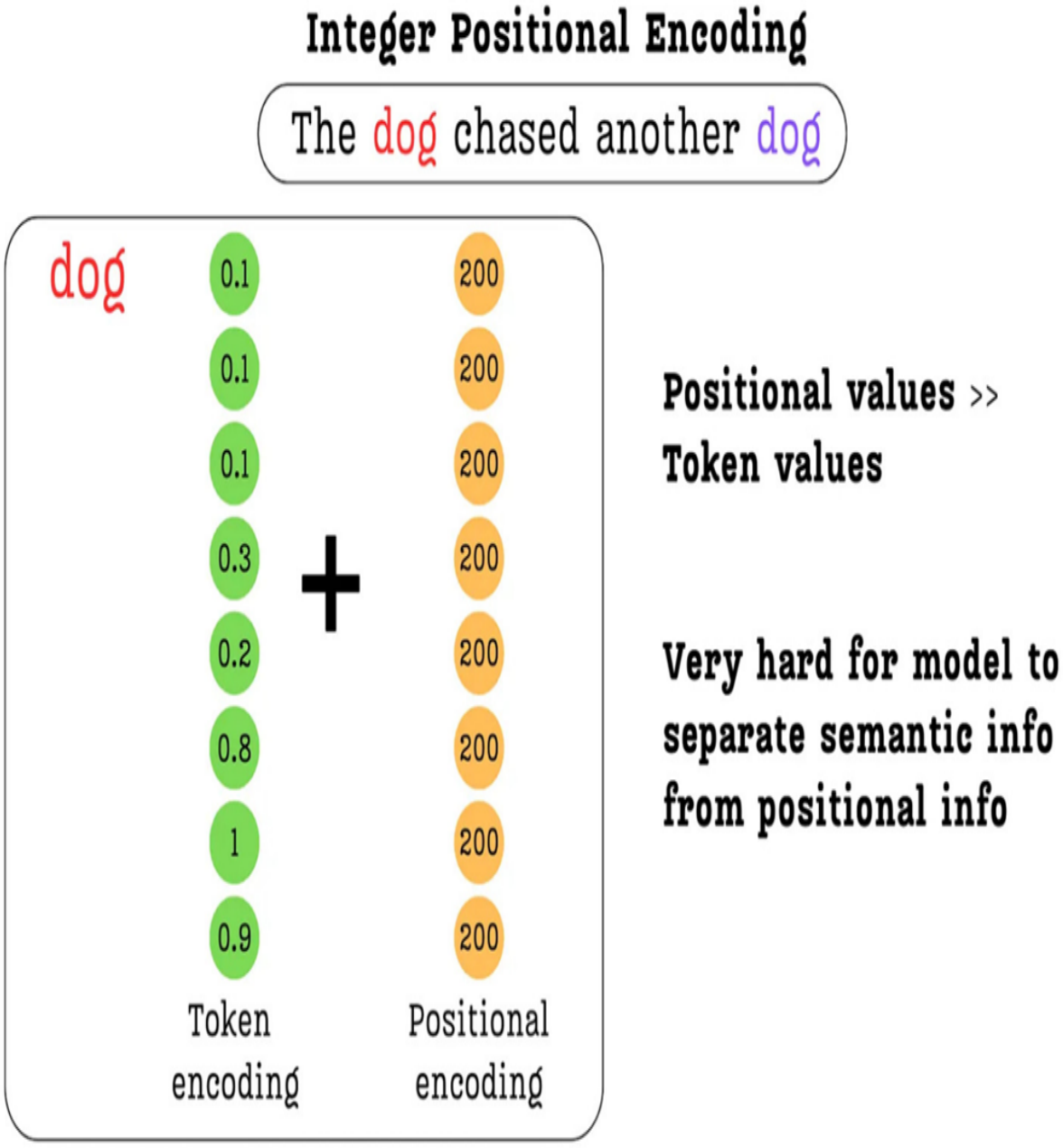
### 3.8.2 The major flaw: Polluting semantic embeddings with large magnitudes

The main problem with Integer Positional Encoding lies in the **magnitude** of the values. Token embeddings, which capture the semantic meaning of words, are typically initialized as small values clustered around zero. This is a deliberate choice in deep learning that helps with training stability. A token embedding for "dog" might look something like `[0.1, 0.3, 0.2, 0.8, ...], where the values are small.`

Now, consider the positional embeddings. While the values are small for the first few tokens (2, 5, etc.), what happens in a model with a large context window? A token at position 200 would have a positional embedding of `[200, 200, 200, ...]`. A token at position 1000 would have `[1000, 1000, 1000, ...]`.

When we add these two vectors together, the large integer values of the positional encoding completely overwhelm the small, nuanced values of the token embedding.

**Figure 3.12 The problem with Integer Positional Encoding. The large magnitude of the positional values completely dominates the small, semantic values of the token encoding, making it very difficult for the model to separate the two sources of information.**




As figure 3.12 illustrates, the sum of the two vectors will be dominated by the positional information. The subtle semantic information the very meaning of the word "dog" is effectively lost in the noise. **We are polluting the semantics.**

This defeats the purpose of having rich token embeddings in the first place. We want the model to learn from the meaning of words, but this naive approach forces the positional information to drown it out. The model would

struggle to learn that the token at position 200 and the token at position 500 are related if they both happen to be the word "dog."

Ideally, we need a method that satisfies two criteria:

1. It must provide a unique encoding for each position.
2. The values of the encoding must be constrained to a small range (e.g., between 0 and 1) so they don't overpower the token embeddings.

This line of thinking leads directly to our second attempt: using a numerical system that is inherently constrained.

# 3.9 Attempt #2: A step forward - Binary positional encodings

The failure of integer positional encodings taught us a valuable lesson: the magnitude of our positional values matters. We need a system that can represent unique positions without using large, unbounded numbers that would overwhelm the semantic token embeddings.

This leads us to a natural next thought: what if we use a numerical system where the values are inherently constrained? This is the core idea behind **Binary Positional Encoding.**

## 3.9.1 Solving the magnitude problem with binary representation

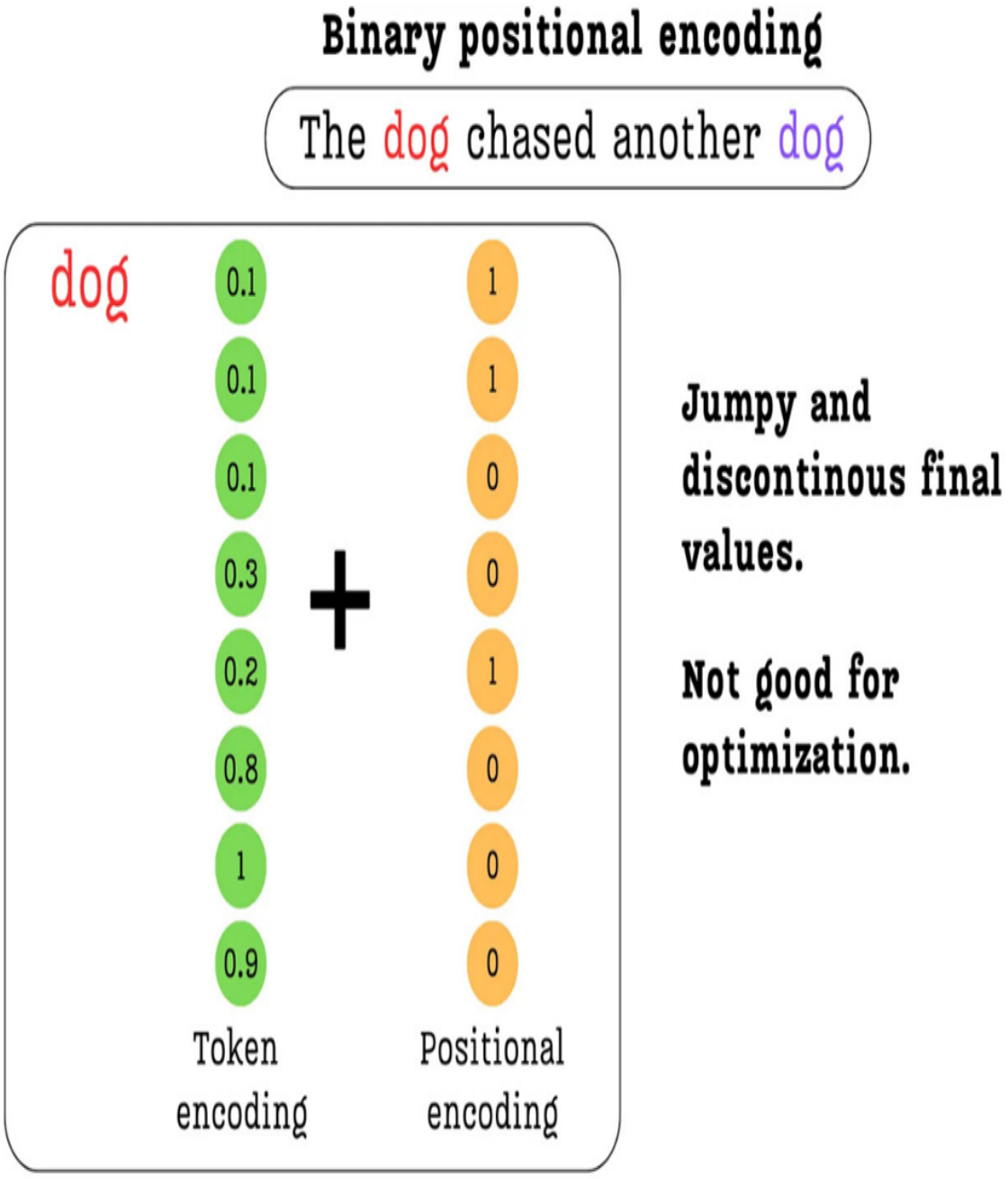
Instead of using the raw integer of a position, we can represent it using its binary form. The binary system uses only two digits, 0 and 1, which perfectly solves our magnitude problem.

Let's consider a token at position 200. If our model's embedding dimension is 8, we can represent this position with an 8-bit binary number:

- **Position 200 (Integer):** `200`
- **Position 200 (8-bit Binary):** `11001000`

We can now create a positional embedding vector directly from this binary representation: `[1, 1, 0, 0, 1, 0, 0, 0]`.

**Figure 3.13 Binary Positional Encoding. The position number is converted to its binary representation, resulting in a vector of 0s and 1s that can be safely added to the token embedding without overwhelming its semantic information.**



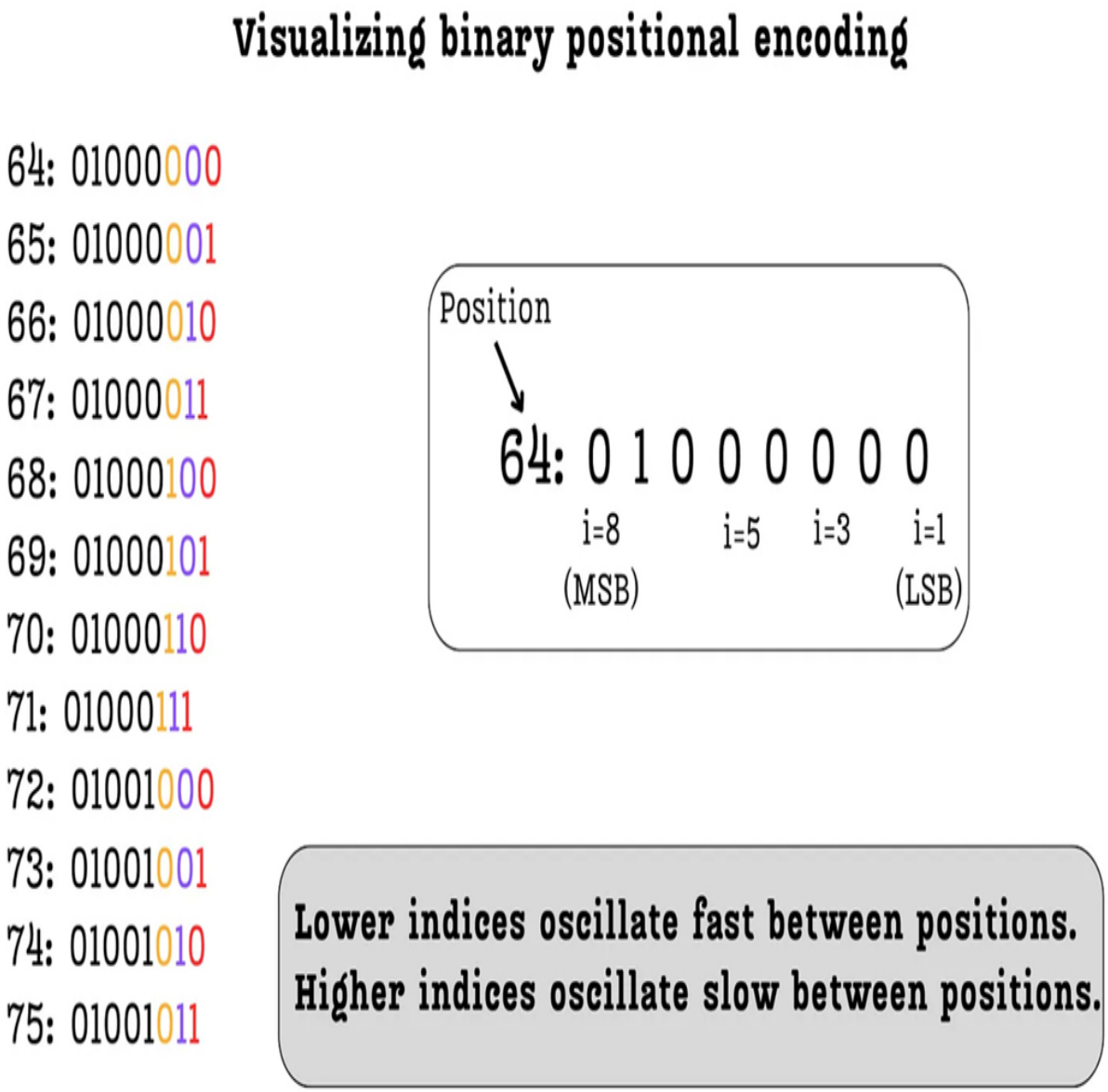
As shown in figure 3.13, this approach is a significant improvement. The positional values are now all 0s or 1s, which are on a similar order of magnitude as the small floating-point numbers in the token embedding. When we add them together, the semantic information is preserved, not polluted. We have successfully solved the primary issue with integer encodings.

This seems like a great solution. It provides a unique, fixed-length, and bounded representation for every position. However, by looking closer at the structure of these binary numbers, we can uncover a much deeper, more interesting pattern that will eventually lead us to the state-of-the-art.

## 3.9.2 Uncovering a deeper pattern: Oscillation frequencies

Binary positional encoding solves the magnitude problem, but it also opens the door to a new way of thinking about positions. Let's analyze the structure of these binary numbers more closely. Within each binary representation, we can think of each digit as occupying an **index**, from the **Least Significant Bit (LSB)** on the right to the **Most Significant Bit (MSB)** on the left.

**Figure 3.14 The oscillating frequency of bits in binary positional encodings. Lower indices (like the LSB) oscillate rapidly across positions, while higher indices (like the MSB) oscillate slowly.**



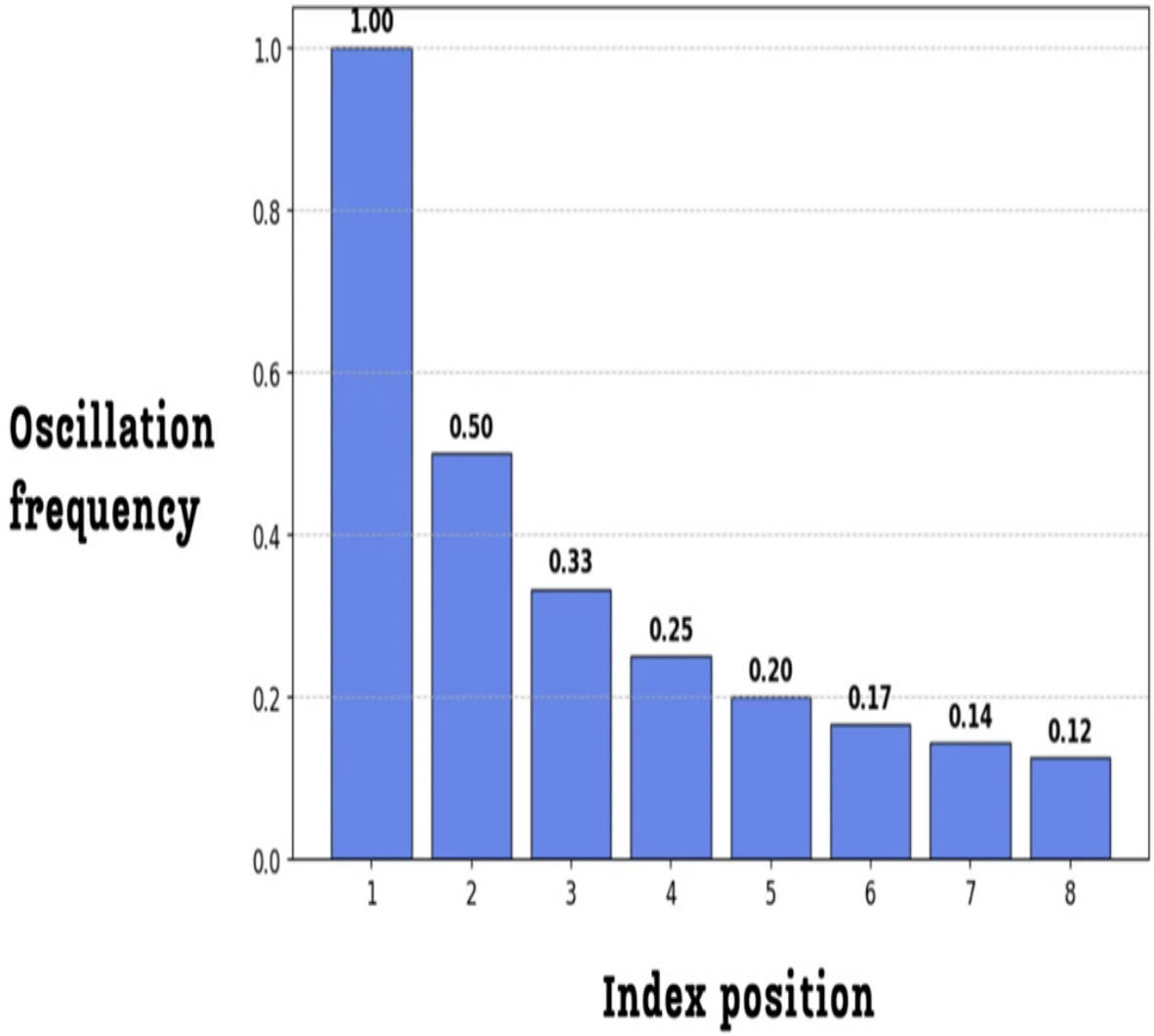

As figure 3.14 illustrates, when we look at a sequence of consecutive positions (from 64 to 75), a clear pattern emerges in how the bits at different indices change:

- **Index 1 (LSB)**: Look at the rightmost column. The values flip at every single position: `0, 1, 0, 1, 0, 1...`. This bit is **oscillating** very rapidly.
- **Index 2**: The next bit flips every two positions: `0, 0, 1, 1, 0, 0...`. It oscillates, but at half the frequency of the LSB.

- **Index 3**: This bit flips every four positions: `0, 0, 0, 0, 1, 1...`. Its frequency is even slower.
- **Index 8 (MSB)**: The leftmost bit doesn't change at all in this range. It oscillates the slowest, flipping only every 128 positions.

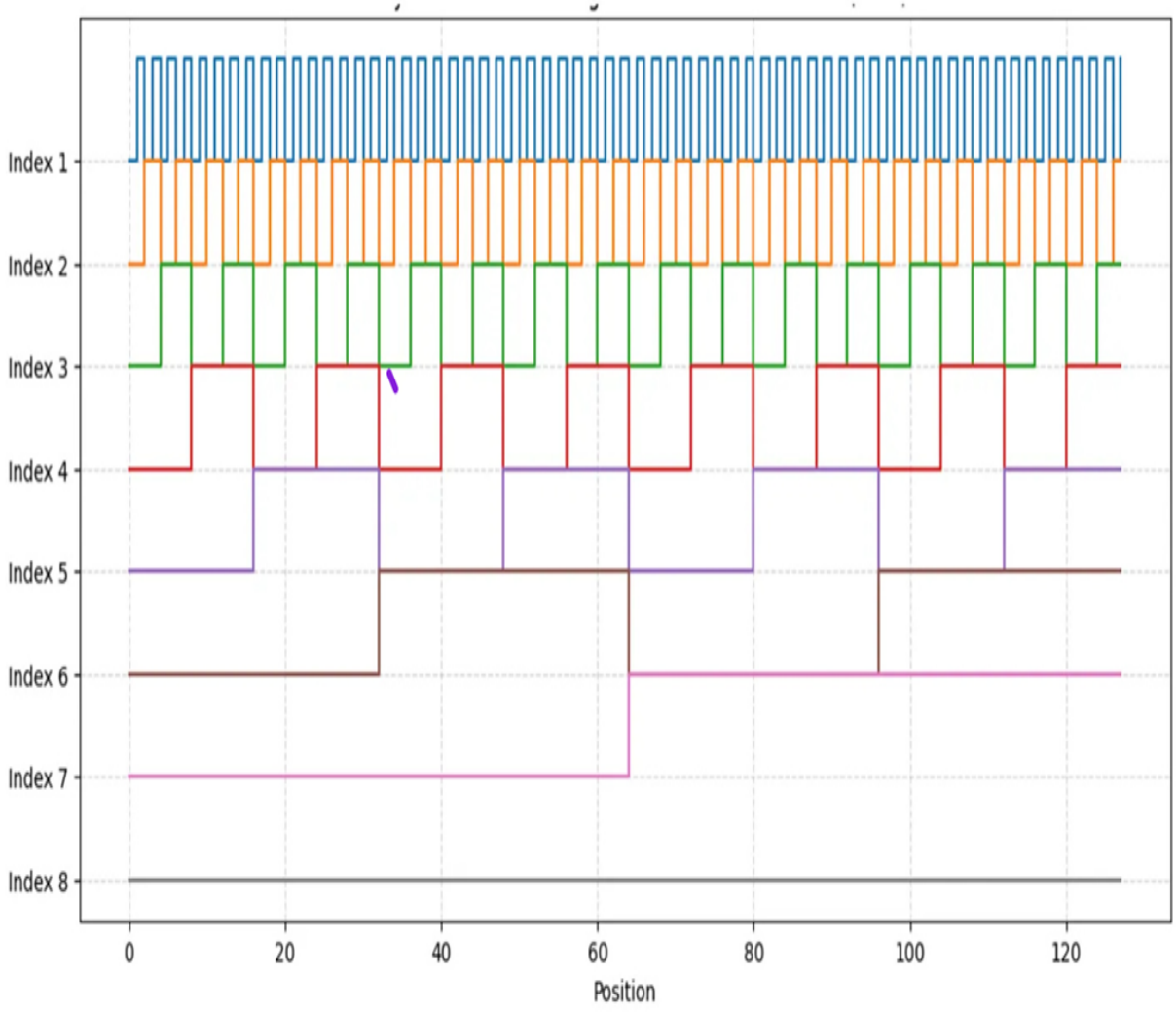
This leads us to a profound observation: **lower indices oscillate fast between positions, while higher indices oscillate slow between positions.** We can visualize this relationship between the index position and its oscillation frequency directly.

**Figure 3.15 The oscillation frequency for each bit index in an 8-bit binary representation. The frequency is highest for the lowest index and decreases exponentially for higher indices.**



This pattern can be seen even more clearly when we plot the value of each index across a longer range of positions.

**Figure 3.16 The oscillation patterns of each bit index across 128 positions. Index 1 (top) oscillates very quickly, while Index 8 (bottom) oscillates very slowly.**




This visualization, shown in figure 3.16, is incredibly important. It reveals that binary encoding doesn't just give each position a unique ID; it decomposes the position into a set of waves, or oscillating signals, each with a different frequency.

1. **The fast-changing, low indices** (like Index 1 and 2) provide **fine-grained information.** They are very good at distinguishing between adjacent positions (e.g., position 64 vs. 65).

2. **The slow-changing, high indices** (like Index 7 and 8) provide **coarse-grained information.** They are good at telling the model that two distant positions (e.g., 5 and 100) are far apart, as their high-index bits will be different.

This discovery that we can represent position using a combination of fast and slow oscillating signals is the key intuition that led to the development of sinusoidal and, eventually, rotary positional encodings. However, binary encoding still has one final, critical flaw.

### 3.9.3 The new problem: The issue with discontinuous jumps

Binary positional encoding has brought us tantalizingly close to a perfect solution. It provides a unique, bounded representation for each position and even decomposes this information into a set of meaningful, multi-frequency signals. So, what's the problem?

The issue lies in the nature of the oscillations themselves. Look again at the plot in figure 3.16. The transition from a 0 to a 1 for any given index is instantaneous and abrupt. These are **discrete jumps**, creating a series of "square waves" rather than smooth curves.

These "jumpy" and discontinuous values are **not good for optimization**. Deep learning models, including Transformers, are trained using gradient-based optimization. This process works best when the functions involved are smooth and continuous. The sharp, step-like changes from 0 to 1 in binary encoding create a "bumpy" landscape for the optimization algorithm to navigate. It becomes difficult for the model to learn the subtle relationships between positions when the underlying positional signal is so abrupt.

Ideally, what we want is a system that keeps the brilliant multi-frequency oscillation pattern we discovered in binary encoding but makes the transitions smooth. We want a graph that has the same properties as figure 3.16, but with smooth, continuous waves instead of sharp, square steps. This immediately suggests an intuition: if we want smooth, oscillating waves, what are the most famous mathematical functions that produce them? **Sine and cosine.**

This exact line of reasoning is what led the authors of the original "Attention Is All You Need" paper to develop the next major innovation in positional encodings. By replacing the discrete, jumpy signals of binary encoding with the smooth, continuous waves of sinusoidal functions, they created a method that is both mathematically elegant and far more effective for training deep neural networks.

# 3.10 Attempt #3: The "Attention Is All You Need" breakthrough - sinusoidal positional encodings

So far we have seen that integer encodings failed due to their large, unbounded values, and binary encodings, while solving the magnitude problem, introduced undesirable discontinuities. This led us to a key insight: we need a method that can represent positions using the multi-frequency oscillating patterns of binary encoding, but with smooth, continuous values.

This is precisely the problem that the authors of the seminal "Attention Is All You Need" paper solved with the introduction of **Sinusoidal Positional Encodings**. This elegant technique uses the sine and cosine functions to create smooth, continuous waves that are far better suited for training deep neural networks.

## 3.10.1 From discrete jumps to smooth waves: Introducing sine and cosine

The core idea of sinusoidal positional encodings is to replace the jumpy 0s and 1s of binary encoding with continuous values that lie on a spectrum from -1 to 1. This is achieved using a now-famous formula that, while looking intimidating at first, is built on the same principles we've already discovered.

**Figure 3.17 Sinusoidal Positional Encoding. The binary values are replaced with continuous values derived from sine and cosine functions, which are then added to the token embedding.**

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{model}})$$

The positional encoding PE for any given token depends on two variables, just as before:

- `pos`: The absolute position of the token in the sequence.
- `i`: The index of the dimension within the embedding vector.

The formula is defined in two parts:

- For **even indices** (`2i`), the value is calculated using a `sin` function.
- For **odd indices** (`2i + 1`), the value is calculated using a `cos` function.

$$PE(pos, 2i) = sin(pos \ / \ 10000\text{^}(2i \ / \ d\_model))$$

$$PE(pos, 2i + 1) = cos(pos \ / \ 10000\text{^}(2i \ / \ d\_model))$$

Let's deconstruct this. Don't be intimidated by math. At its heart, this formula is designed to do exactly what binary encoding did: create a set of waves that oscillate at different frequencies. The term `1 / 10000^(2i / d_model)` is simply the **frequency** of the wave.

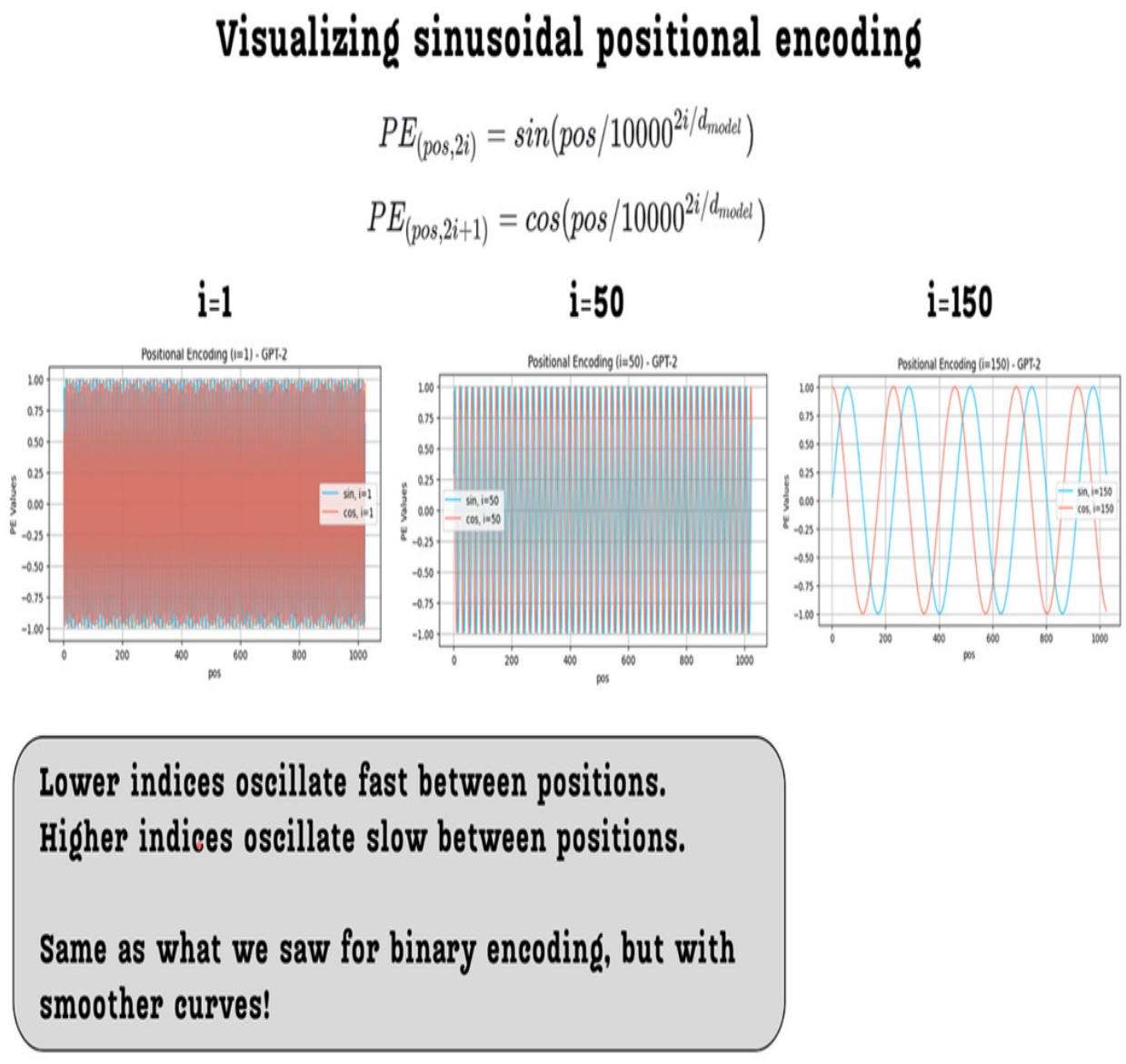
Notice that the index i is in the denominator. This means:

- For **low indices** (small `i`), the frequency is high, and the wave oscillates very fast.

- For **high indices** (large `i`), the frequency is low, and the wave oscillates very slowly.

This is the exact same property we discovered with binary encoding! Sinusoidal encodings preserve the crucial intuition of using fast-changing signals for fine-grained local information and slow-changing signals for coarse-grained global information.

**Figure 3.18 Visualization of sinusoidal positional encodings for a GPT-2-sized model. The plots show the encoding values for different indices (i) across 1024 positions.**



As figure 3.18 illustrates, the resulting curves are exactly what we wanted:

- **They have the same multi-frequency property as binary encodings:** The wave for index i=1 oscillates much faster than the wave for i=150.
- **They are smooth and continuous:** The jumpy, discrete steps have been replaced with smooth sine and cosine waves, which are much better for the gradient-based optimization used to train LLMs.

This formula is a brilliant piece of engineering. It solves the discontinuity problem of binary encodings while preserving their most important feature. However, the use of both sine and cosine together unlocks an even deeper, more powerful property.

## 3.10.2 The power of rotation: Encoding relative positions

The sinusoidal formula works, but why the specific choice of pairing `sin` for even indices and `cos` for odd indices? This isn't an arbitrary decision. This specific pairing endows the positional encodings with a remarkable property: **the positional encoding for any position can be represented as a linear rotation of the encoding for any other position**.

This is a crucial property. It means there is a simple, consistent mathematical relationship between the encodings of different positions. If the model can learn this simple relationship, it can generalize its understanding of positions far more effectively than if it had to memorize each position's encoding independently.

To understand how this works, let's visualize it. In the sinusoidal positional encoding , we can think of each pair of dimensions (2i and 2i+1) as the coordinates of a 2D vector.

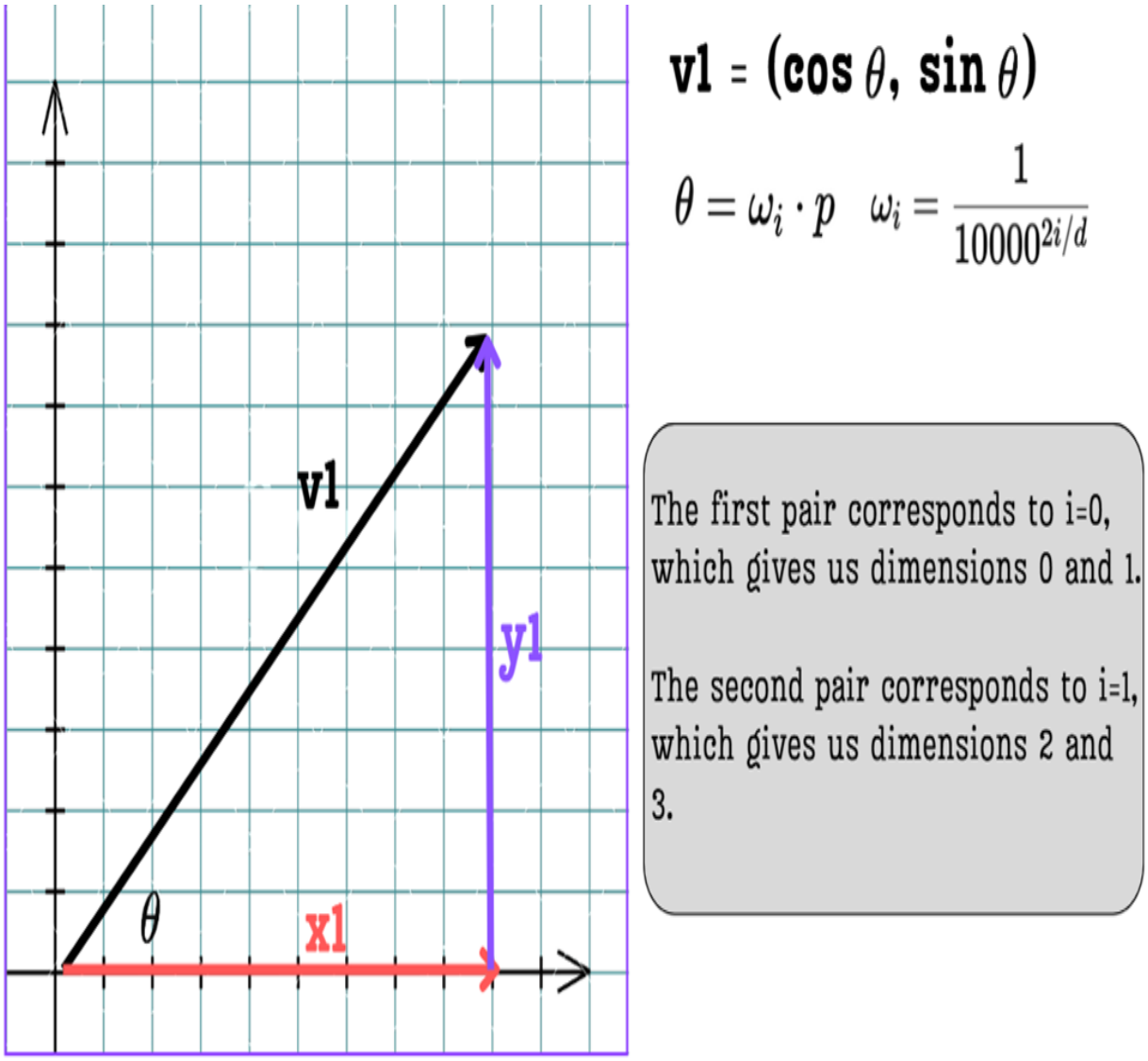
- Let's define the y-coordinate as the value at the even index (2i), which is $\sin(\theta)$.
- Let's define the x-coordinate as the value at the odd index (2i+1), which is $\cos(\theta)$.

Here, the angle $\theta$ is determined by two factors: the token's position in the sequence and the index of the dimension pair. The formula is:

$$\theta = p / 10000\text{^}(2i / d\_model)$$

where p represents the absolute position (or index) of the token in the input sequence.

**Figure 3.19 A pair of sinusoidal positional encoding values can be viewed as the coordinates of a 2D vector v1 on a unit circle, defined by the angle θ.**

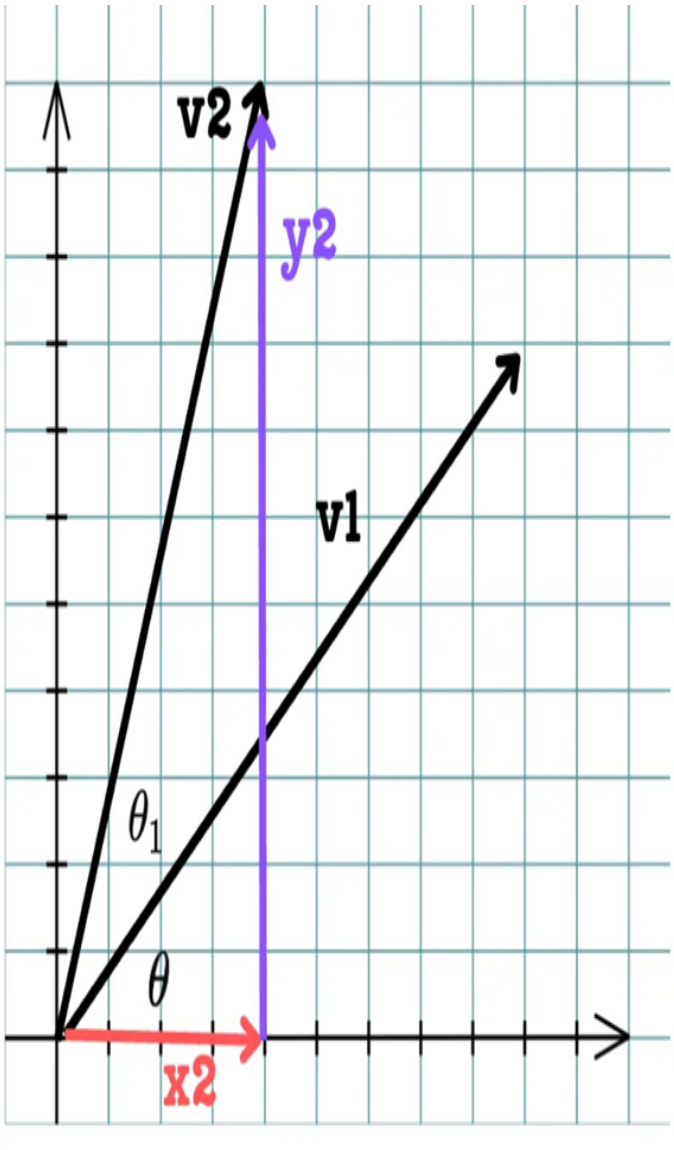



As shown in figure 3.19, for a given position `p` and an index pair `i`, the positional encoding values form a vector `v1` on a circle. Now, here is the

puzzle: how can we find the positional encoding for a new position that is **k steps away**, `p + k`, using our existing encoding for position `p`?

The answer is remarkably elegant. It turns out that the positional encoding vector for position `p + k` is simply the original vector for position `p` **rotated by a fixed angle.**

**Figure 3.20 The positional encoding vector v2 for a future position `p + k` is a simple rotation of the original vector `v1` for position p.**




$$\mathbf{v1} = (\cos\theta, \sin\theta)$$

$$\theta = \omega_i \cdot p \quad \omega_i = \frac{1}{10000^{2i/d}}$$

$$\mathbf{v2} = (\cos(\theta+\theta_1), \sin(\theta+\theta_1))$$

$$\theta_1 = \omega \cdot k$$

v1 and v2 are rotations of each other.

**What this means is: Relative positional encodings are just rotations of each other!**

As illustrated in figure 3.20, if we want to find the positional encoding for a position that is `k` steps away, we just need to rotate our original vector `v1` by an angle $\theta_1 = k * \omega_i$. This is a direct consequence of the trigonometric identities `sin(A+B)` and `cos(A+B)`.

This means that relative positional relationships are encoded as simple rotations. The model doesn't need to learn complex, arbitrary patterns. It can learn that to understand the relationship between a token at position `p` and a token at position `p+2`, it just needs to apply a consistent "two-step rotation."

This is the genius of using the `sin` and `cos` pair.

- It provides the smooth, multi-frequency signals we need.
- It embeds a simple, linear relationship between positions, making it much easier for the Transformer to learn and generalize positional patterns.

This powerful technique, introduced in the original Transformer paper, became the gold standard for positional encodings for years. However, it still has one subtle, lingering flaw.

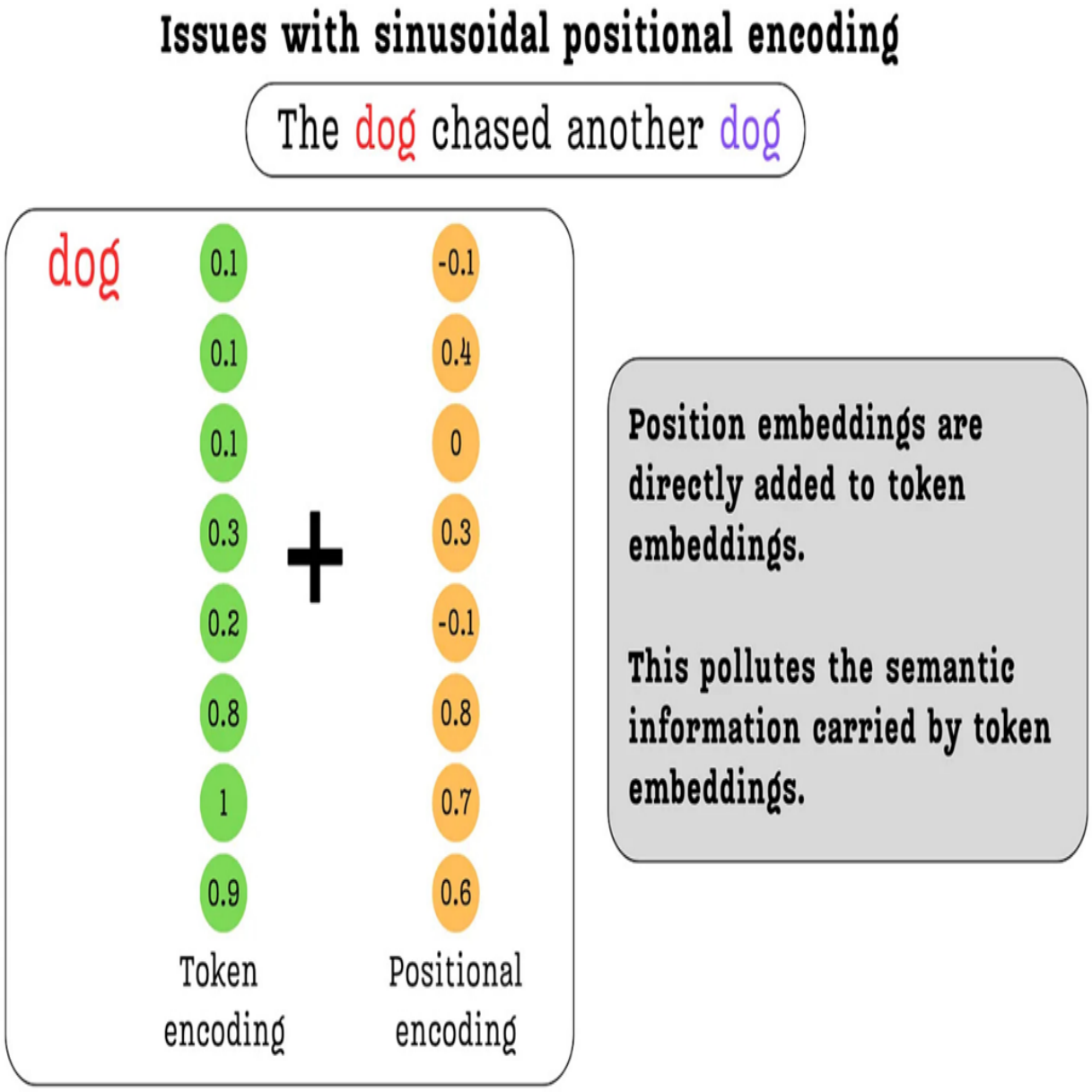
## 3.10.3 The remaining flaw: Still polluting the token embeddings

Sinusoidal positional encodings are a brilliant solution. They are continuous, they capture multi-frequency positional signals, and they encode relative positions as simple rotations. They seem to solve all of our problems. So why did the field move on to something else like RoPE?

The answer lies in how these positional encodings are integrated into the model.

If we look back at the overall architecture (Figure 3.11), we see that the positional embeddings are **directly added** to the token embeddings before they enter the first Transformer block.

**Figure 3.21 The fundamental issue with sinusoidal positional encodings. The positional information is added directly to the token's semantic embedding, potentially altering or "polluting" its original meaning.**




As figure 3.21 highlights, even though the magnitude of the sinusoidal values is small (between -1 and 1), the very act of addition mixes the positional signal with the semantic signal. The vector that represents the meaning of the word "dog" is fundamentally altered before the model even begins to process it. This is the **semantic pollution** problem, or more formally representation entanglement.

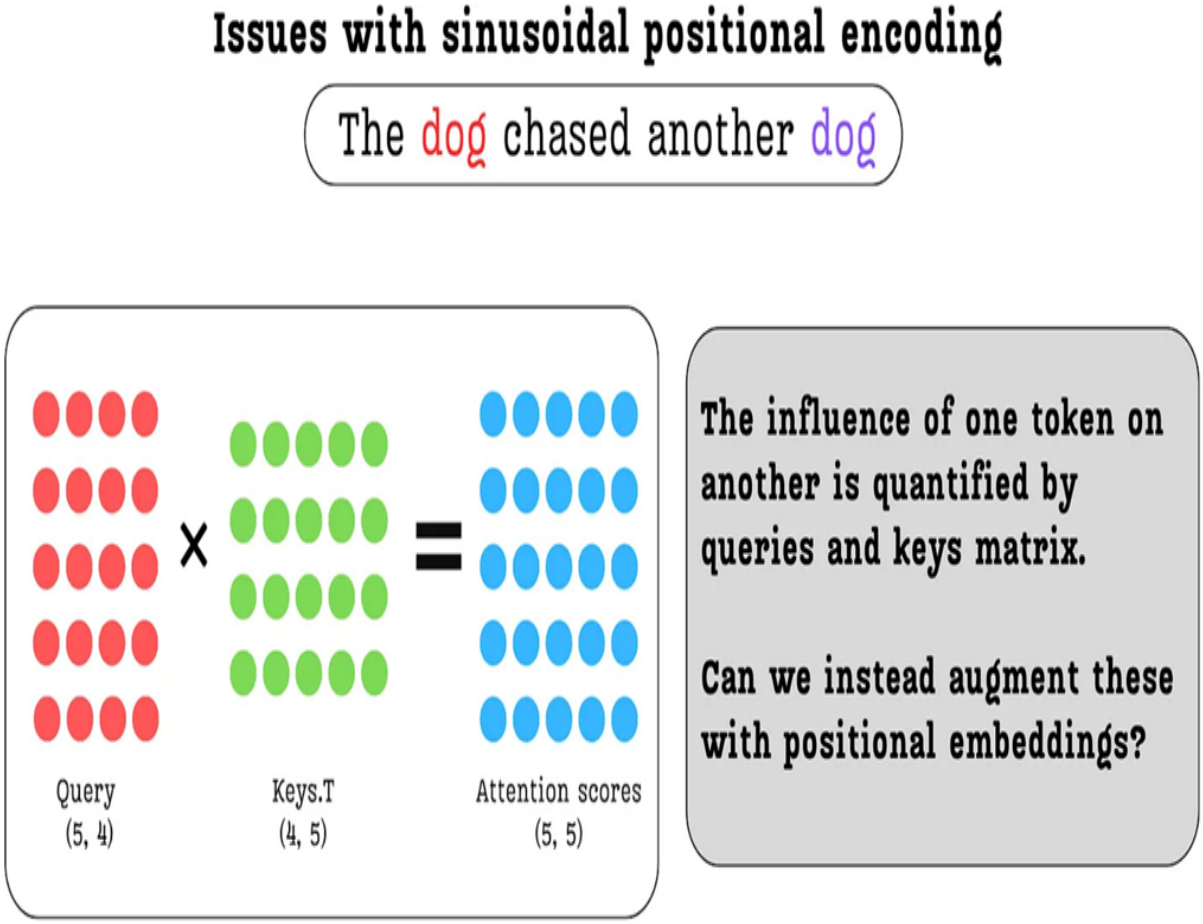
It is worth noting that while this is a theoretical issue, it is not a catastrophic one. Transformers are remarkably powerful; their deep stacks of layers can learn to separate and reinterpret the mixed token and positional information.

Indeed, models trained with additive sinusoidal encodings have achieved excellent performance for years. However, the core insight remained: could we find a cleaner method that avoids this entanglement from the start?

Ideally, we want the semantic information from the token embeddings to be preserved as cleanly as possible. This led researchers to ask a critical question: must we inject positional information at the very beginning?

The attention mechanism is the part of the model where the relationships between tokens and thus their relative positions truly matter. The influence of one token on another is quantified by the interaction between their Query and Key vectors.

**Figure 3.22 The attention score calculation. The influence of one token on another is quantified by the dot product of their respective Query and Key vectors. This core interaction is where relative position truly matters.**



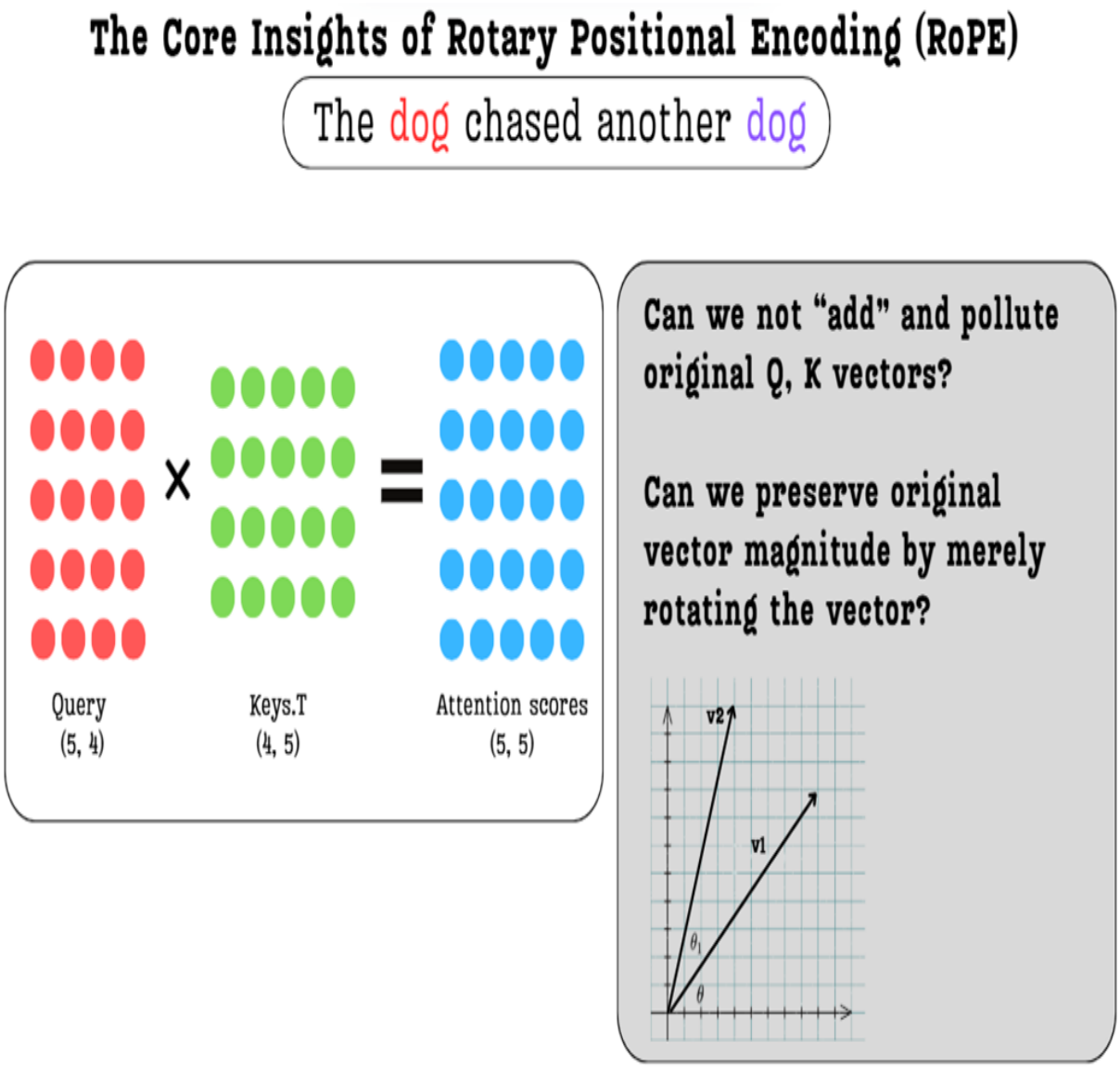

This insight, prompted by the diagram in figure 3.22, is the final step in our journey. The diagram reminds us that the entire measure of relevance between tokens boils down to the interaction between their Query and Key vectors. This begs a critical question: if this is where positional relationships are quantified, why are we modifying the token embeddings at the very beginning?

This leads to a new, cleaner approach:

1. **Don't touch the initial token embeddings.** Let them carry pure semantic information into the Transformer blocks.
2. **Inject positional information later,** directly into the Query and Key vectors right before the attention scores are calculated.

But how should we inject this information? If we simply add a positional vector to the Query and Key vectors, we might still change their magnitude and alter their learned representations. This leads to the final, elegant idea.

**Figure 3.23 The core ideas of RoPE. We can avoid polluting the original vectors by rotating them instead of adding to them, and we can apply this rotation directly to the Query and Key vectors where positional information is most relevant.**




As figure 3.23 suggests, what if we combine the best ideas we've discovered?

- What if we inject the positional information directly into the **Query and Key vectors**?
- And what if, instead of adding a vector, we simply **rotate** them, preserving their original magnitude and learning semantic direction?

These two ideas together form the foundation of **Rotary Positional Encoding (RoPE)**, the state-of-the-art technique that we will now explore in detail.

# 3.11 The state-of-the-art: Rotary Positional Encoding (RoPE)

We have seen the flaws in naive approaches and the brilliance of sinusoidal encodings, which introduced the powerful concept of encoding relative positions as rotations. However, sinusoidal encodings still suffer from one lingering issue: they pollute the semantic token embeddings by being added to them directly at the start of the Transformer block.

This is where **Rotary Positional Encoding (RoPE)** enters the picture. RoPE takes the best ideas from sinusoidal encodings and refines them into a more elegant and effective solution. It is built on two simple but profound insights that address the final remaining problem.

## 3.11.1 The core insights: Injecting position into attention and preserving magnitude

The development of RoPE started from answering two fundamental questions:

**Question 1: Where is the best place to add positional information?**

The attention mechanism is the part of the model where the relationships between tokens, and thus their relative positions, truly matter. The influence of one token on another is quantified by the interaction between their **Query** and **Key** vectors. Instead of modifying the token embeddings at the very beginning, why not inject the positional information directly into the Query and Key vectors, right where it's needed most? This would allow the pure,

semantic information from the token embeddings to flow into the Transformer block untouched.

**Question 2: How can we add this information without corrupting the vectors?**

When we add a positional vector to a token embedding, we change its magnitude and direction in vector space. This "pollutes" the original semantic representation. Is there a way to modify a vector to encode new information while preserving its original length and, to some extent, its core direction? The answer is **rotation**. If we simply rotate a vector, we change its coordinates, but its magnitude remains unchanged.

These two ideas together form the foundation of Rotary Positional Encoding:

> Instead of adding a positional vector to the initial token embeddings, RoPE rotates the **Query and Key vectors** directly within the attention mechanism. The angle of rotation is determined by the token's position.

This approach is brilliant because it solves both problems at once. We avoid semantic pollution by modifying the Q and K vectors later in the process, and we preserve the vectors' learned magnitudes by using rotation instead of addition. In high-dimensional vector spaces, the direction of a vector captures its core semantic meaning. By only rotating the vectors, we change their orientation to encode positional information without altering their length (magnitude) or fundamentally distorting their learned semantic direction. This ensures the model's understanding of a token's meaning remains intact while being augmented with positional context.

Now, let's see how this is implemented in practice.

## 3.11.2 The mechanism: Rotating query and key vectors

Now that we have the core idea rotating Query and Key vectors we need to understand the mechanics of how this is actually done. If you understand this mechanism, you'll never forget RoPE, because it's a very visual and intuitive concept. The process can be broken down into a few simple steps. For this

demonstration, we will focus on a single Query vector (the exact same process applies to the Key vectors as well).

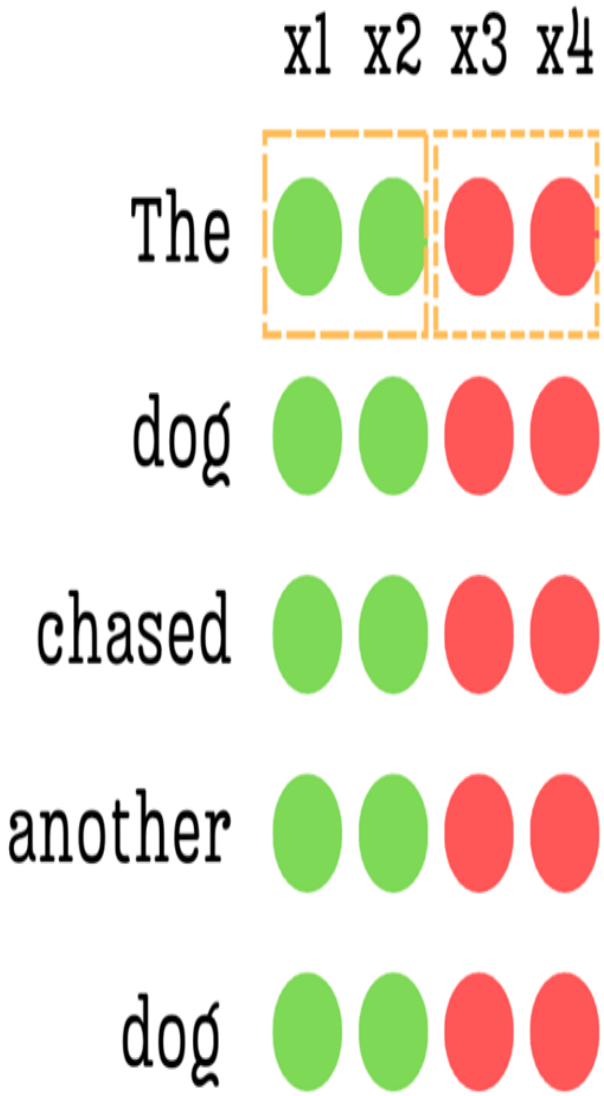
### Step 1: Grouping the vector into pairs

The rotation operation is a 2D concept. However, our Query and Key vectors are high-dimensional (e.g., a 4-dimensional vector in our simplified example). How can we rotate a 4D vector?

RoPE's clever solution is to not treat it as a single 4D vector at all. Instead, it groups the dimensions into pairs For this example, let's assume the dimension of a single attention head (d_head) is 4. We will be working with the individual Query or Key vector for a single token within this head.

**Figure 3.24 The dimensions of a Query or Key vector are paired up for rotation.**

# Rotary positional encoding (RoPE)




## Original Query/Key vectors

As shown in figure 3.24, for a 4-dimensional vector `[x1, x2, x3, x4]`, we create two independent 2D groups:

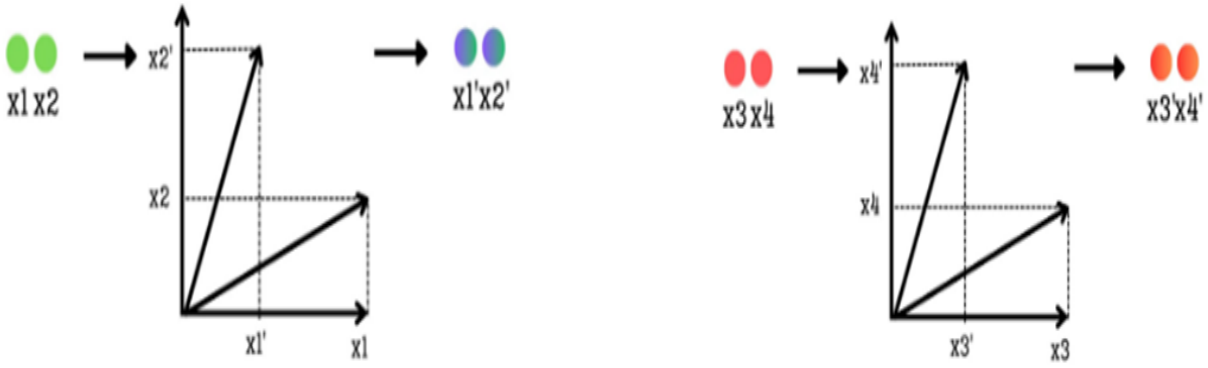
- **Group 1:** Consists of the first two dimensions, `(x1, x2)`.
- **Group 2:** Consists of the next two dimensions, `(x3, x4)`.

Each of these pairs will now be treated as a 2D vector that we can rotate on a plane. This process is repeated for the entire length of the vector. If the vector were 128-dimensional, we would have 64 pairs to rotate.

### Step 2: Rotating each pair by a position-dependent angle

With our dimensions grouped into 2D pairs, we can now apply the rotation. Each pair is rotated by an angle, θ, which encodes the token's position.

**Figure 3.25 Each pair of dimensions is treated as a 2D vector and rotated by a position-dependent angle θ.**



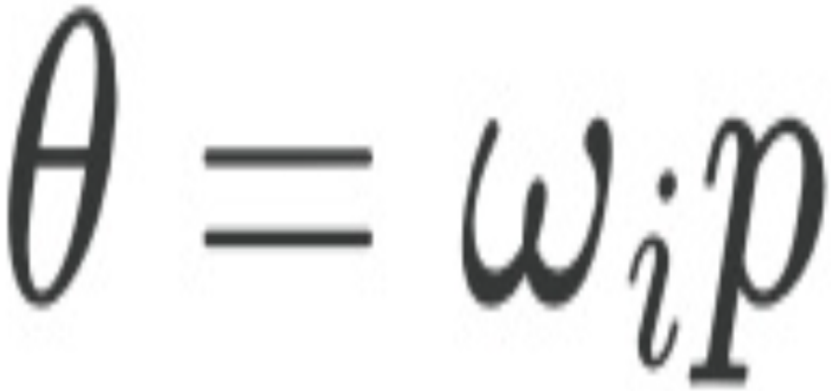

As illustrated in figure 3.25, the process for each group is identical:

- The pair `(x1, x2)` is treated as a 2D vector. It is then rotated by an angle θ to produce a new vector with coordinates `(x1', x2')`.
- Simultaneously, the pair `(x3, x4)` is treated as another 2D vector. It is rotated by a different angle to produce `(x3', x4')`.

This rotation is the core of RoPE. It injects the positional information into the vector. But what determines the angle of rotation?

The angle θ is calculated using the exact same logic we discovered in sinusoidal encodings. It depends on two things: the token's absolute position (p) and the index of the pair (i).

**Figure 3.26 The formula for the rotation angle θ.**



The formula is $\theta = p * \omega_i$, where $\omega_i = 1 / 10000^{(2i / d)}$.

Let's break this down:

- p: The absolute position of the token (e.g., 0, 1, 2, ...). A token further down the sequence will have a larger p, and thus a larger base rotation.

- `i`: The index of the pair we are rotating. For the first pair `(x1, x2)`, `i=0`. For the second pair `(x3, x4)`, `i=1`, and so on.
- `d`: The total dimension of the vector (in our case, 4).

This formula brilliantly re-implements the multi-frequency oscillation pattern.

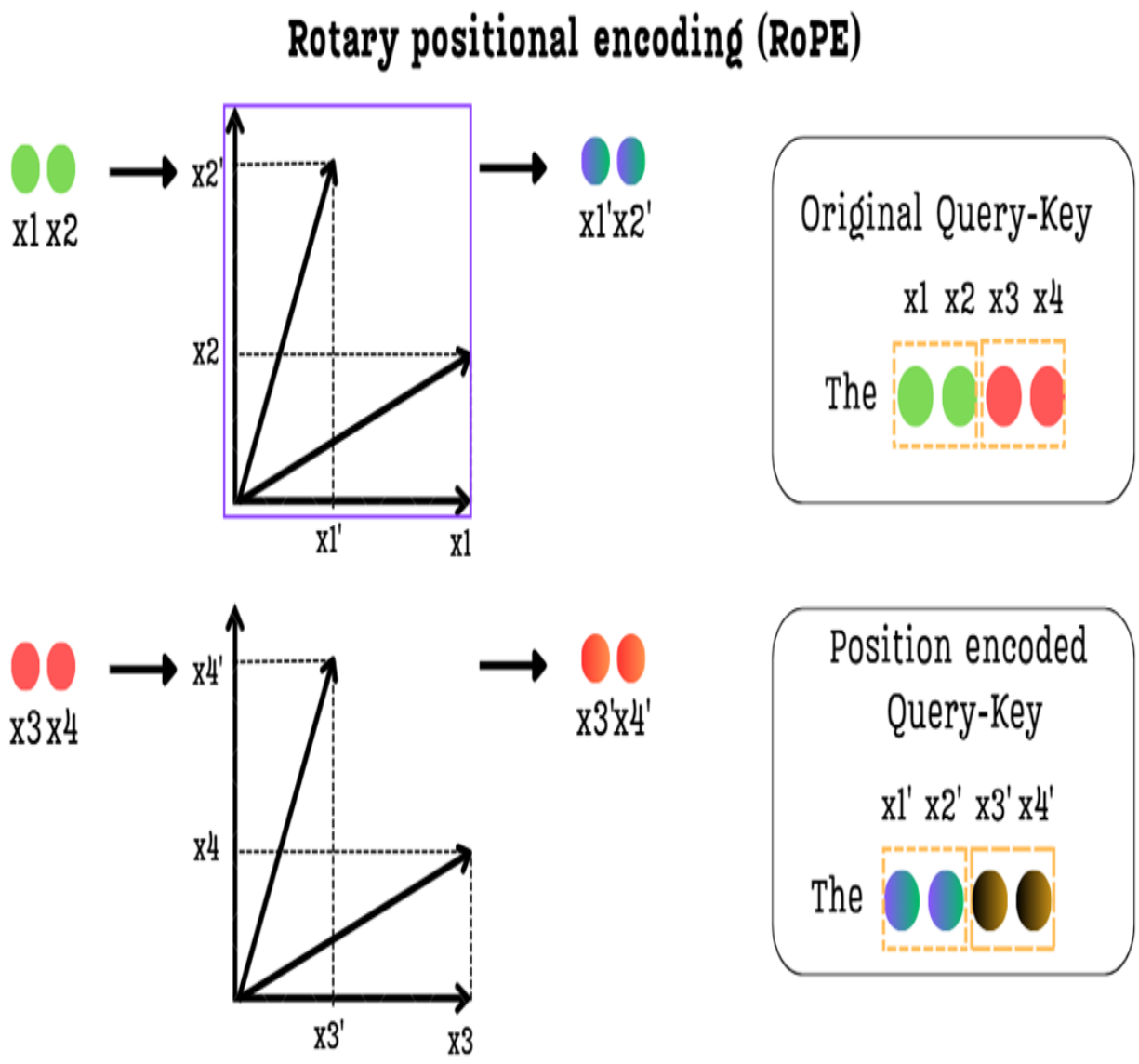
- The first pair (`i=0`) will be rotated with a high frequency, changing significantly with each new position `p`.
- The second pair (`i=1`) will be rotated with a lower frequency, changing more slowly across positions.

This is how RoPE preserves the crucial property of using fast oscillations to capture fine-grained local relationships and slow oscillations to capture coarse-grained, long-range dependencies.

### Step 3: Assembling the final position-encoded vector

After rotating each pair of dimensions independently, we reassemble them to form the final, position-encoded Query or Key vector.

**Figure 3.27 The final step of the RoPE mechanism. The rotated 2D pairs are reassembled to create the final position-encoded Query or Key vector.**




As shown in figure 3.27, the original vector `[x1, x2, x3, x4]` is transformed into the new vector `[x1', x2', x3', x4']`.

This new vector is a masterpiece of information engineering. It has successfully encoded the token's positional information directly into its structure, and it has done so while achieving two critical goals that previous methods could not.

**Benefit 1: Magnitude is preserved**

The most important property of a 2D rotation is that it does not change the length (the magnitude) of the vector.

- The magnitude of the vector `(x1, x2)` is the same as the rotated vector `(x1', x2')`.
- The magnitude of the vector `(x3, x4)` is the same as the rotated vector `(x3', x4')`.

Because we are only rotating parts of the original vector, the overall magnitude of the final vector is preserved. This is a huge advantage. It means we can inject rich positional information without distorting the learned semantic representations of our Query and Key vectors. We have solved the "semantic pollution" problem.

**Benefit 2: Relative positions are encoded**

Just like with sinusoidal encodings, the use of rotation means that the relationship between two tokens at different positions can be expressed as a simple, linear transformation. The attention score between a Query at position m and a Key at position n will depend on their relative distance (`m-n`), which is encoded in the difference of their rotation angles. This allows the model to learn relative positional relationships in a much more natural and generalizable way.

This is the complete geometrical intuition of Rotary Positional Encoding. It is an incredibly elegant solution that combines the best ideas we've seen so far:

- It injects positional information directly into the **Query and Key vectors,** avoiding the need to modify the initial token embeddings.
- It uses **rotation** to preserve the magnitude of the original vectors.
- It re-uses the brilliant **multi-frequency formula** from sinusoidal encodings to capture both fine-grained and coarse-grained positional relationships.

This powerful and efficient mechanism has become the standard for most modern, high-performance LLMs. However, as we will see, integrating this elegant solution with the equally innovative MLA architecture from DeepSeek presents one final, fascinating challenge.

# 3.12 The new challenge: Why standard RoPE and MLA don't mix

We have now mastered two of the most important innovations in modern Transformer architecture:

1. **Multi-Head Latent Attention (MLA):** A brilliant solution for the KV Cache memory bottleneck that works by caching a single, compressed latent representation of the Keys and Values.
2. **Rotary Positional Encoding (RoPE):** An elegant solution for encoding positional information by rotating the Query and Key vectors before the attention calculation.

Individually, each of these techniques is a well-crafted engineering solution. The natural next step would be to combine them to create a truly state-of-the-art attention block. However, when we try to do this, we run into a fundamental incompatibility.

Let's think about the logical flow. The standard RoPE implementation requires us to rotate the Key vectors based on their absolute position before we compute attention. But the core idea of MLA is to compute a position-agnostic latent representation of the Keys that we can store in a cache.

Herein lies the conflict:

- **RoPE says:** "I need to modify every Key vector differently based on its unique position in the sequence."
- **MLA's cache says:** "I need to store a simple, compressed representation that doesn't depend on position, so I can decompress it efficiently."

If we apply RoPE to the Key vectors before we try to compress them with MLA's down-projector, the compression will fail. The down-projector would have to learn to "un-rotate" every vector's unique positional information before it could compress the underlying content, which is an impossibly complex task. The beautiful simplicity of the latent cache is broken.

Conversely, if we apply RoPE after decompressing the cache, it's too late. The attention calculation is the very next step, and we would have to re-compute the rotation for every key in the entire history at every single step, which would completely negate the speed benefits of caching.

This incompatibility between the position-dependent nature of RoPE and the position-agnostic nature of the MLA cache is the final puzzle we must solve. The DeepSeek team's solution to this problem is a clever and elegant modification to the RoPE mechanism itself, which we will build from scratch further in this chapter only.

# 3.13 The incompatibility problem: Why standard MLA and RoPE don’t work together

To see exactly why standard MLA and RoPE don't work together, we must look at the mathematics of the "absorption trick" that makes MLA so efficient.

As we established, the efficiency of MLA comes from a clever mathematical rearrangement of the attention score formula. The original formula $Q * K^T$ was transformed into:

Attention Scores = $X * (Wq * Wuk^T) * (X * Wdkv)^T$

This trick works because the two weight matrices, Wq and Wuk, are adjacent to each other. Their product, $(Wq * Wuk^T)$, can be pre-computed as a single, fixed matrix. This allows the model to avoid caching the full Key matrix and instead only cache the much smaller latent matrix, cKV = X * Wd-kv. This absorption is only possible because the weight matrices are **position-agnostic**; their values do not change based on a token's position.

Now, let's recall how RoPE works. It injects positional information by applying a rotation function, RoPE(), directly to the Query and Key vectors after they have been projected.

```
Q_rotated = RoPE(Q, position)
K_rotated = RoPE(K, position)
```

Crucially, the RoPE() function is **position-dependent.** The rotation applied to the vector for a token at position 5 is different from the rotation applied to a token at position 10.

Herein lies the problem. If we try to combine these two techniques naively, the RoPE() function ends up sitting directly between the two matrices we need to absorb. The attention score calculation would look something like this:

$$Attention\ Scores\ = RoPE(X * Wq) * RoPE((X * Wd - kv) * Wuk)^{T}$$

The position-dependent `RoPE()` function acts as a barrier, preventing Wq and Wuk from being multiplied together. The absorption trick is broken.

This has bad consequences for inference efficiency. Without the absorption trick, we would be forced to re-compute the full, rotated Key matrix for every token in the entire history at every single generation step. This would completely negate the speed benefits of caching and defeat the entire purpose of MLA.

This incompatibility between the position-dependent nature of RoPE and the position-agnostic requirement of the MLA cache is the final puzzle we must solve. The DeepSeek team's solution to this problem is a clever and elegant modification to the architecture itself, which we will now build from scratch.

# 3.14 The DeepSeek solution: Decoupled rotary position embedding

To solve the incompatibility between MLA and RoPE, the DeepSeek team devised an elegant solution: if the positional information is breaking our absorption trick, let's simply take it out of the main path. This is the core idea behind **Decoupled Rotary Position Embedding.**

The insight is to split the attention calculation into two parallel streams:

A Content Path: This path is responsible for understanding the semantic meaning of the tokens ("what"). It will use a pure, standard MLA mechanism without any positional information, allowing the absorption trick to work perfectly.

A Position Path: This path is responsible for understanding the relative order of the tokens ("where"). It will use a new, specialized set of projections where RoPE can be applied freely.

The final attention scores will be the sum of the scores from these two independent paths. This allows the model to calculate content-based relevance and position-based relevance separately and then combine them to make a final, informed decision.

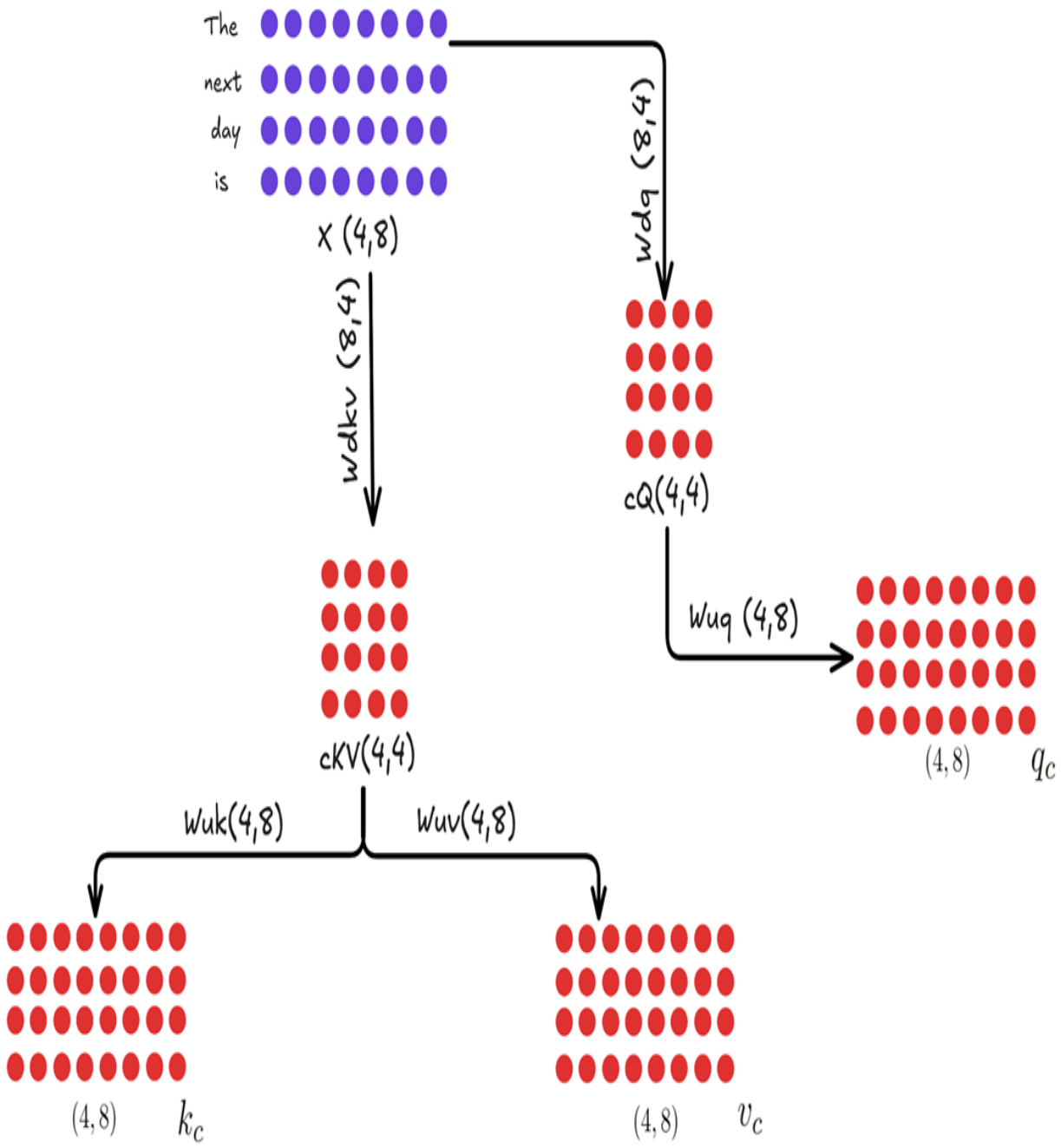
## 3.14.1 The new architecture: A visual and mathematical deep dive

This decoupled approach requires us to introduce a new set of weight matrices specifically for the position path. Let's walk through the full architecture, looking at each path in detail.

### A. The content path (no RoPE)

This path is almost identical to the "pure" MLA we built in section 3.3. Its goal is to produce Query, Key, and Value matrices that are based purely on the semantic content of the input tokens.

**Figure 3.28 The Content Path of the Decoupled Architecture. This is a standard MLA block that produces content-based Query (Qc), Key (Kc), and Value (Vc) matrices.**




As shown in figure 3.28, the process is as follows:

- The input X is down-projected to create a latent query `cQ` and a latent key-value `cKV`.
- These are then up-projected to create the final, full-dimensional matrices:
    - `Qc = (X * Wdq) * Wuq`

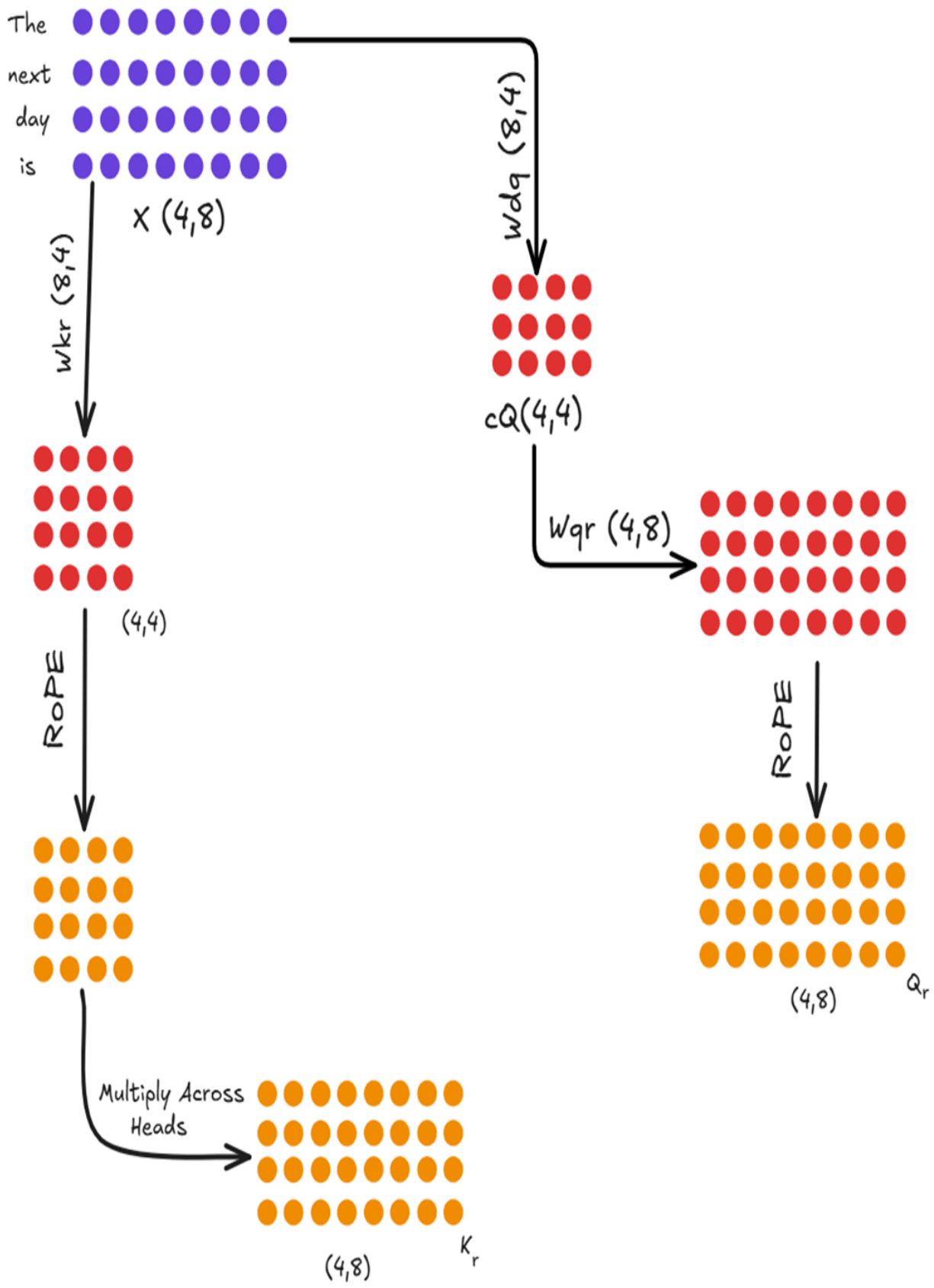
- `Kc = (X * Wdkv) * Wuk`
- `Vc = (X * Wdkv) * Wuv`

Crucially, no RoPE is applied anywhere in this path. This means that when we calculate the content-based attention scores ($Qc * Kc^T$), the absorption trick remains fully intact. The cKV matrix is the only thing from this path that needs to be cached. The Vc matrix will be used later to compute the final context vector.

## B. The position path (RoPE applied)

Running in parallel to the Content Path is a new, specialized path designed exclusively to handle positional information. Its goal is to produce a Query and a Key matrix that are encoded with RoPE.

**Figure 3.29 The Position Path of the Decoupled Architecture. This new path creates specialized Query (Qr) and Key (Kr) matrices that are infused with Rotary Positional Encodings.**


The
next
day
is
X (4,8)
Wdq (8,4)
Wkr (8,4)
cQ(4,4)
Wqr (4,8)
(4,4)
RoPE
RoPE
(4,8)
$Q_r$
Multiply Across Heads
(4,8)
$K_r$

1. **Calculating the positional key (Kr):**
    - The input `X` is multiplied by a new weight matrix, `Wkr`.
    - **RoPE is then applied** to the result.
    - The DeepSeek paper notes a key optimization here: this `Kr` projection is **shared across all heads.** The resulting single-head matrix is then repeated or "multiplied" to match the full number of heads. This saves a significant amount of memory and computation. The final result is the Kr matrix.
2. **Calculating the positional query (Qr):**
    - The Query path is slightly more complex, mirroring the down-projection/up-projection of the content path to save activation memory.
    - The input `X` is first down-projected using `Wdq` to get the same latent query `cQ` from the content path.
    - This `cQ` is then multiplied by a new weight matrix, `Wqr`.
    - Finally, **RoPE is applied** to this result to produce the final `Qr` matrix.
    - Unlike the keys, the positional queries are **not shared**. `Wqr` is a full multi-headed matrix, producing a unique positional query for each head.

At the end of this path, we have two new matrices, `Qr` and `Kr`, which contain rich, relative positional information. The only part that needs to be cached for this path is the shared, rotated key matrix, `Kr`.

The positional query matrix, Qr, does not need to be cached. This follows the same fundamental principle we established for standard KV caching in chapter 2: during autoregressive generation, we only ever need the query information for the single, most recent token to predict the next one. Since the query vector for the current token must always be computed fresh, there is no benefit to caching the query vectors from past tokens.

## 3.14.2 Combining the paths to calculate final attention

Now that we have the outputs from both parallel paths, we can combine them to calculate the final attention scores. The full Query matrix `Q` is the concatenation of the content query `Qc` and the positional query `Qr`. The full Key matrix `K` is the concatenation of `Kc` and `Kr`.

The final attention score is the dot product of these two combined matrices.

**Figure 3.30 The final attention score calculation. The scores are the sum of the content-based scores and the position-based scores.**

$$\text{Attention Scores} = Q \times K^T$$

$$= [Qc; Qr] \cdot [Kc; Kr]^T$$

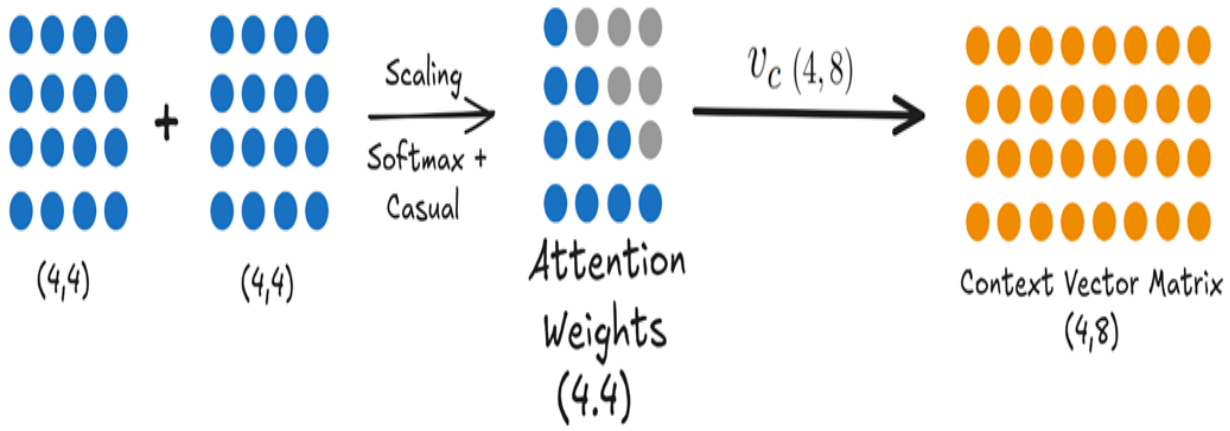
$$= Qc \cdot Kc^T + Qr \cdot Kr^T$$

As the mathematical identity in figure 3.30 shows, this is equivalent to calculating the attention scores for each path separately and then adding them together:

$$Attention\ Scores = (Qc * Kc^T) + (Qr * Kr^T)$$

This is the beauty of the decoupled design. We calculate the position-agnostic, cache-friendly content scores and add them to the position-dependent, RoPE-infused scores. Finally, these combined scores are used to create the attention weights, which are then multiplied by the content-only Value matrix to produce the final context vector.

**Figure 3.31 The final step. The combined attention scores are processed and multiplied by the content-based Value matrix (Vc) to produce the final output.**




This elegant, decoupled system successfully solves the incompatibility problem. It allows the absorption trick to be used on the content-heavy part of the calculation (the Qc * $Kc^T$ term) while still incorporating the full power of RoPE in a separate, specialized path. It truly achieves the best of both worlds.

### 3.14.3 The cost of decoupling: A trade-off between memory and computation

The Decoupled RoPE architecture is a brilliant solution, but it's not entirely "free." By creating a separate, parallel path for positional information, we have introduced new computations and new weight matrices. Let's analyze the costs and trade-offs of this design.

**The new computational cost**

In a standard MHA or even the "pure" MLA we designed, the core computation for attention scores is a single large matrix multiplication: Q * K.T.

In the decoupled architecture, we now perform two separate matrix multiplications that are added together:

1. `Qc @ Kc.T (the content scores)`
2. `Qr @ Kr.T (the positional scores)`

Furthermore, we've added new projection steps to generate `Qr` and `Kr` in the first place. This means that for each token, the model is performing more floating-point operations (FLOPs) than it would in a simpler architecture. This is a deliberate trade-off: **we are accepting a slight increase in computational cost to gain a massive reduction in memory cost.**

**Why is this a good trade-off?**

In modern, large-scale LLMs, the primary bottleneck during long-context inference is almost always **memory bandwidth**, not raw computation. The process of loading the massive KV cache from the GPU's high-bandwidth memory (HBM) into the much faster on-chip SRAM for computation is the slowest part of the process.

- **Standard MHA:** Has fewer computations but is severely bottlenecked by the time it takes to read its enormous 400 GB KV cache.
- **Decoupled MLA+RoPE:** Has slightly more computations but is much faster overall because it only needs to read a tiny ~7 GB cache.

The time saved by avoiding the memory bottleneck far outweighs the extra time spent on the additional matrix multiplications. DeepSeek made a calculated engineering decision to trade a resource they had in abundance (compute) to save a resource that was critically scarce (memory bandwidth).

**The new parameter cost**

The decoupled path also introduces new learnable weight matrices: Wqr and Wkr. This means the total parameter count of the model increases slightly. However, these matrices are relatively small compared to the enormous feed-forward networks in each Transformer block. The increase in the model's overall size is negligible, but it's a factor to consider.

In summary, decoupled architecture isn't magic, it's a masterful piece of engineering that makes a highly favorable trade-off. It strategically increases the amount of computation and the number of parameters by a small amount to solve the much larger, more critical problem of memory bandwidth, ultimately leading to a faster and more efficient model.

### 3.14.4 The new inference loop in action: What happens when a new token arrives

Now that we understand the complete decoupled architecture, let's trace the full, end-to-end journey of a single new token. This will solidify our understanding of how MLA and RoPE work together in an efficient, autoregressive loop.

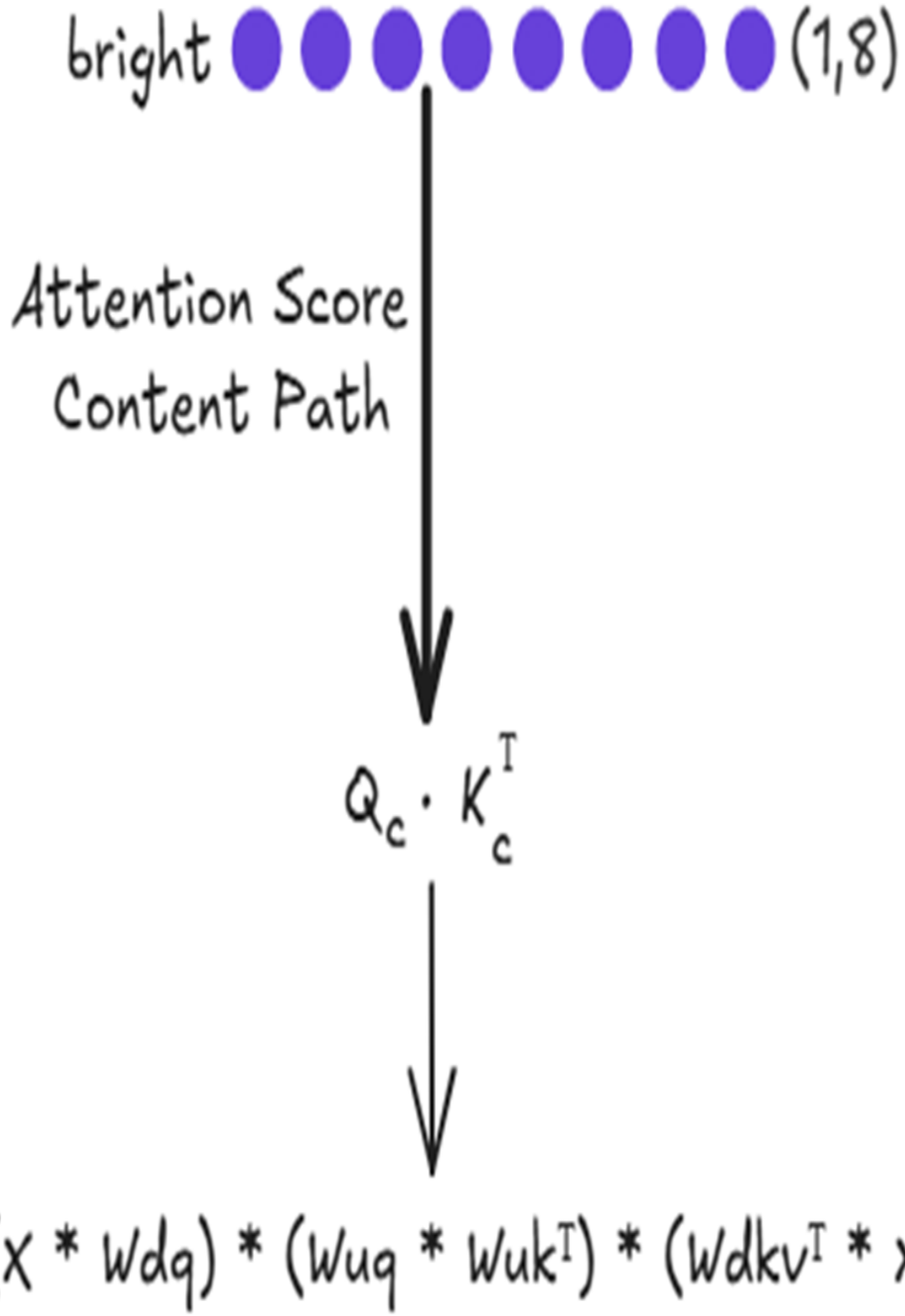
Imagine our model has already processed the sequence "`The next day is`" and has just generated the token "`bright.`" The input for this inference step is only the embedding for "`bright,`" which we'll call X_bright. Our goal is to use this single new token to predict the next one.

The process happens in two parallel paths the Content Path and the Position Path which are then combined.

#### The content path: Leveraging the absorption trick

This path is responsible for calculating the content-based attention scores. It uses the pure MLA mechanism, which is highly efficient thanks to the absorption trick and the latent KV cache.

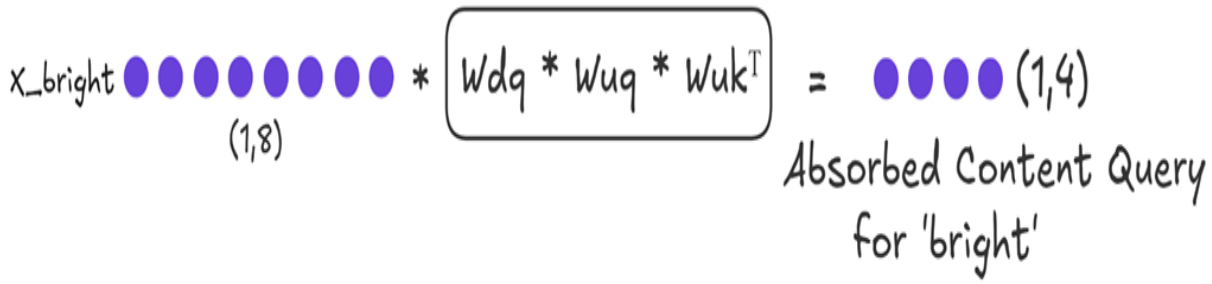
**Figure 3.32 The computational flow for the content-based attention score.**


bright
(1,8)
Attention Score
Content Path
$Q_c \cdot K_c^T$
$(X * Wdq) * (Wuq * Wuk^T) * (Wdkv^T * X^T)$

As shown in figure 3.32, the final attention score qc * $kc^T$ is calculated from a series of matrix multiplications. Let's break down how this works for our new token, "bright."

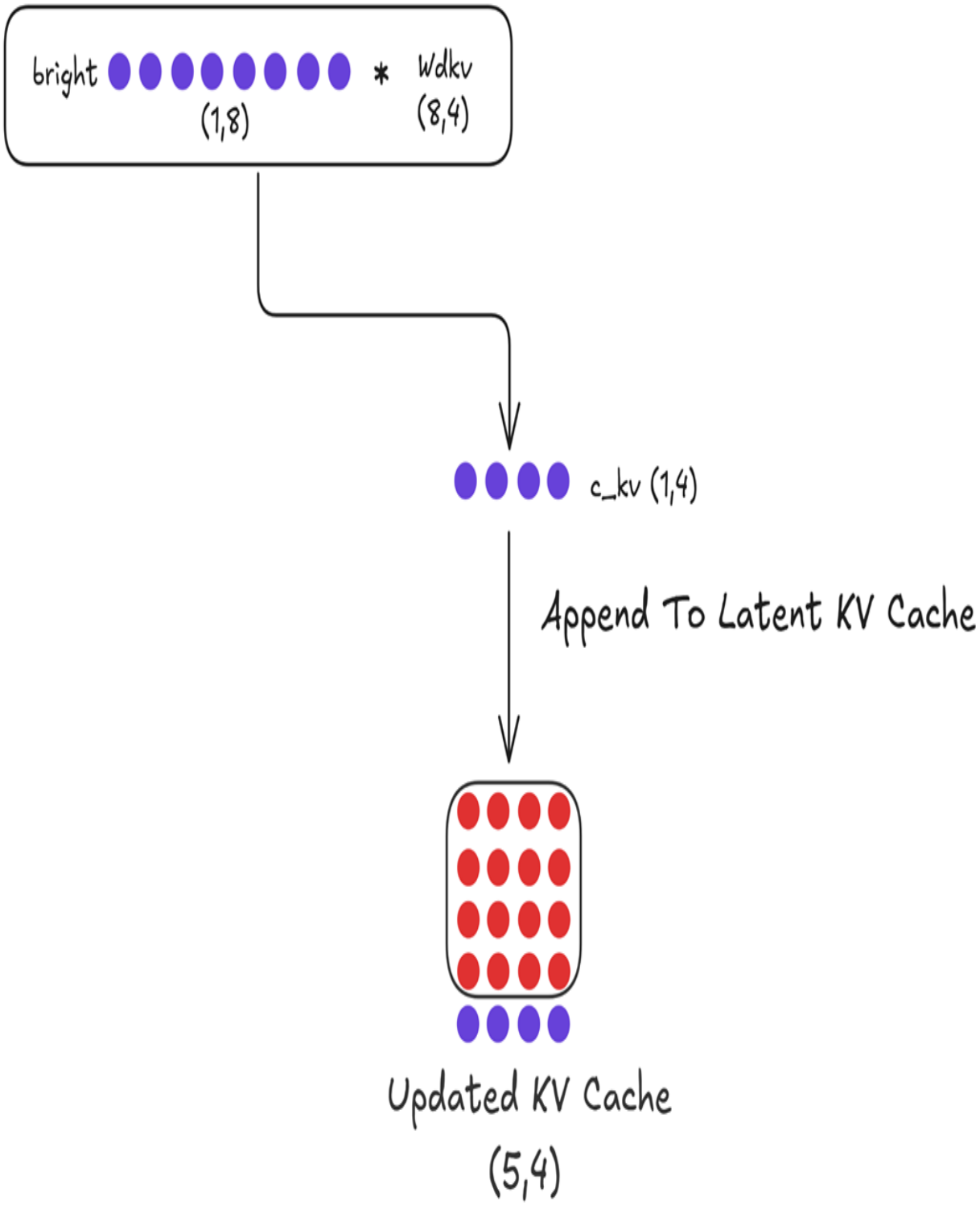
The first thing we need is the query for our new token. Because of the absorption trick, we don't just compute a standard query. Instead, we compute an "absorbed query" that already includes the Key up-projection matrix.

**Figure 3.33 This single multiplication gives us the query part we need for the content score.**




Next, we need to update our historical context. We take the input embedding for our new token, X_bright, and compute its compressed representation.

**Figure 3.34 Updating the Latent KV Cache. The new token's embedding is down-projected to create a new latent vector, which is then appended to the existing cache.**




As shown in figure 3.34:

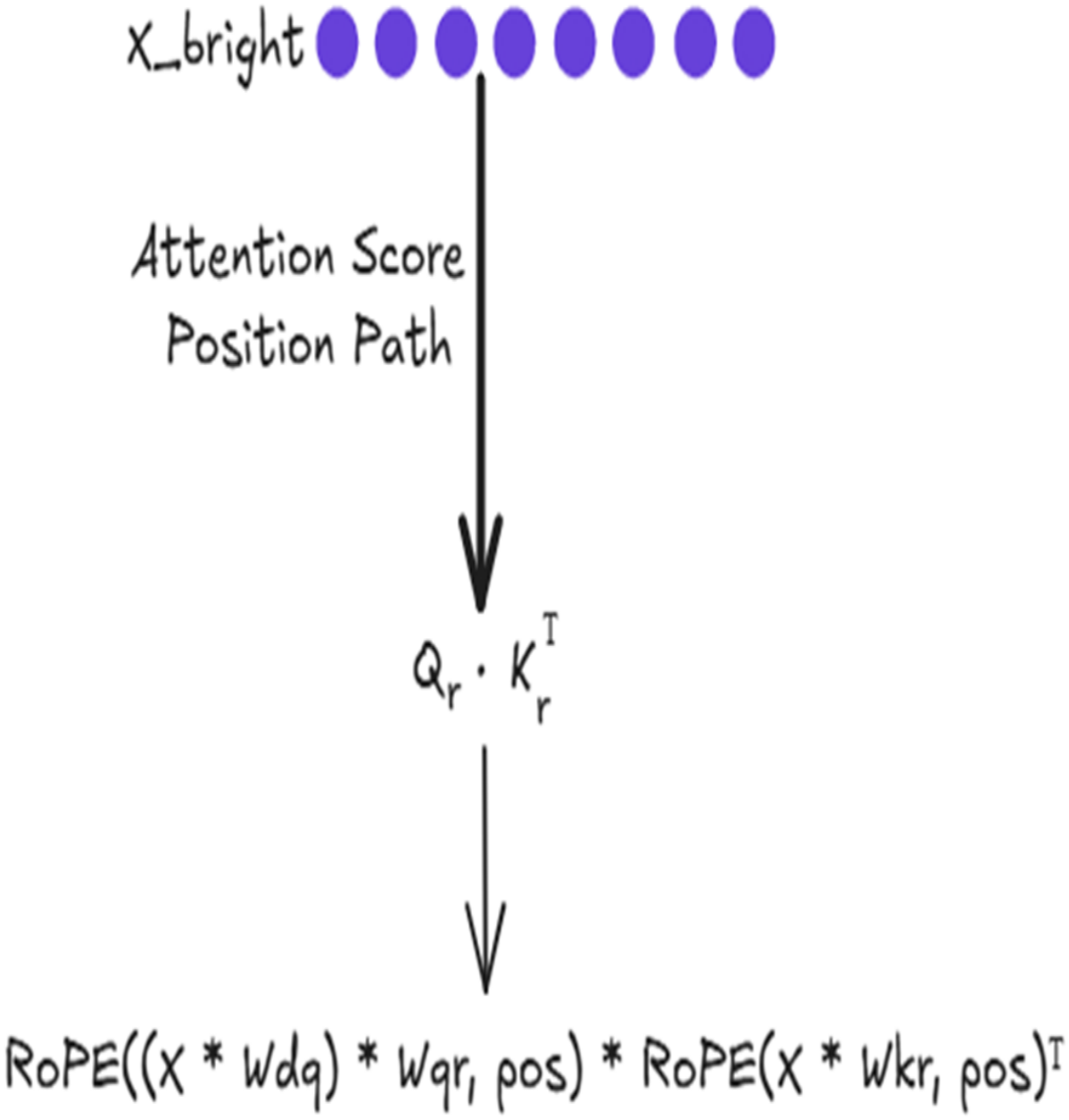
1. `X_bright` (shape 1, 8) is multiplied by the down-projector Wdkv (shape 8, 4).
2. This produces the new **latent vector** for "`bright`" (shape 1, 4).
3. This new vector is appended to the existing `Latent KV Cache` (which was shape 4, 4), resulting in an **Updated KV Cache** of shape (5, 4).

This updated cache is the only piece of historical information we need for the entire content path. Now we can calculate the content score. We multiply our "Absorbed Content Query" by the transpose of our `Updated KV Cache`. This gives us the final content-based attention scores for the token "`bright.`"

## The position path: Injecting rotational information

Running in parallel to the content path is the specialized path for handling positional information. Its goal is to compute the position-based attention scores using RoPE.

**Figure 3.35 The computational flow for the position-based attention score.**




As shown in figure 3.35, the final positional score qr * $kr^T$ is also the result of a series of transformations. You might recall that the position-dependent nature of RoPE was what broke the absorption trick in our initial attempt to combine it with MLA. So why is it not a problem here?

The answer lies in the decoupled design. The Position Path operates in parallel and does not rely on the cKV latent cache from the Content Path.

Since there is no "absorption trick" to break in this specialized path, we are free to apply RoPE as needed to infuse the Qr and Kr vectors with positional information

Let's break down how this works for our new token, "bright", which is at position 4 (assuming 0-indexing).

First, we need to create the query vector that will be encoded with positional information. This involves a down-projection and an up-projection, just like the content query, to save activation memory.

Next, we need the Key matrix for the positional path. This involves computing the new key for "bright" and appending it to the cached keys from previous tokens. With the fresh Qr and the updated Kr Cache, we can now compute the final positional attention scores by taking their dot product: $Qr * Kr\_cache^T$.

# 3.15 Quantifying the gains: The final cache memory comparison

The multi-step, decoupled process of the fused MLA-RoPE architecture is a masterpiece of engineering. But what does it achieve in practice? Let's quantify the two main benefits we set out to achieve: a dramatic reduction in cache size and the preservation of the model's expressive performance.

In the previous chapter, we established the formulas for the memory cost of MHA, MQA, and GQA. Now, we can define the final formula for our advanced MLA with Decoupled RoPE.

The crucial insight is that at each step, we only need to store two things in memory:

- **The Latent KV Cache**: This stores the compressed content information for all past tokens. Its dimension per token is `d_latent`.
- **The Decoupled Key Cache**: This stores the rotated positional information for all past tokens. Its dimension per token is `d_rope`.

Therefore, the new formula for the size of the cache is:

$$Size\_MLA + Size\ \ RoPE = l * b * (d\_latent + d\_rope) * s * 2$$

Let's break this down:

- l (layers), b (batch size), s (sequence length), and 2 (bytes per parameter) are the same as before.
- The `(d_latent + d_rope)` term replaces the `g * h` from GQA or the `n * h` from MHA.
- The * 2 factor that previously accounted for storing two separate matrices (one for Keys and one for Values) is gone. In this new architecture, we are caching specialized, combined representations (d_latent for content and d_rope for the positional key), not separate K and V matrices.

Let's plug in the numbers for a model at the scale of DeepSeek-V2, where n=128 and h=128:

- **MHA Cache Dimension:** 2 * 128 * 128 = 32,768
- **MLA+RoPE Cache Dimension** ( In the DeepSeek paper, d_latent is 4 * h = 512, and d_rope is h / 2 = 64. The total is 512 + 64 = 576.

The final reduction in the size of the cached data is: 32,768 / 576 ≈ 57 times.

This is an incredible result. By decoupling content and position and using a compressed latent cache, the MLA+RoPE architecture reduces the memory footprint of the KV cache by nearly two orders of magnitude compared to standard MHA. The theoretical 400 GB cache we calculated collapses to a much more manageable ~7 GB.

# 3.16 Building MLA + decoupled RoPE from scratch

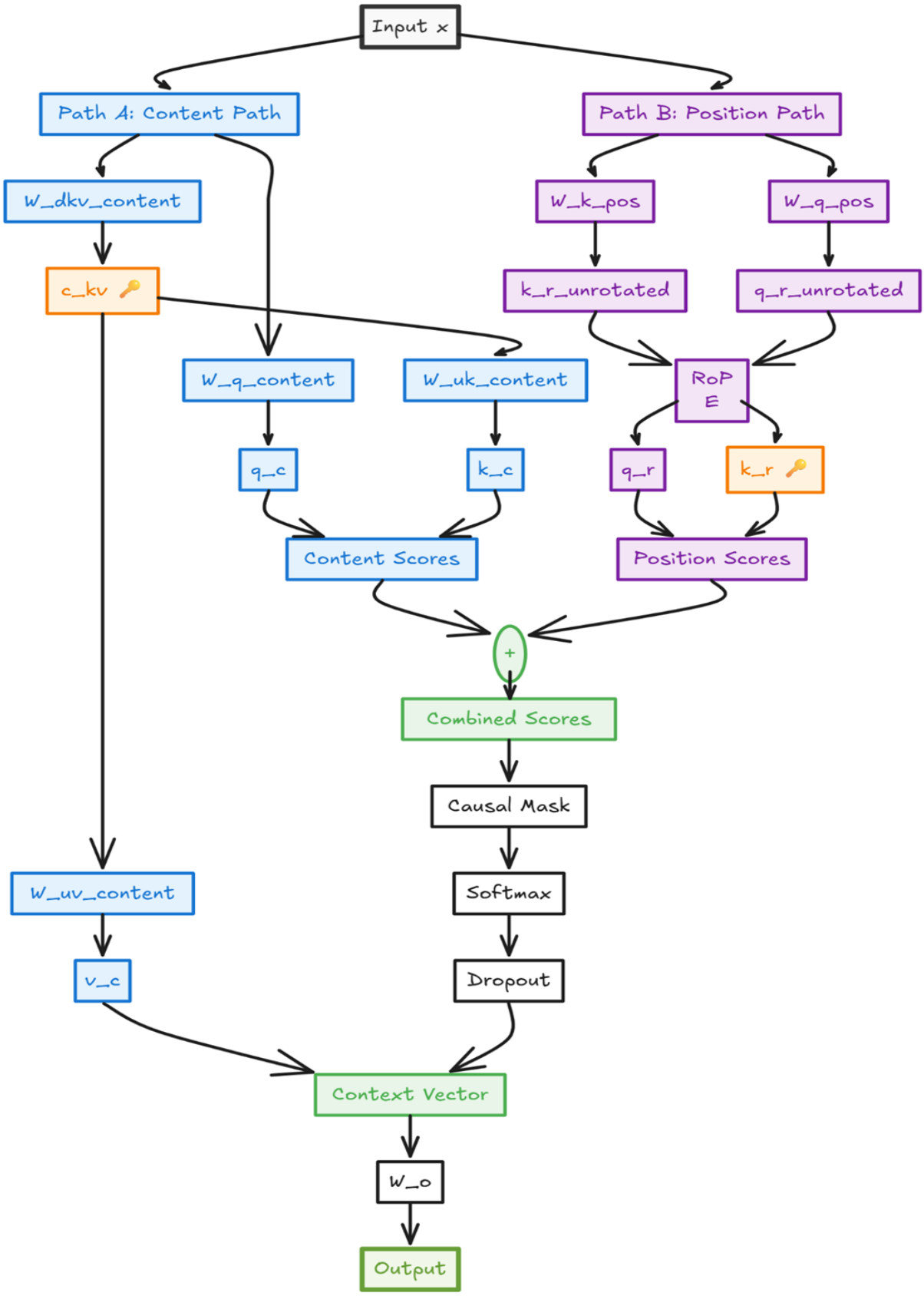
We have now covered all the theoretical and mathematical foundations of Multi-Head Latent Attention and Decoupled Rotary Position Embedding.

The final step is to translate this knowledge into a functional PyTorch module. The complete, executable code for this chapter can be found in the book's official GitHub repository. We highly recommend having it open to follow along with the implementation:

https://github.com/VizuaraAI/DeepSeek-From-Scratch/tree/main/ch03

Our implementation will construct the DeepSeekAttention module, which contains the entire decoupled architecture. Before we write the code, let's look at a complete visual blueprint of the module's forward pass in figure 3.36.

**Figure 3.36 A detailed flowchart of the DeepSeekAttention module. The input tensor x is processed through two parallel streams: the blue Content Path, which uses pure MLA for cache efficiency, and the purple Position Path, which applies RoPE. The two cached tensors, c_kv and k_r, are highlighted with key icons. The outputs are combined to produce the final context vector.**


Input x
Path A: Content Path
Path B: Position Path
W_dkv_content
W_k_pos
W_q_pos
c_kv
k_r_unrotated
q_r_unrotated
W_q_content
W_uk_content
RoPE
q_c
k_c
q_r
k_r
Content Scores
Position Scores
+
Combined Scores
Causal Mask
W_uv_content
Softmax
v_c
Dropout
Context Vector
W_o
Output

This diagram is a direct map of the code we are about to build. It clearly shows the two parallel pathways:

1. **Path A: The Content Path**: This stream is responsible for understanding the semantic content of the tokens. It uses the pure MLA architecture, creating the compressed latent tensor c_kv which is the only content-based value we need to cache. It also produces the content-based Query (q_c), Key (k_c), and Value (v_c) matrices.
2. **Path B: The Position Path**: This stream is dedicated to understanding the relative positions of tokens. It creates specialized Key (k_r_unrotated) and Query (q_r_unrotated) vectors, which are then infused with positional information by the RoPE module. The resulting rotated key, k_r, is the second value we cache.

The attention scores from these two paths are added together. The resulting attention weights are then applied to the content-only Value (v_c) to produce the final context vector.

Breaking down a complex architecture like this into smaller, manageable pieces is a key engineering skill. We will follow this principle by constructing our DeepSeekAttention module in stages, starting with the self-contained RoPE helper class.

## Part 1: The RotaryPositionalEncoding Helper Class

Before we can build the main attention module, we need a way to implement RoPE. It's a self-contained concept, making it the perfect candidate for our first code snippet. We will present the RotaryPositionalEncoding class in its entirety because it's a single, cohesive unit.

**Listing 3.2 The RotaryPositionalEncoding Helper Module**

```
import torch
import torch.nn as nn

class RotaryPositionalEncoding(nn.Module):
    def __init__(self, d_head, max_seq_len=2048):
        super().__init__()
        theta = 1.0 / (10000 ** (
        ➥ torch.arange(0, d_head, 2).float()
```

```
        ➥ / d_head))  #A
        self.register_buffer('theta', theta)

        positions = torch.arange(max_seq_len).unsqueeze(1)
        freqs = positions * self.theta.unsqueeze(0)

        self.register_buffer('freqs_cis',
        ➥ torch.polar(torch.ones_like(freqs),
        ➥ freqs))  #B

    def forward(self, x):
        seq_len = x.shape[2]
        x_complex = x.float().reshape(*x.shape[:-1], -1, 2)
        x_complex =
        ➥torch.view_as_complex(x_complex)  #C

        freqs_cis = self.freqs_cis[:seq_len, :].unsqueeze(0).
        ➥unsqueeze(0)

        x_rotated = x_complex * freqs_cis  #D

        x_rotated = torch.view_as_real(x_rotated)
        x_rotated = x_rotated.flatten(3)
        return x_rotated.type_as(x)
```

## Step 2: Initializing the Main Attention Class

With our RotaryPositionalEncoding module ready, we can now define the main DeepSeekAttention class. The __init__ method is responsible for creating all the learnable weight matrices (nn.Linear layers) and instantiating our RoPE helper.

Notice how the weight matrices are grouped into two distinct sections in the code, mirroring our decoupled architecture. One set is for the content path, which uses the standard MLA projections. The other set is for the position path, which creates the specialized Query and Key vectors that will be infused with positional information via RoPE.’

**Listing 3.3 Initializing the DeepSeekAttention Module**

```
class DeepSeekAttention(nn.Module):
    def __init__(self, d_model, num_heads, d_latent,
```

```
➥ d_rope, dropout=0.0):
    super().__init__()
    # ... (asserts and initializations) ...
    self.d_rope = d_rope

    # --- Content Path (Pure MLA) ---
    self.W_q_content = nn.Linear(d_model, d_model)
    self.W_dkv_content = nn.Linear(d_model,
    ➥ d_latent)  #A
    self.W_uk_content = nn.Linear(d_latent, d_model)
    self.W_uv_content = nn.Linear(d_latent, d_model)

    # --- Position Path (RoPE Applied) ---
    self.W_k_pos = nn.Linear(d_model,
    ➥ d_rope * num_heads)  #B
    self.W_q_pos = nn.Linear(d_model, d_rope * num_heads)
    self.rope =
    ➥RotaryPositionalEncoding(d_rope)  #C

    # --- Final Output Projection ---
    self.W_o = nn.Linear(d_model, d_model)
    # ... (dropout and mask) ...
```

## Step 3: The Forward Pass - Combining Content and Position

The forward method handles the entire decoupled attention calculation. It takes the input tensor x and passes it through both the content and position paths simultaneously to compute two separate attention score matrices. These are then added together to form the final scores, which are used to generate the context vector.

This implementation is a direct translation of the architecture we derived. It keeps the cache-friendly "absorption trick" viable for the content path while cleanly integrating the position-dependent rotations in a separate, parallel stream.

**Listing 3.4 The forward Method of DeepSeekAttention**

```
def forward(self, x):
    batch_size, seq_len, _ = x.shape

    # --- Content Path Calculation ---
    q_c = self.W_q_content(x).view(
```

```
➥          batch_size, seq_len, self.num_heads, self.d_head
➥      ).transpose(1, 2)
        c_kv = self.W_dkv_content(x)
        k_c = self.W_uk_content(c_kv).view(
➥          batch_size, seq_len, self.num_heads, self.d_head
➥      ).transpose(1, 2)
        v_c = self.W_uv_content(c_kv).view(
➥          batch_size, seq_len, self.num_heads, self.d_head
➥      ).transpose(1, 2)

        # --- Position Path Calculation ---
        q_r_unrotated = self.W_q_pos(x).view(
➥          batch_size, seq_len, self.num_heads, self.d_rope
➥      ).transpose(1, 2)
        k_r_unrotated = self.W_k_pos(x).view(
➥          batch_size, seq_len, self.num_heads, self.d_rope
➥      ).transpose(1, 2)

        q_r = self.rope(q_r_unrotated) #A
        k_r = self.rope(k_r_unrotated)

        # --- Combining Paths for Final Attention Score ---
        content_scores = (q_c @ k_c.transpose(-2, -1)) / \
➥          (self.d_head ** 0.5)
        position_scores = (q_r @ k_r.transpose(-2, -1)) / \
➥          (self.d_rope ** 0.5)

        attn_scores = content_scores +
        ➥position_scores  #B

        # --- Final Steps (Masking, Softmax, Output) ---
        attn_scores = attn_scores.masked_fill(
➥          self.mask[:, :, :seq_len, :seq_len], float('-inf'))

        attn_weights = torch.softmax(attn_scores, dim=-1)
        attn_weights = self.dropout(attn_weights)

        context_vector = (attn_weights @ v_c).transpose(1, 2) \
➥          .contiguous().view(batch_size, seq_len, self.d_model)

        output = self.W_o(context_vector)
        return output
)
```

With this final snippet, we have constructed a complete, from-scratch implementation of the DeepSeekAttention module. This single class encapsulates a series of advanced techniques:

- **A memory-efficient content pathway** using Multi-Head Latent Attention.
- **A separate, specialized position pathway** for handling relative positions.
- **Clean integration of Rotary Positional Encoding** where it is most effective.
- **A decoupled fusion** of these two pathways through simple addition.

This architecture masterfully resolves the inherent conflict between the position-agnostic caching of MLA and the position-dependent nature of RoPE. By trading a slight increase in computational cost for a massive reduction in memory bandwidth requirements, this design achieves the best of both worlds, enabling efficient and powerful long-context inference.

# 3.17 The Payoff: An Empirical Head-to-Head Comparison

Having explored the theoretical underpinnings of Multi-Head Latent Attention and its fusion with Decoupled RoPE, it's time to put our knowledge to the test. Theory is one thing, but empirical results provide the final verdict. The central promise of MLA is to break the painful trade-off between the high performance of MHA and the memory efficiency of its leaner cousins, MQA and GQA. Does it truly deliver the best of both worlds?

To answer this, we conducted a head-to-head comparison by building and training four models from scratch. Each model uses a different attention mechanism, but all other hyperparameters from model size and dataset to the number of training iterations are held constant to ensure a fair and direct comparison. This experiment will allow us to move beyond theory and quantify the real-world impact of DeepSeek's architectural innovations.

The code used to run this experiment and generate the following results is available for you to explore and replicate in the book's GitHub repository.

Bonus Code Link:

https://github.com/VizuaraAI/DeepSeek-From-Scratch/tree/main/ch03/02-bonus-code

To properly evaluate the trade-offs, we will train and benchmark four distinct attention architectures:

1. **Multi-Head Attention (MHA):** Our baseline for maximum performance. Each head has its own unique Query, Key, and Value projections, offering the highest expressive power at the cost of a large KV cache.
2. **Multi-Query Attention (MQA):** The first efficiency-focused variant. All heads share a single Key and Value projection, drastically reducing the KV cache size but potentially limiting the model's learning capacity.
3. **Grouped-Query Attention (GQA):** A compromise between MHA and MQA. Query heads are divided into groups, with each group sharing a common Key and Value projection. This offers a middle ground in both performance and efficiency.
4. **Multi-Head Latent Attention (MLA):** The DeepSeek innovation. As we've seen, it uses a compressed latent cache and decoupled RoPE to aim for the performance of MHA with the memory footprint closer to MQA/GQA.

Before we begin the training process, it's crucial to confirm that our models are of a comparable size. A significant difference in the total number of learnable parameters would make for an unfair comparison. The table below shows the parameter counts for each of our four model variants, demonstrating that they are all within a very similar range.

**Table 3.1 Parameters of model variants**

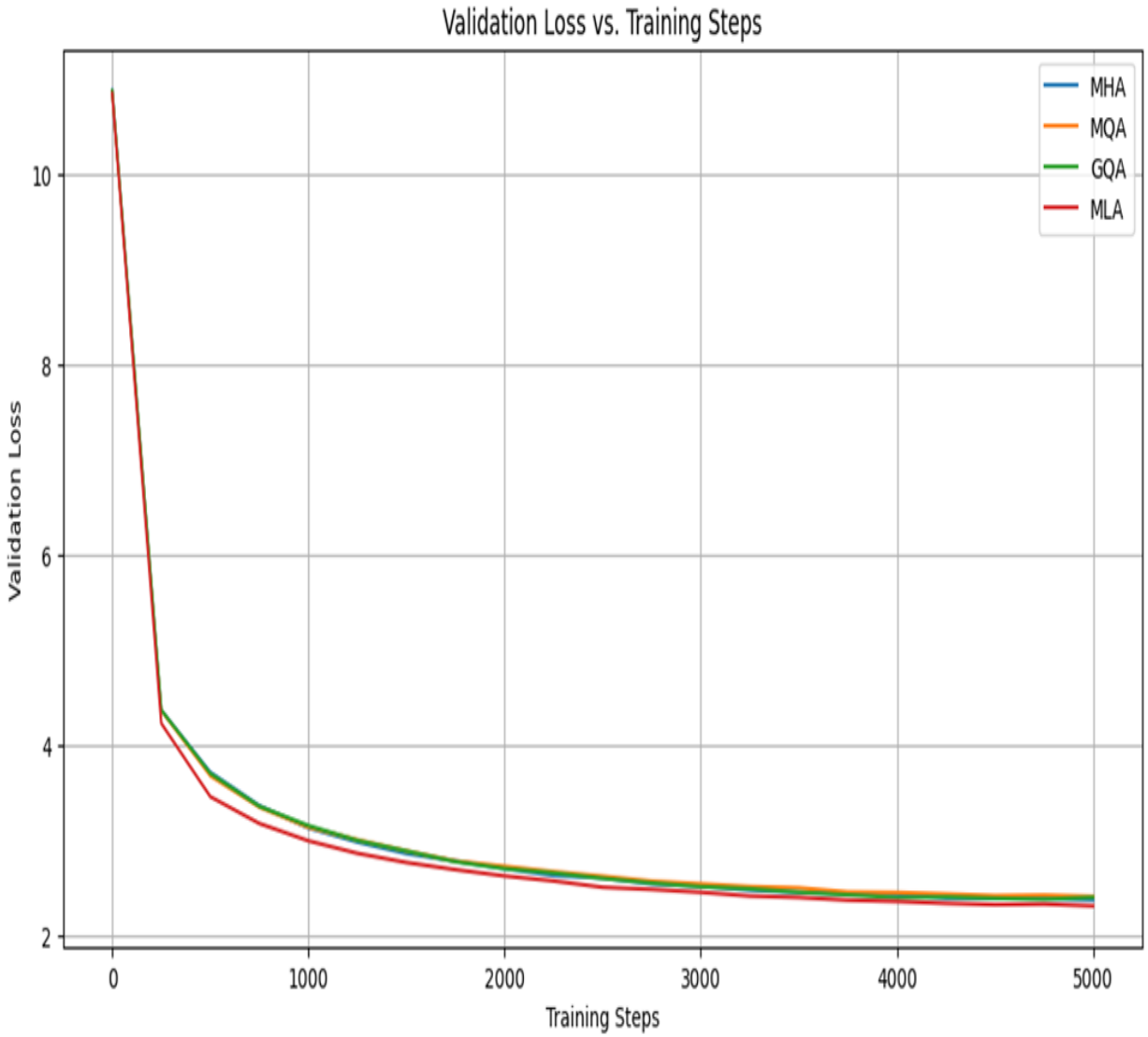
| Attention Type | Total Parameter |
|---|---|
| Multi-Head Attention (MHA) | 17,653,248 |

| | |
|---|---|
| Grouped-Query Attention (GQA) | 16,965,120 |
| Multi-Query Attention (MQA) | 17,260,032 |
| Multi-Head-Latent Attention (MLA) | 17,391,104 |

The total number of learnable parameters is nearly identical across all four architectures, ensuring a fair comparison of their performance and efficiency. The minor differences stem from the unique projection matrices required by each attention mechanism.

Our first objective is to answer the most critical question: does MLA's memory-saving design compromise its learning performance? A lower validation loss indicates better generalization to unseen data, which is our primary measure of a model's effectiveness. The plot in figure 3.37 tracks the validation loss for all four models over 5,000 training iterations.

**Figure 3.37 A comparison of the validation loss curves for the four attention mechanisms. The MHA and MLA models achieve a consistently lower final loss than their MQA and GQA counterparts, indicating superior learning. Both models were trained for 5,000 iterations.**




The learning curves reveal an idea. While all four models successfully learn to process the TinyStories dataset, their performance diverges. The MHA and MLA curves track each other closely, achieving the lowest validation loss and demonstrating the strongest learning capability. MQA, as expected, lags slightly behind due to its reduced expressivity, with GQA occupying the middle ground. This plot provides our first piece of powerful evidence: the MLA architecture does not sacrifice performance. It learns just as effectively as the full, resource-intensive MHA model, successfully preserving the expressive power of having unique, multi-headed projections.

**Table 3.2 Comparison of attention types**

| Attention Type | Best Validation Loss | KV Cache Memory Size (MB) | Best Validation Perplexity |
|---|---|---|---|
| MHA | 2.3721 | 48.00 | 10.72 |
| MQA | 2.4080 | 6.00 | 11.11 |
| GQA | 2.3776 | 24.00 | 10.78 |
| MLA | 2.3089 | 12.00 | 10.06 |

The results in table 3.2 are definitive. While the validation perplexity confirms what we saw in the learning curves—that MLA achieves the best performance (10.06), closely followed by MHA (10.72) the KV Cache column reveals the other half of the story. MLA dramatically reduces the memory footprint by 4x compared to MHA (12.00 MB vs. 48.00 MB), validating the core motivation behind the latent cache architecture.

This head-to-head experiment provides a clear verdict. By decoupling the content and position pathways and leveraging a compressed latent cache, the DeepSeek architecture successfully breaks the performance-memory trade-off. The result is an attention mechanism that is not only powerful but also remarkably efficient, providing a solid foundation for the scalable and intelligent models we aim to build. We will turn to the other key component of the Transformer layer in chapter 4. We will explore how Mixture-of-Experts (MoE) replaces the standard Feed-Forward Network to unlock another dimension of efficient scaling and we will also see Deepseek Innovation.

# 3.18 Summary

- **Multi-Head Latent Attention (MLA)** breaks the performance-memory trade-off by caching a compressed latent representation of Keys and Values, which is then decompressed on-the-fly to reconstruct unique, full-sized matrices for each head during the attention calculation.
- Applying positional information by adding it to the initial token embeddings pollutes their semantic meaning before they are processed

by the attention mechanism.

- **Rotary Positional Encoding (RoPE)** solves this by rotating the Query and Key vectors directly within the attention block, which injects positional information later in the process and preserves the vectors' original learned magnitudes.
- The efficiency of MLA depends on a mathematical "**absorption trick**" that requires position-agnostic weights, which is fundamentally incompatible with the position-dependent transformations of standard RoPE.
- The **Decoupled RoPE architecture** resolves this conflict by separating attention into two parallel streams: a content path that uses a cache-efficient MLA, and a position path that handles RoPE. The final attention scores are the **sum of the outputs from both paths.**

# 4 Mixture-of-Experts (MoE) in DeepSeek: Scaling intelligence efficiently

## This chapter covers

- Mixture of Experts (MoE) and how sparsity enables efficient scaling
- A hands-on, mathematical walkthrough of the MoE layer
- DeepSeek's advanced solutions for load balancing

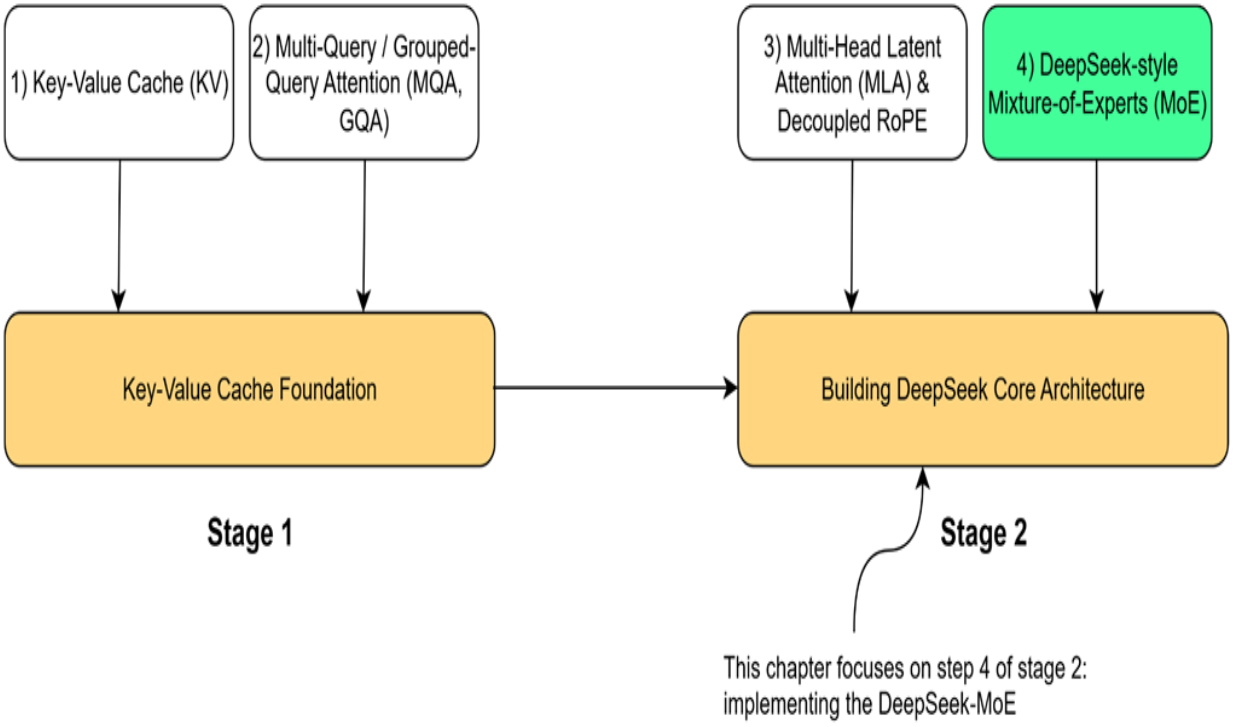
The idea of Mixture of Experts (MoE) is not new; its roots trace back to a seminal 1991 paper on adaptive expert systems. However, its application to large-scale language models is a more recent development, and one that DeepSeek has pushed to its notable limits. While other models like Mistral's Mixtral brought MoE into the mainstream for LLMs, DeepSeek built upon this foundation, introducing novel tricks and techniques of its own.

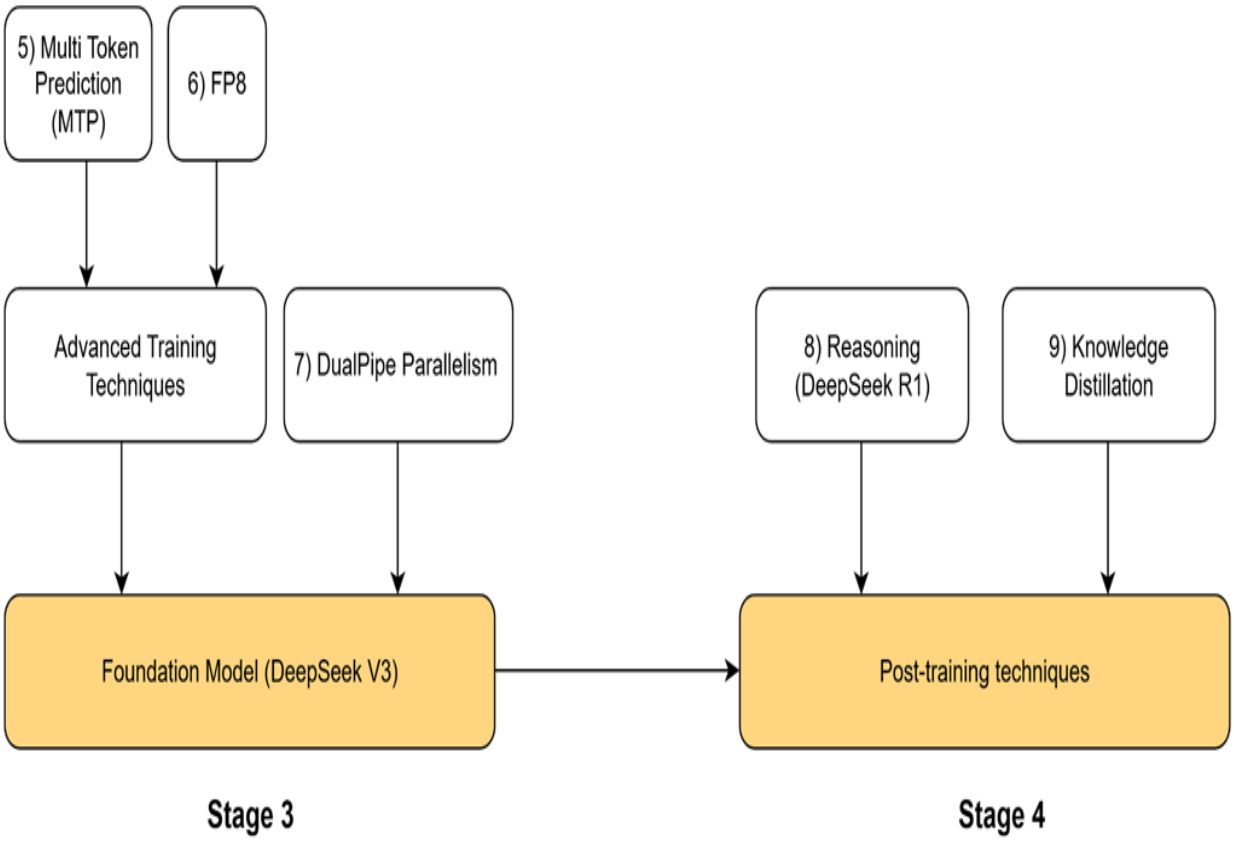
Now let‚Äôs open the black box of this mechanism. As illustrated in figure 4.1, our roadmap will cover:

1. The core intuition behind MoE and the concept of sparsity.
2. A detailed, mathematical, hands-on demonstration of how the MoE mechanism is implemented.
3. An exploration of the critical challenge of "load balancing" and the standard solutions.
4. A deep dive into the specific innovations DeepSeek introduced in their MoE architecture, from shared experts to their auxiliary-loss-free balancing.
5. Finally, we will put it all
6. together by coding a complete, functional MoE language model from scratch.

**Figure 4.1 Our four-stage journey to build the DeepSeek model. This chapter focuses on the highlighted component, DeepSeek-style Mixture-of-Experts (MoE), the second major innovation**

**in the core architecture.**


1) Key-Value Cache (KV)
2) Multi-Query / Grouped-Query Attention (MQA, GQA)
3) Multi-Head Latent Attention (MLA) & Decoupled RoPE
4) DeepSeek-style Mixture-of-Experts (MoE)
Key-Value Cache Foundation
Building DeepSeek Core Architecture
Stage 1
Stage 2
This chapter focuses on step 4 of stage 2: implementing the DeepSeek-MoE


5) Multi Token Prediction (MTP)
6) FP8
Advanced Training Techniques
7) DualPipe Parallelism
8) Reasoning (DeepSeek R1)
9) Knowledge Distillation
Foundation Model (DeepSeek V3)
Post-training techniques
Stage 3
Stage 4

Let's begin by understanding how MoE fits into the Transformer architecture and the core idea that makes it so powerful.

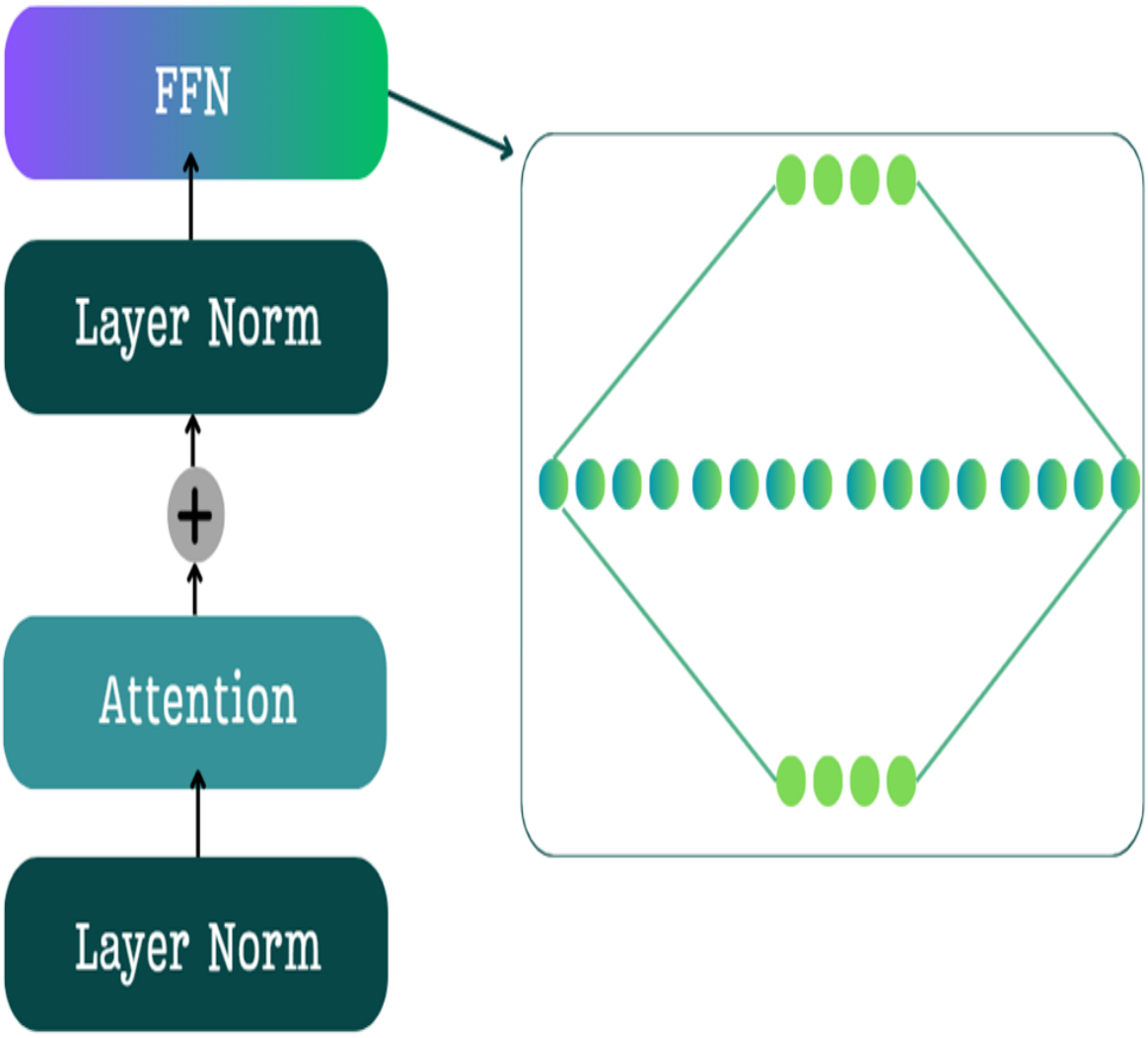
# 4.1 The intuition behind mixture of experts

To understand Mixture of Experts, we must first look at the component it replaces within the standard Transformer block: the Feed-Forward Network (FFN). The FFN acts as the primary processing unit within each Transformer layer, a dense neural network that accounts for the majority of the model's parameters and its computational workload. This change marks a genuine revolution in design. Instead of relying on one massive, all-purpose neural network, we now substitute it with a diverse number of smaller, specialized neural networks each an ‚Äúexpert‚Äù in its own right, MoE allows models to grow far larger and more knowledgeable without a proportional increase in computational cost.

## 4.1.1 The problem with dense FFNs in transformer: High parameter count and computational cost

As we know following the multi-head attention layer and layer norm, the resulting context vectors are processed by a Feed-Forward Network (FFN), the architecture of which is depicted in figure 4.2. The FFN consists of a two-layer neural network that implements an "expansion-contraction" sequence. As figure 4.2 illustrates, the first linear layer expands the embedding dimension (e.g., by a factor of 4), while the second linear layer contracts it back to its original size. This design is critical for the model's performance, as the expanded intermediate layer provides a richer, higher-dimensional space for the model to capture complex patterns before the output is passed to the subsequent block.

**Figure 4.2 The standard Feed-Forward Network (FFN) in a Transformer block, featuring an expansion-contraction architecture.**



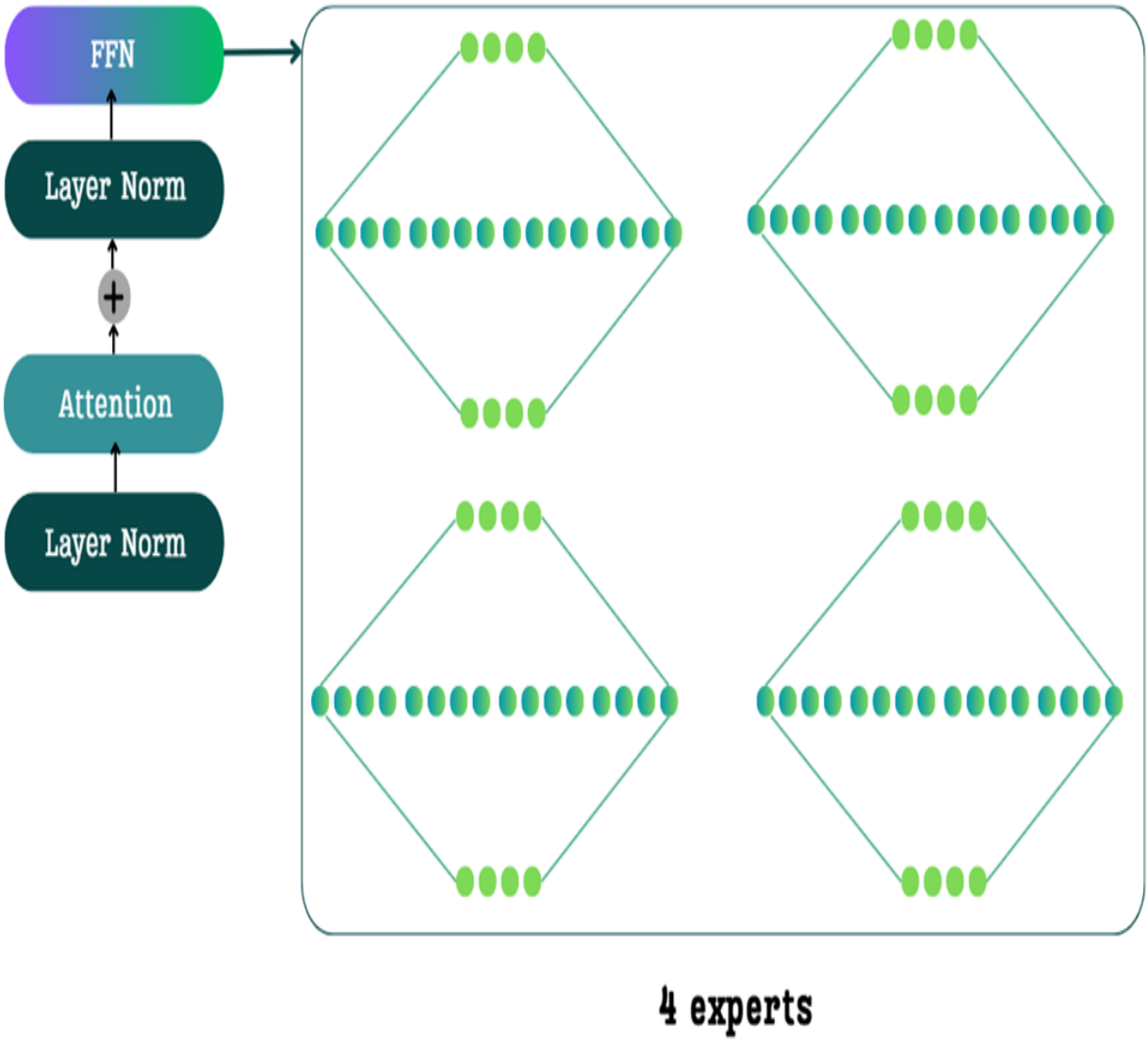

However, this FFN is dense. This means that for every single input token, all of the millions of parameters in the FFN are activated and used in the computation. This has two major consequences:

1. **High Training Cost**: The large number of parameters in the FFNs across all Transformer blocks accounts for a significant portion of the model's total training time.
2. **High Inference Cost**: Similarly, during inference, these dense computations contribute significantly to the latency of generating each new token.

### 4.1.2 The sparsity solution: Activating only a subset of experts per token

The core idea of Mixture of Experts is to replace the single, large, dense FFN with a collection of multiple, smaller FFNs, which we call "experts." The traditional FFN is a single, dense network. In an MoE architecture, this single block is replaced by a set of parallel expert networks, as shown in figure 4.3.

**Figure 4.3 The architectural change of MoE. The single, dense FFN is replaced by a collection of four smaller, specialized expert networks.**



You might think that replacing one network with four would quadruple the computational cost, but this is where the magic of MoE comes in, driven by a single concept: **sparsity.**

**Sparsity**: In an MoE model, for any given input token, only a small subset of the total experts is activated. The rest remain dormant or inactive.

For example, in the 4-expert model shown in figure 4.3, a token might only be routed to one or two of them. The other experts are not used for that specific token, meaning their parameters are not loaded and their computations are not performed.

This is the source of MoE's incredible efficiency. We get the benefit of having a huge number of total parameters (the sum of all experts), but the computational cost for any single token is very low, as we only activate a small fraction of them. This allows us to build models that are vastly more knowledgeable (more total parameters) without a proportional increase in the cost of training or inference.

### 4.1.3 Expert specialization: The "why" behind sparsity

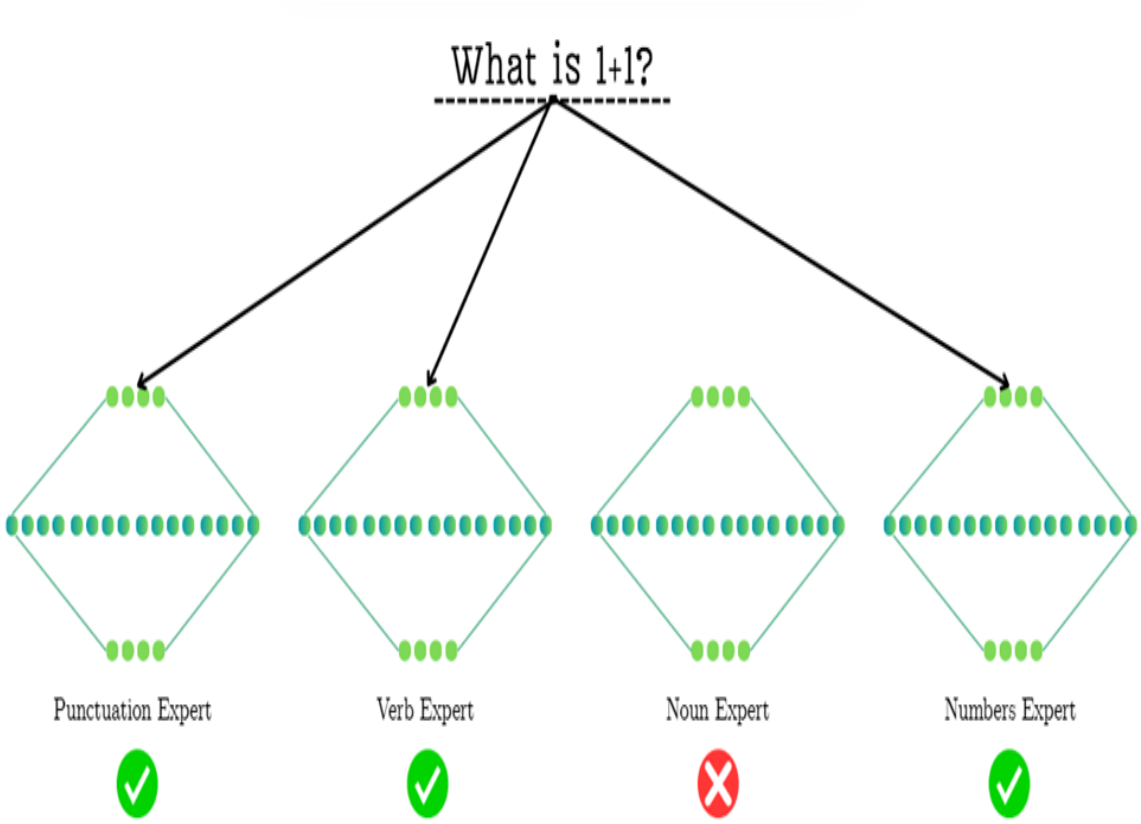
The concept of sparsity activating only a few experts per token is what makes MoE efficient. But why does this work? Why can we get away with ignoring most of the experts? The answer lies in **expert specialization.**

During the massive pre-training process, each expert network learns to become highly specialized in handling specific types of information or performing particular tasks. Instead of one giant, generalist FFN that must know how to do everything, we have a committee of specialists.

This specialization was proven quantitatively in the 2022 paper "ST-MoE: Designing Stable and Transferable Sparse Expert Models," (https://arxiv.org/pdf/2202.08906) which analyzed what different experts in a trained model had learned to do. The results were fascinating and provided a clear window into the mind of an MoE model.

**Figure 4.4 An example of expert routing in a Mixture-of-Experts model. For the input "What is 1+1?", the router must decide which specialized experts to activate. The routing mechanism**

**might prioritize grammatical components (like the question mark and the verb "is"), highlighting how routing is a nuanced decision based on learned patterns.**




To make this concrete, let's trace how the model might process the simple question "What is 1+1?", as illustrated in figure 4.4. The routing network must look at the incoming tokens and select the most appropriate specialists from its available committee:

- **Punctuation Experts:** The router first encounters the question mark (?). Having learned to recognize punctuation, it activates the **Punctuation Expert.** This expert is highly specialized in understanding the role of punctuation marks and how they influence the meaning and structure of a sequence.
- **Verb Experts:** The token is a common verb. The router, recognizing this, sends the token to the **Verb Expert.** This specialist has learned the grammatical and semantic patterns associated with actions and states of being, and it is best equipped to handle this part of the input.

- **Dormant Experts** (e.g., Noun Experts): Notice that the **Noun Expert** is not activated. Since the query contains no significant nouns, the router intelligently saves computation by keeping this expert dormant. The same would apply to a **Proper Noun Expert;** since there are no names like "Martin" or "DeepSeek," that specialist is not needed.
- **Domain-Specific Experts:** This is where the process becomes even more granular. You might expect the **Numbers Expert** to be the most critical for handling "1+1". However, as the diagram shows, the router might have learned that the signals from the question mark and the verb are stronger or more immediately recognizable. In this case, it prioritizes the grammatical experts, leaving the mathematical ones dormant. This highlights the complex and sometimes non-obvious decisions the routing mechanism makes, activating only the experts it deems most relevant for a given token based on its training.

This is the beauty of the MoE design. When an input token arrives, the model doesn't need to consult its entire knowledge base. It uses a small, efficient **routing network** (which we will build later) to ask, "Who is the specialist expert for this type of token?" It then sends the token only to the relevant experts.

A token representing a comma does not need to activate the expert that specializes in mathematical verbs. By routing the token only to the punctuation expert, the model saves an enormous amount of computation.

## 4.2 The mechanics of MoE: A hands-on mathematical walkthrough

Now that we have the core intuition behind Mixture of Experts, it's time to open the black box and see how it is actually implemented. We will trace the journey of a batch of tokens step-by-step, from the input that enters the MoE block to the final output it produces.

To make the mathematics as clear as possible, we will switch from the previous conceptual example to a new, standard input sequence that is easier to visualize in matrix form. For the remainder of this walkthrough, we will use the input "The next day is." Our goal is to understand how the model

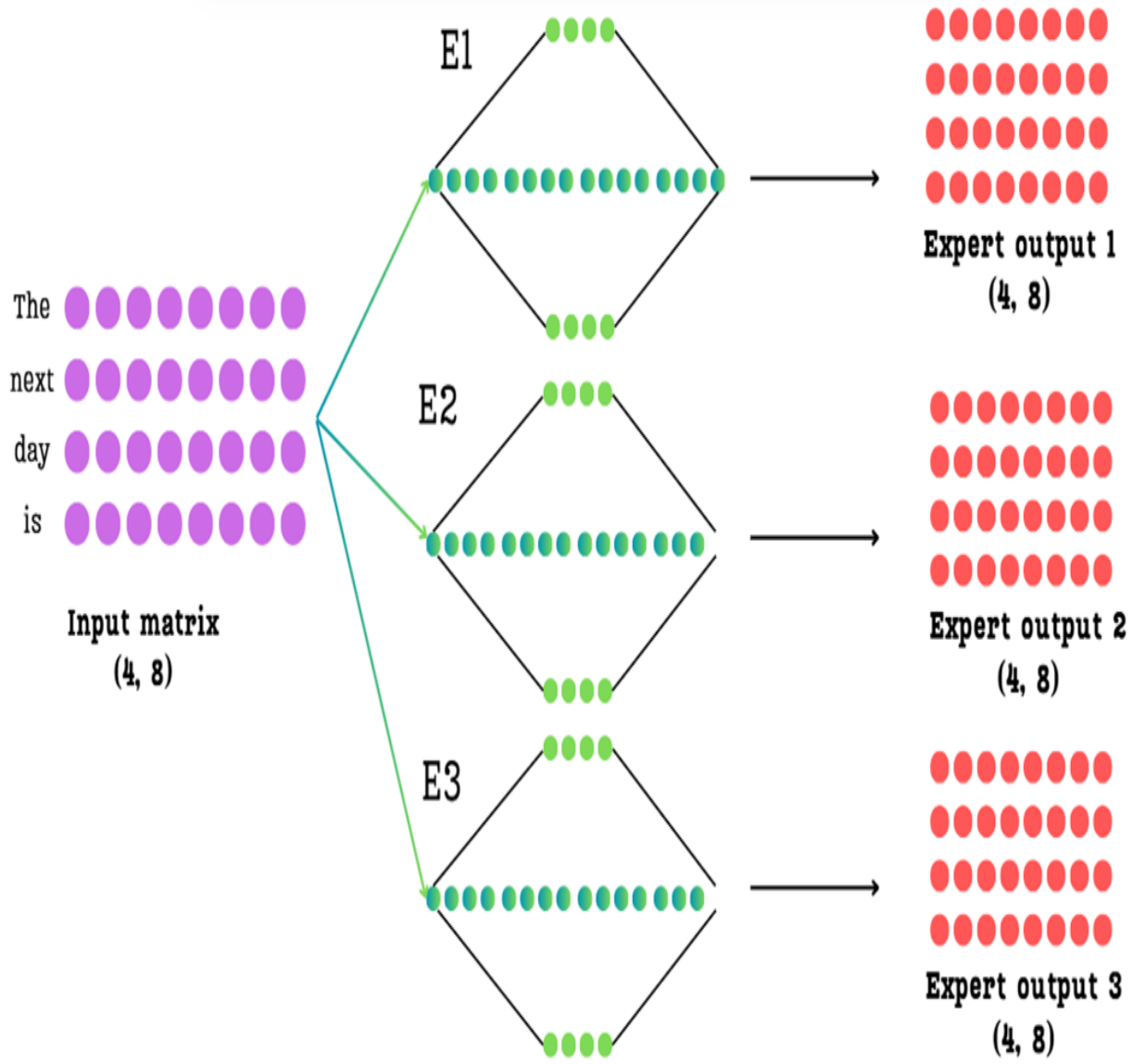
uses sparsity and routing to efficiently combine the knowledge of multiple experts for this sequence.

### 4.2.1 The goal: Combining multiple expert outputs into one

Let's start by defining our setup. We have an **input matrix** of shape (4, 8), representing four tokens (`"The"`, `"next"`, `"day"`, `"is"`), each with an 8-dimensional embedding. This matrix is the output from the preceding attention layer in the Transformer block.

Let's assume for our simplified example that our MoE layer contains three separate expert networks (E1, E2, E3), though in a real model this number would be much larger. As we know, each expert is a full Feed-Forward Network. If we were to pass our input matrix through each of these experts, we would get three separate output matrices.

**Figure 4.5 The initial challenge of MoE. The input matrix is passed through each of the three expert networks in parallel, resulting in three separate expert output matrices.**




As figure 4.5 shows, this leaves us with a challenge. We start with one (4, 8) input matrix, but we end up with three (4, 8) output matrices. The Transformer architecture, however, expects only a single matrix to pass to the next layer.

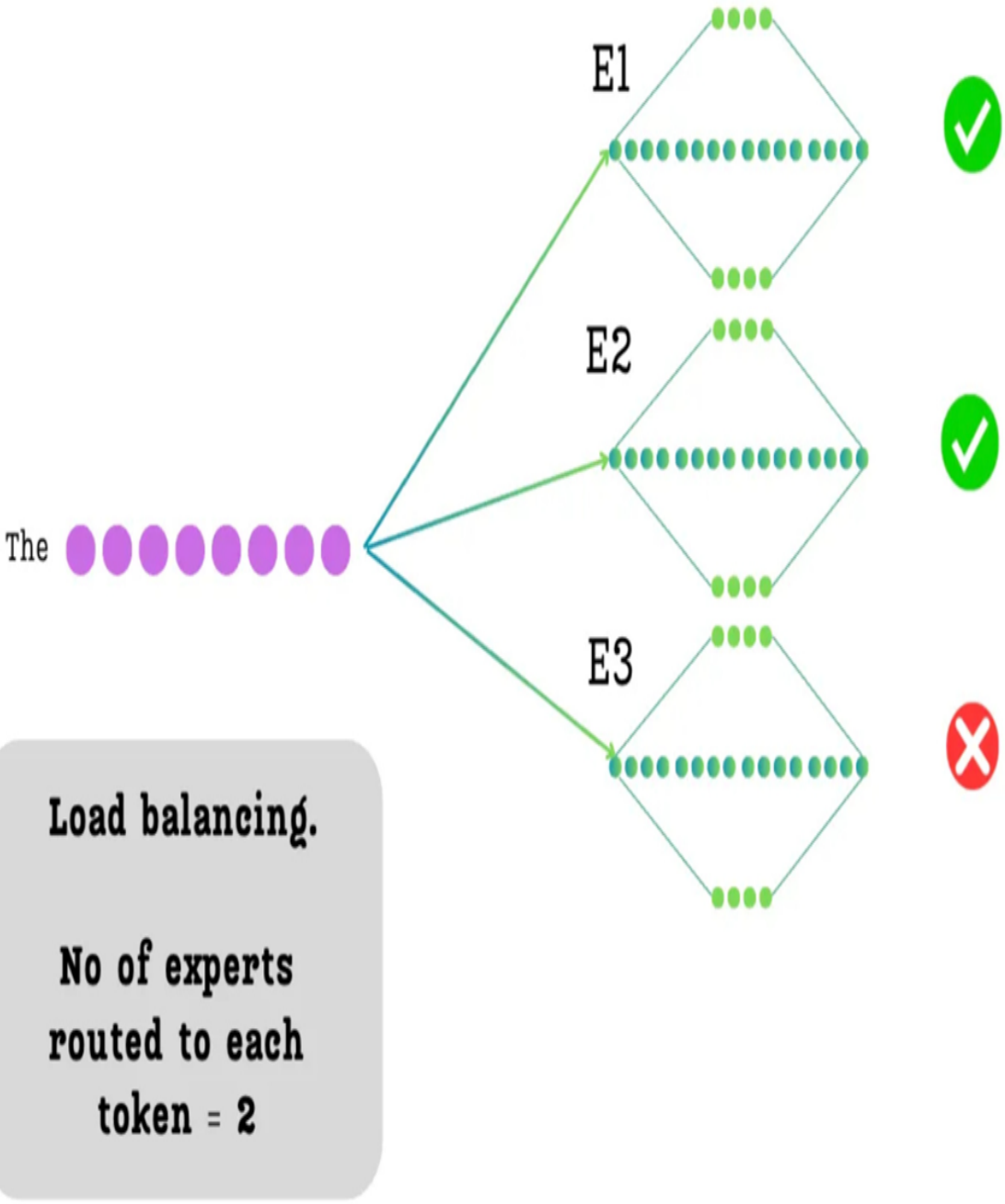
Our entire task in this section is to figure out: **How do we intelligently combine these three expert outputs into a single, final output matrix of the same (4, 8) shape?** The answer lies in two key concepts we introduced intuitively back in **Section 4.1**: sparsity and routing.

### 4.2.2 Sparsity in action: Top-K selection for load balancing

The first step in processing tokens with a Mixture of Experts (MoE) model is selecting which experts to use for each token before merging their outputs. As we've discussed, the core efficiency of MoE comes from sparsity: not every token is processed by every expert.

We enforce this by making a simple but powerful decision: for each token, we will only select the top-k most relevant experts. The value of k is a hyperparameter we choose. For our example, we will set k=2. This means that out of our three available experts, only two will be activated for any given token.

**Figure 4.6 The principle of sparsity or load balancing. For each token, we decide to route it to only a subset (k=2) of the available experts.**




As illustrated in figure 4.6, when the token "The" is processed, it will only be sent to two of the three experts (in this example, E1 and E2 are selected, while E3 is ignored). This act of selective activation is what prevents the model from having to perform the full computation of all experts for every token.

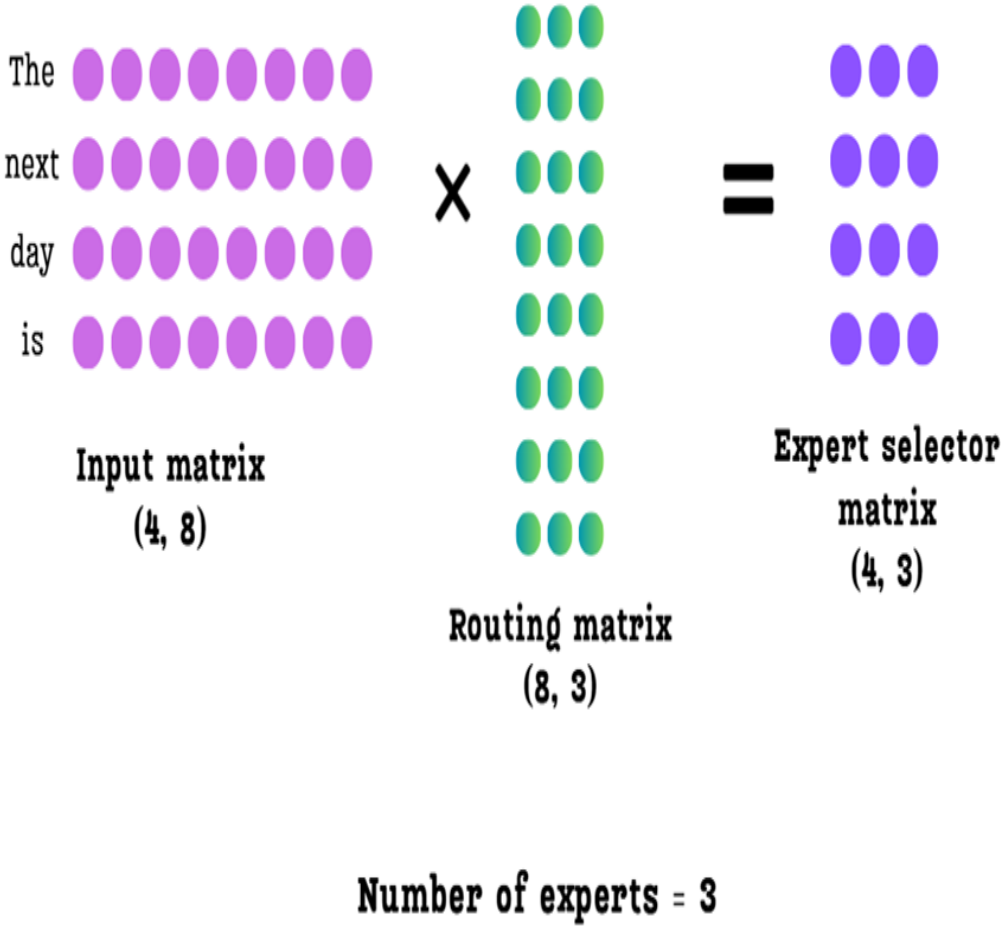
This immediately raises the next logical question: how does the model decide which two experts to select? And once selected, how does it know how much importance, or **weightage**, to give to each of them? This is the job of the routing mechanism.

### 4.2.3 The routing mechanism: From input to expert scores

To decide which experts are best suited for each token, the model uses a small, learnable neural network called a **router**. The router's job is to look at the input tokens and produce a score for each expert, for each token.

This is implemented as a simple linear layer. We take our `Input Matrix` and multiply it by a trainable `Routing Matrix`.

**Figure 4.7 The routing mechanism. The input matrix is multiplied by a learned routing matrix to produce an expert selector matrix, which contains a raw score for each expert for each token.**




Let's break down the dimensions shown in figure 4.7:

- **Input Matrix**: Shape (4, 8) - four tokens, each with an 8-dimensional embedding.
- **Routing Matrix**: Shape (8, 3) - the input dimension must match the input matrix (8), and the output dimension is the number of experts we have (3). This is a learned weight matrix.
- **Expert Selector Matrix**: Shape (4, 3) - the result of the multiplication.

This Expert Selector Matrix (also often called the logits matrix) is the crucial output of the router. Let's interpret its structure:

- Each **row** corresponds to one of our input tokens (`"The", "next", "day", "is"`).
- Each **column** corresponds to one of our experts (`E1, E2, E3`).
- The value at each position (`row, column`) is a raw, unnormalized score indicating how suitable that expert is for that token.

For example, the value in the first row, second column is the score for routing the token "`The`" to `Expert 2`. Now that we have these scores, we can use them to implement our top-k selection.

## 4.2.4 From scores to weights: Top-K selection and softmax normalization

We now have our `Expert Selector Matrix`, which contains the raw scores. Our next task is to use these scores to achieve two goals:

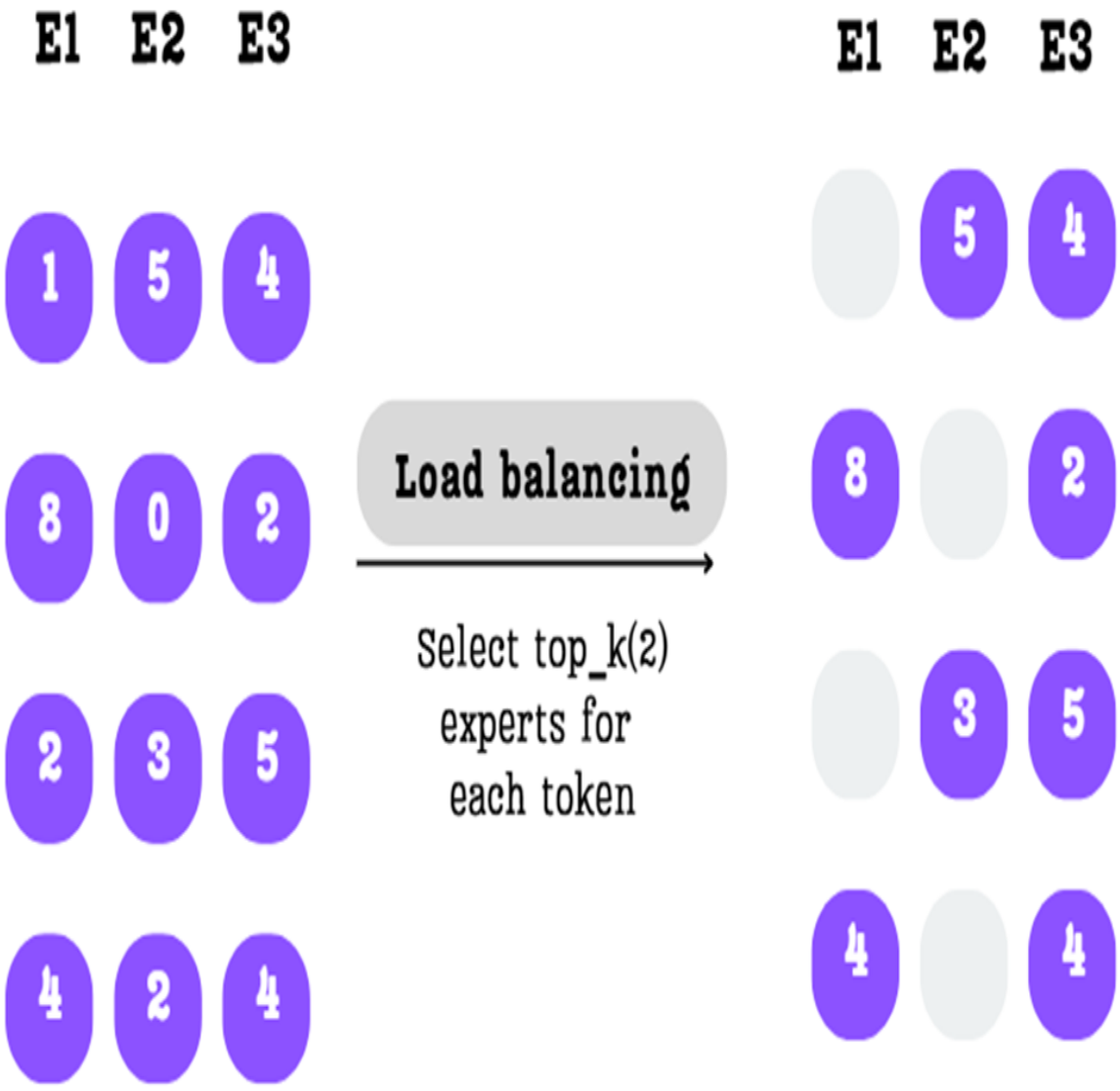
3. Select only the top 2 experts for each token.
4. Convert the scores for those selected experts into a set of weights that sum to 1.

This is a three-step process.

### Step A: Select the Top-K Experts

First, for each row (each token), we identify the `k=2` experts with the highest scores. The scores for all other experts are discarded.

**Figure 4.8 The top-k selection process. For each row, only the two highest scores are kept, and the rest are masked out.**

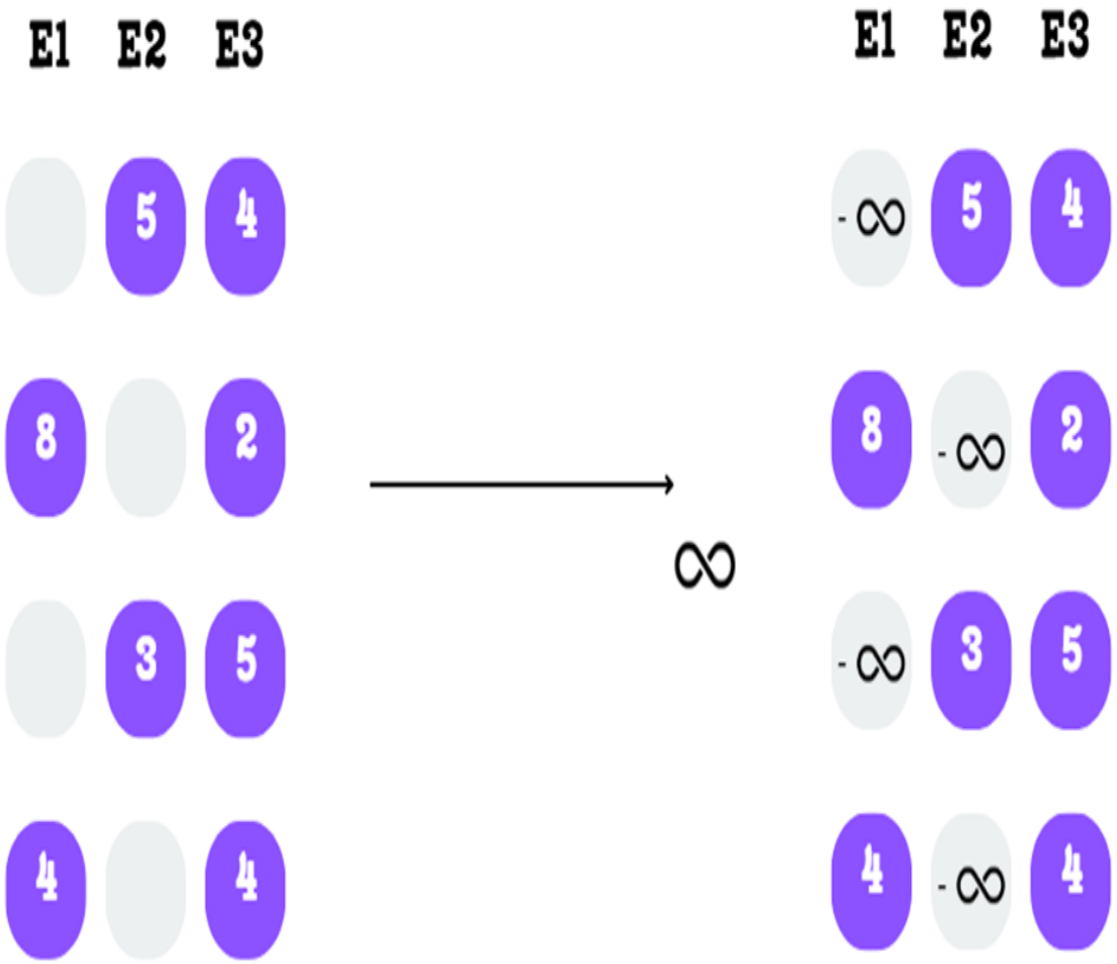



As shown in figure 4.8, for the first token, the scores 5 and 4 (for E2 and E3) are the highest, so the score 1 (for E1) is masked out. This is repeated for every token, fulfilling our sparsity requirement.

**Step B: Masking with Negative Infinity**

To mathematically eliminate the non-selected experts, we replace their scores with negative infinity. This might seem like a strange choice, but it's a clever trick that works perfectly with the softmax function in the next step.

**Figure 4.9 The masked scores are replaced with negative infinity in preparation for the softmax function.**




## Step C: Applying Softmax

Finally, we apply the **softmax function** to each row of this new matrix. The softmax function has two properties that are perfect for our needs:

1. It converts any real numbers into a probability distribution where all values are between 0 and 1, and the sum of the values in each row is exactly 1.
2.